Mathematical Programming and Operations Research

Modeling, Algorithms, and Complexity

Examples in Excel and Python

(Work in progress)

Edited by: Robert Hildebrand

Contributors: Robert Hildebrand, Laurent Poirrier, Douglas Bish, Diego Moran

Version Compilation date: July 21, 2025

Preface

This entire book is a working manuscript. The first draft of the book is yet to be completed.

This book is being written and compiled using a number of open source materials. We will strive to properly cite all resources used and give references on where to find these resources. Although the material used in this book comes from a variety of licences, everything used here will be CC-BY-SA 4.0 compatible, and hence, the entire book will fall under a CC-BY-SA 4.0 license.

MAJOR ACKNOWLEDGEMENTS

I would like to acknowledge that substantial parts of this book were borrowed under a CC-BY-SA license. These substantial pieces include:

- "A First Course in Linear Algebra" by Lyryx Learning (based on original text by Ken Kuttler). A majority of their formatting was used along with selected sections that make up the appendix sections on linear algebra. We are extremely grateful to Lyryx for sharing their files with us. They do an amazing job compiling their books and the templates and formatting that we have borrowed here clearly took a lot of work to set up. Thank you for sharing all of this material to make structuring and formating this book much easier! See subsequent page for list of contributors.
- "Foundations of Applied Mathmatics" with many contributors. See `https://github.com/Foundations-of-Applied-Mathematics`. Several sections from these notes were used along with some formatting. Some of this content has been edited or rearranged to suit the needs of this book. This content comes with some great references to code and nice formatting to present code within the book. See subsequent page with list of contributors.
- "Linear Inequalities and Linear Programming" by Kevin Cheung. See `https://github.com/dataopt/lineqlpbook`. These notes are posted on GitHub in a ".Rmd" format for nice reading online. This content was converted to LaTeXusing Pandoc. These notes make up a substantial section of the Linear Programming part of this book.
- Linear Programming notes by Douglas Bish. These notes also make up a substantial section of the Linear Programming part of this book.

I would also like to acknowledge Laurent Porrier and Diego Moran for contributing various notes on linear and integer programming.

I would also like to thank Jamie Fravel for helping to edit this book and for contributing chapters, examples, and code.

Contents

Introduction

Letter to instructors

This is an introductory book for students to learn optimization theory, tools, and applications. The two main goals are (1) students are able to apply the of optimization in their future work or research (2) students understand the concepts underlying optimization solver and use this knowledge to use solvers effectively.

This book was based on a sequence of course at Virginia Tech in the Industrial and Systems Engineering Department. The courses are *Deterministic Operations Research I* and *Deterministic Operations Reserach II*. The first course focuses on linear programming, while the second course covers integer programming an nonlinear programming. As such, the content in this book is meant to cover 2 or more full courses in optimization.

The book is designed to be read in a linear fashion. That said, many of the chapters are mostly independent and could be rearrranged, removed, or shortened as desited. This is an open source textbook, so use it as you like. You are encouraged to share adaptations and improvements so that this work can evolve over time.

Letter to students

This book is designed to be a resource for students interested in learning what optimizaiton is, how it works, and how you can apply it in your future career. The main focus is being able to apply the techniques of optimization to problems using computer technology while understanding (at least at a high level) what the computer is doing and what you can claim about the output from the computer.

Quite importantly, keep in mind that when someone claims to have *optimized* a problem, we want to know what kind of gaurantees they have about how good their solution is. Far too often, the solution that is provided is suboptimal by 10%, 20%, or even more. This can mean spending excess amounts of money, time, or energy that could have been saved. And when problems are at a large scale, this can easily result in millions of dollars in savings.

For this reason, we will learn the perspective of *mathematical programming* (a.k.a. mathematical optimization). The key to this study is that we provide gaurantees on how good a solution is to a given problem. We will also study how difficult a problem is to solve. This will help us know (a) how long it might take to solve it and (b) how good a of a solution we might expect to be able to find in reasonable amount of time.

We will later study *heuristic* methods - these methods typically do not come with gaurantees, but tend to help find quality solutions.

Note: Although there is some computer programming required in this work, this is not a course on programming. Thanks to the fantastic modelling packages available these days, we are able to solve complicated problems with little programming effort. The key skill we will need to learn is *mathematical modeling*: converting words and ideas into numbers and variables in order to communicate problems to a compter so that it can solve a problem for you.

As a main element of this book, we would like to make the process of using code and software as easy as possible. Attached to most examples in the book, there will be links to code that implements and solves the problem using several different tools from Excel and Python. These examples can should make it easy to solve a similar problem with different data, or more generally, can serve as a basis for solving related problems with similar structure.

How to use this book

Skim ahead. We recommend that before you come across a topic in lecture, that you skim the relevant sections ahead of time to get a broad overview of what is to come. This may take only a fraction of the time that it may take for you to read it.

Read the expected outcomes. At the beginning of each section, there will be a list of expected outcomes from that section. Review these outcomes before reading the section to help guide you through what is most relevant for you to take away from the material. This will also provide a brief look into what is to follow in that section.

Read the text. Read carefully the text to understand the problems and techniques. We will try to provide a plethora of examples, therefore, depending on your understanding of a topic, you many need to go carefully over all of the examples.

Explore the resources. Lastly, we recognize that there are many alternative methods of learning given the massive amounts of information and resources online. Thus, at the end of each section, there will be a number of superb resources that available on the internet in different formats. There are other free textbooks, informational websites, and also number of fantastic videos posted to youtube. We encourage to explore the resources to get another perspective on the material or to hear/read it taught from a differnt point of view or in presentation style.

Outline of this book

This book is divided in to 4 Parts:

Part I Linear Programming,

Part II Integer Programming,

Part III Discrete Algorithms,

There are also a number of chapters of background material in the Appendix.

The content of this book is designed to encompass 2-3 full semester courses in an industrial engineering department.

Work in progress

This book is still a work in progress, so please feel free to send feedback, questions, comments, and edits to Robert Hildebrand at open.optimization@gmail.com.

1. Resources and Notation

Here are a list of resources that may be useful as alternative references or additional references.

Free notes and textbooks

- Mathematical Programming with Julia by Richard Lusby & Thomas Stidsen
- Linear Programming by K.J. Mtetwa, David
- A first course in optimization by Jon Lee
- Introduction to Optimizaiton Notes by Komei Fukuda
- Convex Optimization by Bord and Vandenberghe
- LP notes of Michel Goemans from MIT
- Understanding and Using Linear Programming - Matousek and Gärtner [Downloadable from Springer with University account]
- Operations Research Problems Statements and Solutions - Raúl PolerJosefa Mula Manuel Díaz-Madroñero [Downloadable from Springer with University account]

Notes, books, and videos by various solver groups

- AIMMS Optimization Modeling
- Optimization Modeling with LINGO by Linus Schrage
- The AMPL Book
- Microsoft Excel 2019 Data Analysis and Business Modeling, Sixth Edition, by Wayne Winston - Available to read for free as an e-book through Virginia Tech library at Orielly.com.
- Lesson files for the Winston Book
- Video instructions for solver and an example workbook
- youtube-OR-course

Gurobi Links

- Go to `https://github.com/Gurobi` and download the example files.
- Essential ingredients
- Gurobi Linear Programming tutorial
- Gurobi tutorial MILP
- GUROBI - Python 1 - Modeling with GUROBI in Python
- GUROBI - Python II: Advanced Algebraic Modeling with Python and Gurobi
- GUROBI - Python III: Optimization and Heuristics
- Webinar Materials
- GUROBI Tutorials

How to prove things

- Hammack - Book of Proof

Statistics

- Open Stax - Introductory Statistics

Linear Algebra

- Beezer - A first course in linear algebra
- Selinger - Linear Algebra
- Cherney, Denton, Thomas, Waldron - Linear Algebra

Real Analysis

- Mathematical Analysis I by Elias Zakon

Discrete Mathematics, Graphs, Algorithms, and Combinatorics

- Levin - Discrete Mathematics - An Open Introduction, 3rd edition
- Github - Discrete Mathematics: an Open Introduction CC BY SA
- Keller, Trotter - Applied Combinatorics (CC-BY-SA 4.0)
- Keller - Github - Applied Combinatorics

Mixed Integer Nonlinear Programming

- Mixed-Integer Nonlinear Optimization Pietro Belotti, Christian Kirches, Sven Leyffer, Jeff Linderoth, Jim Luedtke, and Ashutosh Mahajan

Programming with Python

- A Byte of Python
- Github - Byte of Python (CC-BY-SA)

Also, go to `https://github.com/open-optimization/open-optimization-or-examples` to look at more examples.

Notation

- 1 - a vector of all ones (the size of the vector depends on context)
- $\forall$ - for all
- $\exists$ - there exists
- $\in$ - in
- $\therefore$ - therefore
- $\Rightarrow$ - implies
- s.t. - such that (or sometimes "subject to".... from context?)
- $\{0,1\}$ - the set of numbers 0 and 1
- $\mathbb{Z}$ - the set of integers (e.g. $1,2,3,-1,-2,-3,...$)
- $\mathbb{Q}$ - the set of rational numbers (numbers that can be written as p/q for $p,q \in \mathbb{Z}$ (e.g. $1,1/6,27/2$)
- $\mathbb{R}$ - the set of all real numbers (e.g. 1, 1.5, π, e, $-11/5$)
- $\setminus$ - setminus, (e.g. $\{0,1,2,3\} \setminus \{0,3\} = \{1,2\}$)
- $\cup$ - union (e.g. $\{1,2\} \cup \{3,5\} = \{1,2,3,5\}$

- $\cap$ - intersection (e.g. $\{1,2,3,4\} \cap \{3,4,5,6\} = \{3,4\}$)
- $\{0,1\}^4$ - the set of 4 dimensional vectors taking values 0 or 1, (e.g. $[0,0,1,0]$ or $[1,1,1,1]$)
- $\mathbb{Z}^4$ - the set of 4 dimensional vectors taking integer values (e.g., $[1,-5,17,3]$ or $[6,2,-3,-11]$)
- $\mathbb{Q}^4$ - the set of 4 dimensional vectors taking rational values (e.g. $[1.5,3.4,-2.4,2]$)
- $\mathbb{R}^4$ - the set of 4 dimensional vectors taking real values (e.g. $[3,\pi,-e,\sqrt{2}]$)
- $\sum_{i=1}^{4} i = 1+2+3+4$
- $\sum_{i=1}^{4} i^2 = 1^2+2^2+3^2+4^4$
- $\sum_{i=1}^{4} x_i = x_1+x_2+x_3+x_4$
- $\square$ - this is a typical Q.E.D. symbol that you put at the end of a proof meaning "I proved it."
- For $x,y \in \mathbb{R}^3$, the following are equivalent (note, in other contexts, these notations can mean different things)
  - $x^\top y$ *matrix multiplication*
  - $x \cdot y$ *dot product*
  - $\langle x,y \rangle$ *inner product*

  and evaluate to $\sum_{i=1}^{3} x_i y_i = x_1 y_1 + x_2 y_2 + x_3 y_3$.

A sample sentence:

$$\forall x \in \mathbb{Q}^n \; \exists y \in \mathbb{Z}^n \setminus \{0\}^n s.t. x^\top y \in \{0,1\}$$

"For all non-zero rational vectors x in n-dimensions, there exists a non-zero n-dimensional integer vector y such that the dot product of x with y evaluates to either 0 or 1."

2. Mathematical Programming

Outcomes

- *Consider an introductory problem*
- *Identify reasons for studying operations research*
- *Define "Mathematical Programming"*
- *Learn about different applications of the tools in this book*
- *Explore the different types of optimization models and what types we will see in this book*

2.1 Warm-Up Scenario

Imagine you're starting a new clothing company and decide to sell shirts. You've opened a store in Roanoke and, based on market research, you estimate the demand for the upcoming weekend:

- **Friday**: 5 shirts
- **Saturday**: 10 shirts
- **Sunday**: 7 shirts

Your supplier can deliver shirts in one day, meaning an order placed on Thursday will arrive on Friday, and so on. As a business owner, you need to decide how many shirts to order and when to place each order to ensure you meet the daily demand without overstocking.

*Defining the Problem To tackle this challenge, you'll need to think strategically about two main factors:

- **Ordering Quantities**: How many shirts to order on each day.
- **Inventory Levels**: How many shirts to keep in stock at the end of each day.

You can define these decisions using variables:

- Let the number of shirts you order on a specific day be a decision variable. For example, you could track the number of shirts ordered on Thursday, Friday, and Saturday.
- Define variables to represent the number of shirts you have in stock at the end of each day.

Your goal is to meet the projected demand while managing costs. Costs could include:

- The expense of ordering shirts.

- The cost of holding inventory.
- Penalties for running out of stock.

Refining the Scenario

As your business grows, additional complexities may arise. For instance:

- The cost of ordering shirts might vary by day due to supplier pricing.
- Holding inventory could incur storage costs.
- A budget might limit how much you can spend on orders.
- You might expand your product line to include pants and socks, each with its own demand and costs.

By clearly defining variables and constraints for these decisions, you can systematically approach the problem and develop an optimal strategy. This step-by-step process of defining the problem, incorporating real-world details, and refining the solution is a fundamental aspect of optimization.

Model Development Cycle

This multi-stage example illustrates the iterative cycle of modeling:

1. **Initial Problem**: Determine ordering quantities to meet demand.
2. **Modeling**: Formulate an optimization model based on available information.
3. **Solution**: Solve the model to find the optimal ordering plan.
4. **New Information**: Incorporate additional real-world complexities (varying costs, inventory costs, budget constraints, additional products, fixed ordering costs).
5. **Refinement**: Update the model to reflect new information and solve again.

Further Extensions

Looking ahead, there are several ways to expand this scenario:

- **Expanding to New Stores**: Opening additional stores introduces location-based demand, transportation costs, and coordination of inventory across multiple sites.
- **Hiring New Staff**: Incorporate labor costs, scheduling constraints, and productivity rates into the model.
- **Producing Clothing In-House**: Transitioning to manufacturing involves modeling production processes, including purchasing raw materials, machine capacities, and processing times.
- **Uncertain Demand**: In reality, demand may be uncertain. Advanced models can incorporate stochastic elements or use probabilistic methods to handle randomness.

In this course, we will focus on deterministic models where all data is known and fixed. Building a strong foundation with these models will prepare us to tackle more complex problems involving uncertainty and randomness in future studies.

2.2 Why Study Operations Research?

Operations Research (OR) is a discipline that applies advanced analytical methods to help make better decisions. In a world where resources are limited and organizations face complex challenges, OR provides the tools and methodologies to model, analyze, and solve problems involving the optimal allocation of scarce resources. Studying OR equips individuals with a systematic and quantitative approach to decision-making, which is invaluable in industries such as logistics, finance, healthcare, and manufacturing.

By understanding OR, you gain the ability to construct mathematical models of real-world systems, allowing for the simulation and optimization of different scenarios. This leads to improved efficiency, reduced costs, and enhanced performance. Moreover, the skills developed through studying OR—such as critical thinking, problem-solving, and data analysis—are highly transferable and sought after in today's data-driven economy.

Operations Research also bridges the gap between theory and practice. It transforms abstract mathematical concepts into actionable strategies that can have a tangible impact on organizational success. Whether it's optimizing supply chains, scheduling flights, or managing portfolios, the applications of OR are vast and impactful.

2.3 What is Mathematical Programming?

Mathematical Programming is a core area within Operations Research that focuses on the formulation and solution of optimization problems. It involves creating mathematical models that represent the decision-making process of a system, where the goal is to find the best possible outcome under given constraints. The term "programming" in this context refers to planning or scheduling, not computer programming.

At the heart of Mathematical Programming is the objective function—a mathematical expression that defines the criterion to be optimized, such as minimizing costs or maximizing profits. The decision variables represent the choices available, and the constraints reflect the limitations or requirements of the problem, such as resource capacities or regulatory conditions.

There are various types of mathematical programming, each suited to different kinds of problems:

- **Linear Programming (LP)**: Deals with problems where both the objective function and constraints are linear.
- **Integer Programming (IP)**: Involves decision variables that must take on integer values.

- **Nonlinear Programming (NLP)**: Handles problems with nonlinear objective functions or constraints.
- **Dynamic Programming (DP)**: Breaks down problems into simpler subproblems and is particularly useful for multistage decision processes.

Mathematical Programming provides a powerful framework for optimizing complex systems and making informed decisions. It combines elements of mathematics, economics, and computer science to develop solutions that are both efficient and effective.

2.4 Applications

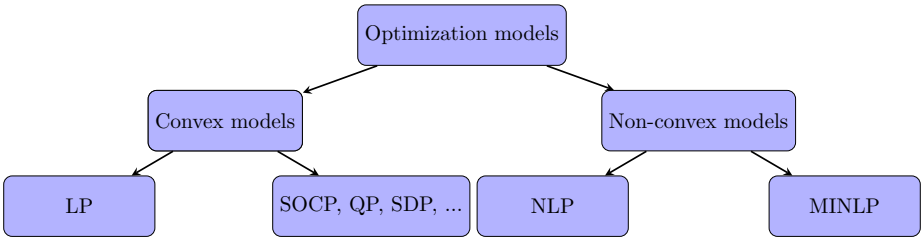
Operations Research and Mathematical Programming have a wide range of applications across various industries and sectors:

- **Transportation and Logistics**: Optimizing routing and scheduling for delivery vehicles, reducing transportation costs, and improving supply chain efficiency.
- **Manufacturing**: Streamlining production processes, managing inventory levels, and scheduling machinery to maximize productivity and minimize waste.
- **Finance**: Portfolio optimization, risk assessment, and capital budgeting to enhance financial performance and manage uncertainties.
- **Healthcare**: Allocating resources such as staff and equipment, scheduling surgeries, and managing patient flow to improve service quality and reduce wait times.
- **Energy Sector**: Optimizing power generation and distribution, planning for renewable energy integration, and managing energy consumption.
- **Telecommunications**: Network design and optimization, bandwidth allocation, and improving the reliability and efficiency of communication systems.
- **Agriculture**: Planning crop rotations, optimizing the use of fertilizers and water, and managing supply chains for agricultural products.
- **Public Sector**: Urban planning, emergency response logistics, and policy analysis to enhance public services and resource utilization.

These applications demonstrate the versatility and impact of OR and Mathematical Programming. By leveraging these tools, organizations can make data-driven decisions that lead to significant improvements in performance, cost savings, and competitive advantage. As global challenges become more complex, the importance of skilled professionals in Operations Research continues to grow.

2.5 Types of Optimization problems

We will state main general problem classes to be associated with in these notes. These are Linear Programming (LP), Mixed-Integer Linear Programming (MILP), Non-Linear Programming (NLP), and Mixed-Integer Non-Linear Programming (MINLP).



Along with each problem class, we will associate a complexity class for the general version of the problem. Complexity classes are discussed in a later portion of the book. Although we will often state that input data for a problem comes from $\mathbb{R}$ (as a real number), when we discuss complexity of such a problem, we actually mean that the data is rational, i.e., from $\mathbb{Q}$ (rational numbers that are encoded in the computer typically), and is given in binary encoding.

CAUTION!

In the following subsections, we will use advanced notation that may be unfamiliar at this point. These mathematical notations will be developed throughout the book.

Furthermore, don't be too concerned about the specific definitions of the complexity classes at this point. For now, you can read that *Polynomial time (P)* means it should be relatively easy to solve, while the other classes might be much more difficult to solve.

2.6 Linear Programming (LP)

Some linear programming background, theory, and examples will be provided in **??**.

Linear Programming (LP):

Polynomial time (P)

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *linear programming* problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \geq 0 \end{aligned} \tag{2.1}$$

Linear programming can come in several forms, whether we are maximizing or minimizing, or if the constraints are $\leq, =$ or $\geq$. One form commonly used is *Standard Form* given as

Linear Programming (LP) Standard Form:

Polynomial time (P)

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *linear programming* problem in *standard form* is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned} \tag{2.2}$$

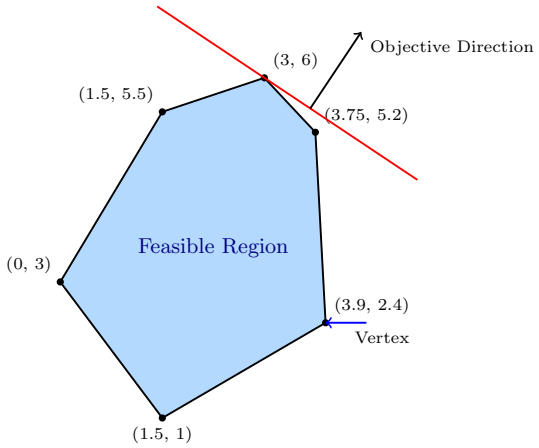


Figure 2.1: Linear programming constraints and objective.

2.7 Mixed-Integer Linear Programming (MILP)

Mixed-integer linear programming will be the focus of Sections **??**,**??**, **??**, and **??**. Recall that the notation $\mathbb{Z}$ means the set of integers and the set $\mathbb{R}$ means the set of real numbers. The first problem of interest here is a *binary integer program* (BIP) where all n variables are binary (either 0 or 1).

Binary Integer programming (BIP):

NP-Complete

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *binary integer programming* problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \{0,1\}^n \end{aligned} \tag{2.1}$$

A slightly more general class is the class of *Integer Linear Programs* (ILP). Often this is referred to as *Integer Program* (IP), although this term could leave open the possibility of non-linear parts.

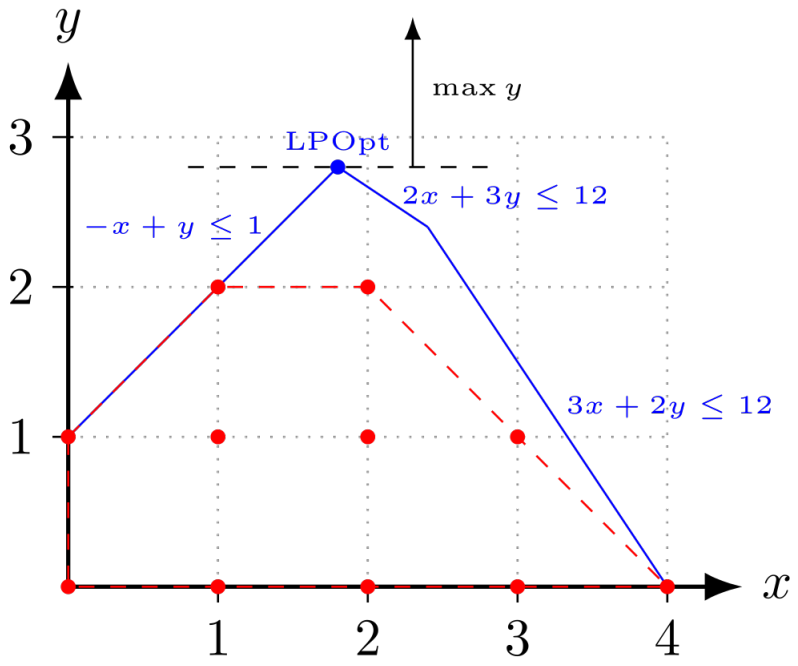


Figure 2.2: Comparing the LP relaxation to the IP solutions.

Figure 2.2

Integer Linear Programming (ILP):

NP-Complete

[1] *IP polytope with LP relaxation*, from `https://en.wikipedia.org/wiki/Integer_programming#/media/File:IP_polytope_with_LP_relaxation.svg`. Fanosta CC BY-SA 4.0., 2019.

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *integer linear programming* problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \mathbb{Z}^n \end{aligned} \tag{2.2}$$

An even more general class is *Mixed-Integer Linear Programming (MILP)*. This is where we have n integer variables $x_1, \ldots, x_n \in \mathbb{Z}$ and d continuous variables $x_{n+1}, \ldots, x_{n+d} \in \mathbb{R}$. Succinctly, we can write this as $x \in \mathbb{Z}^n \times \mathbb{R}^d$, where $\times$ stands for the *cross-product* between two spaces.

Below, the matrix A now has $n+d$ columns, that is, $A \in \mathbb{R}^{m \times n+d}$. Also note that we have not explicitly enforced non-negativity on the variables. If there are non-negativity restrictions, this can be assumed to be a part of the inequality description $Ax \leq b$.

Mixed-Integer Linear Programming (MILP):

NP-Complete

Given a matrix $A \in \mathbb{R}^{m \times (n+d)}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^{n+d}$, the *mixed-integer linear programming* problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \mathbb{Z}^n \times \mathbb{R}^d \end{aligned} \tag{2.3}$$

2.8 Non-Linear Programming (NLP)

NLP:

NP-Hard

Given a function $f(x)\colon \mathbb{R}^d \to \mathbb{R}$ and other functions $f_i(x)\colon \mathbb{R}^d \to \mathbb{R}$ for $i = 1, \ldots, m$, the *nonlinear programming* problem is

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & f_i(x) \leq 0 \quad \text{for } i = 1, \ldots, m \\ & x \in \mathbb{R}^d \end{aligned} \tag{2.1}$$

Nonlinear programming can be separated into convex programming and non-convex programming. These two are very different beasts and it is important to distinguish between the two.

2.8.1. Convex Programming

Here the functions are all **convex!**

Convex Programming:

Polynomial time (P) (typically)

Given a convex function $f(x)\colon \mathbb{R}^d \to \mathbb{R}$ and convex functions $f_i(x)\colon \mathbb{R}^d \to \mathbb{R}$ for $i = 1,\ldots,m$, the *convex programming* problem is

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & f_i(x) \leq 0 \quad \text{for } i = 1,\ldots,m \\ & x \in \mathbb{R}^d \end{aligned} \tag{2.2}$$

Observe that convex programming is a generalization of linear programming. This can be seen by letting $f(x) = c^\top x$ and $f_i(x) = A_i x - b_i$.

2.8.2. Non-Convex Non-linear Programming

When the function f or functions f_i are non-convex, this becomes a non-convex nonlinear programming problem. There are a few complexity issues with this.

IP AS NLP As seen above, quadratic constraints can be used to create a feasible region with discrete solutions. For example

$$x(1-x) = 0$$

has exactly two solutions: $x = 0, x = 1$. Thus, quadratic constraints can be used to model binary constraints.

Binary Integer programming (BIP) as a NLP:

NP-Hard

Given a matrix $A \in \mathbb{R}^{m\times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *binary integer programming* problem

is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \le b \\ & \cancel{x \in \{0,1\}^n} \\ & x_i(1-x_i) = 0 \quad \text{for } i = 1,\dots,n \end{aligned} \tag{2.3}$$

Alternatively, consider the transformation where $x_i \in \{-1,1\}$.

$$\begin{aligned} & \min c^\top x \\ & \text{s.t. } Ax \le b \\ & \quad x \in \{-1,1\}^n \end{aligned}$$

This can be reformulated with a single nonconvex constraint as

$$\begin{aligned} \min \quad & c^\top x \\ \text{s.t.} \quad & Ax \le b \\ & -1 \le x_j \le 1, \quad 1 \le j \le n, \\ & \|x\|^2 \ge n. \end{aligned}$$

2.8.3. Machine Learning

Machine learning problems are often cast as continuous optimization problems, which involve adjusting parameters to minimize or maximize a particular objective. Frequently they are convex optimization problems, but many turn out to be nonconvex. Here are two examples of how these problems arise at a glance. We will see examples in greater detail later in the book.

Loss Function Minimization

In supervised learning, this objective is typically a loss function L that quantifies the discrepancy between the predictions of a model and the true data labels. The aim is to adjust the parameters θ of the model to minimize this loss. Mathematically, this can be represented as:

$$\min_\theta L(\theta) = \min_\theta \frac{1}{N} \sum_{i=1}^{N} l(y_i, f(x_i;\theta)) \tag{2.4}$$

where N is the number of data points, l is a per-data-point loss (e.g., squared error for regression or cross-entropy for classification), y_i is the true label for the i-th data point, and $f(x_i;\theta)$ is the model's prediction for the i-th data point with parameters θ.

Clustering Formulation

Clustering, on the other hand, seeks to group or partition data points such that data points in the same group are more similar to each other than those in other groups. One popular method is the k-means clustering algorithm. The objective of k-means is to partition the data into k clusters by minimizing the within-cluster sum of squares (WCSS). The mathematical formulation can be given as:

$$\min_{\mathbf{c}_1,\ldots,\mathbf{c}_k} \sum_{j=1}^{k} \sum_{x_i \in C_j} \left\| x_i - \mathbf{c}_j \right\|^2 \tag{2.5}$$

where C_j represents the j-th cluster and $\mathbf{c}_j$ is the centroid of that cluster.

This encapsulation presents a glimpse into how ML problems are framed mathematically. In practice, numerous algorithms, constraints, and regularizations add complexity to these basic formulations.

2.9 Mixed Integer Non-Linear Programming (MINLP)

MINLP:

NP-Hard

Mixed Integer Nonlinear Programming (MINLP) combines elements of integer programming and nonlinear programming. In MINLP, some or all of the decision variables are constrained to be integers, and the objective function or constraints are nonlinear. A general MINLP problem can be formulated as:

$$\begin{aligned} \min \quad & f(x,y) \\ \text{s.t.} \quad & g_i(x,y) \leq 0 \quad \text{for } i = 1,\ldots,m \\ & h_j(x,y) = 0 \quad \text{for } j = 1,\ldots,p \\ & x \in \mathbb{R}^n, y \in \mathbb{Z}^k \end{aligned} \tag{2.1}$$

where f is the objective function, g_i and h_j are constraint functions, x is a vector of continuous variables, and y is a vector of integer variables.

MINLP is particularly challenging due to the nonlinearity in the objective and/or constraints and the discrete nature of some decision variables. It finds applications in various fields such as industry for process optimization, computational geometry, and machine learning for hyperparameter tuning.

2.9.1. Convex MINLP

In the convex case, both the objective function and the constraints are convex functions. This subclass is easier to solve compared to its non-convex counterpart.

Convex MINLP:

NP-Hard, but *Polynomial time (P)* in fixed dimension (typically)

For a convex MINLP, the problem is defined as:

$$\begin{aligned} \min \quad & f(x,y) \quad \text{(convex)} \\ \text{s.t.} \quad & g_i(x,y) \leq 0 \quad \text{(convex constraints)} \\ & x \in \mathbb{R}^n, y \in \mathbb{Z}^k \end{aligned} \tag{2.2}$$

Convex MINLPs, while still challenging, are typically more tractable due to the properties of convexity, which allow for more efficient solution methods.

2.9.2. Non-Convex MINLP

Non-convex MINLPs are significantly harder due to the presence of non-convex functions, which can lead to multiple local minima.

Non-Convex MINLP:

NP-Hard(in fact, undecidable)

The non-convex MINLP problem is formulated as:

$$\begin{aligned} \min \quad & f(x,y) \quad \text{(non-convex)} \\ \text{s.t.} \quad & g_i(x,y) \leq 0 \quad \text{(possibly non-convex constraints)} \\ & x \in \mathbb{R}^n, y \in \mathbb{Z}^k \end{aligned} \tag{2.3}$$

Non-convex MINLPs pose significant computational challenges due to the possibility of multiple local optima and the inherent complexity of integer constraints. These problems are common in real-world applications where decisions are discrete, and the system behavior is non-linear and complex.

Complexity and Applications

MINLP problems are known for their computational complexity, primarily due to the combination of non-linearity and integrality. The non-convex variants, in particular, are *NP-Hard*, making them some of the most challenging problems in optimization.

In practice, MINLP models find extensive applications across various domains. In industry, they are used for complex decision-making processes like supply chain optimization and production planning. In computational geometry, MINLP techniques help in solving problems like optimal packing or layout design. Furthermore, in the realm of machine learning, MINLPs are crucial for tasks like feature selection and hyperparameter optimization where discrete choices and nonlinear relationships are involved.

The versatility of MINLP models, combined with their inherent complexity, makes them a fascinating and essential area of study in the field of optimization.

Category	Software (License)
Linear programming (free)	CDD, CLP, GLPK, LPSOLVE, OSQP, SCIP
Mixed Integer Linear programming (free)	CBC, GLPK, LPSOLVE, SCIP
Linear programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), KNITRO, LINPROG, MOSEK (free for academia), XPRESS (free for academia)
Mixed Integer Linear programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), INTLINPROG, KNITRO, MOSEK (free for academia), XPRESS (free for academia)
Quadratic programming (free)	OSQP, CLP, DAQP (mixed-binary), OOQP, QPC, QPOASES, QUADPROGBB (nonconvex QP)
Quadratic programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), KNITRO, MOSEK (free for academia), QUADPROG, XPRESS (free for academia)
Mixed Integer Quadratic programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), KNITRO, MOSEK (free for academia), XPRESS (free for academia)
Second-order cone programming (free)	ECOS, SCS, SDPT3, SEDUMI
Second-order cone programming (commercial)	COPT (free for academia), CPLEX (free for academia), CONEPROG, GUROBI (free for academia), MOSEK (free for academia), XPRESS (free for academia)
Mixed Integer Second-order cone programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), MOSEK (free for academia), XPRESS (free for academia)
Semidefinite programming (free)	CSDP, DSDP, LOGDETPPA, PENLAB, SCS, SDPA, SDPLR, SDPT3, SDPNAL, SEDUMI
Semidefinite programming (commercial)	COPT (free for academia), LMIlab (not recommended), MOSEK (free for academia), PENBMI, PENSDP (free for academia)
General nonlinear programming and other solvers	BARON, FILTERSD, FMINCON, GPPSOY, IPOPT, KNITRO, LMIrank, MPT, NOMAD, PENLAB, SNOPT, SPARSEPOP

Table 2.1: Optimization Software and their Licenses

2.10 Optimization Under Uncertainty

Optimization problems often involve uncertainty in their parameters, whether due to measurement errors, variability in input data, or unpredictable future events. Addressing uncertainty is crucial for developing solutions that are effective in real-world settings. Two primary approaches to optimization under uncertainty are stochastic optimization and robust optimization.

2.10.1. Stochastic Optimization

Stochastic optimization deals with problems where some parameters are uncertain but can be described probabilistically. In this framework, the optimization process incorporates the expected values, variances, or other statistical characteristics of uncertain parameters. Solutions are often evaluated based on their performance over a distribution of possible scenarios.

For example, in supply chain management, uncertain demand can be modeled using probability distributions. A company may aim to minimize costs while ensuring a high service level under varying demand conditions. Similarly, in finance, portfolio optimization often accounts for uncertain future returns by maximizing expected returns while managing risk.

2.10.2. Robust Optimization

Robust optimization, on the other hand, focuses on creating solutions that perform well across the worst-case realizations of uncertainty. Rather than assuming a specific probability distribution, robust optimization works with uncertainty sets, which define the range of possible values that uncertain parameters can take. The goal is to find solutions that are feasible and effective for all possible scenarios within the uncertainty set.

Applications of robust optimization include power grid operations, where uncertainties in demand and renewable energy generation need to be managed, and scheduling problems, such as airline crew scheduling, where disruptions must be minimized under varying operational conditions.

2.10.3. Applications

Optimization under uncertainty has a wide array of applications across industries:

- **Healthcare:** Designing vaccination strategies to minimize the spread of diseases while accounting for uncertain infection rates and patient behaviors.
- **Transportation:** Managing traffic flow or public transit schedules under uncertain demand and travel times.
- **Energy:** Planning renewable energy investments and operations under uncertain weather conditions and energy prices.

- **Manufacturing:** Ensuring production schedules are resilient to supply chain disruptions and variability in raw material quality.

Addressing the complexity of uncertain data typically makes these problems much harder than their deterministic counterparts. While these methods are powerful and applicable to a wide range of real-world scenarios, we will not focus on optimization under uncertainty in this textbook.

Part I

Linear Programming

3. Modeling: Linear Programming

Outcomes

1. *Define what a linear program is*
2. *Understand how to model a linear program*
3. *View many examples and get a sense of what types of problems can be modeled as linear programs.*

Linear Programming, also known as Linear Optimization, is the starting point for most forms of optimization. It is the problem of optimization a linear function over linear constraints.

In this chapter, we will define what this means, how to setup a linear program, and discuss many examples. Examples will be connected with code in Excel and Python (using with PuLP or Gurobipy modeling tools) so that you can easily start solving optimization problems. Tutorials on these tools will come in later chapters.

As a warm up, consider the following word problem.
Question: Find the age of Justin and Brad, given that the sum of their ages is equal to 76 and after 7 years, the age of Brad will be twice the age of Justin.

Solution: Let x and y denote the age of Justin and Brad, respectively. Then

$$x+y=76 \tag{1}$$

$$2(x+7)=y+7. \tag{2}$$

On solving the foregoing equations we get

$$x=23 \quad \text{and} \quad y=53.$$

We will need to think about assigning variables to decisions and instead of equations guiding these variables, we may have inequalites that bound these variables in different ways. Thus, there can be many possible solutions.

3.1 Introductory LP Example

We begin with a simple example.

Example 3.1: Toy Maker Excel PuLP Gurobipy

Consider the problem of a toy company that produces toy planes and toy boats. The toy company can sell its planes for \$10 and its boats for \$8 dollars. It costs \$3 in raw materials to make a plane and \$2 in raw materials to make a boat. A plane requires 3 hours to make and 1 hour to finish while a boat requires 1 hour to make and 2 hours to finish. The toy company knows it will not sell anymore than 35 planes per week. Further, given the number of workers, the company cannot spend anymore than 160 hours per week finishing toys and 120 hours per week making toys. The company wishes to maximize the profit it makes by choosing how much of each toy to produce.

We can represent the profit maximization problem of the company as a linear programming problem. Let x_1 be the number of planes the company will produce and let x_2 be the number of boats the company will produce. The profit for each plane is $\$10 - \$3 = \$7$ per plane and the profit for each boat is $\$8 - \$2 = \$6$ per boat. Thus the total profit the company will make is:

$$z(x_1, x_2) = 7x_1 + 6x_2 \tag{3.1}$$

The company can spend no more than 120 hours per week making toys and since a plane takes 3 hours to make and a boat takes 1 hour to make we have:

$$3x_1 + x_2 \leq 120 \tag{3.2}$$

Likewise, the company can spend no more than 160 hours per week finishing toys and since it takes 1 hour to finish a plane and 2 hour to finish a boat we have:

$$x_1 + 2x_2 \leq 160 \tag{3.3}$$

Finally, we know that $x_1 \leq 35$, since the company will make no more than 35 planes per week. Thus the complete linear programming problem is given as:

$$\begin{aligned} \max \quad & z(x_1, x_2) = 7x_1 + 6x_2 \\ s.t. \quad & 3x_1 + x_2 \leq 120 \\ & x_1 + 2x_2 \leq 160 \\ & x_1 \leq 35 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \end{aligned} \quad . \tag{3.4}$$

Exercise 3.1: Chemical Manufacturing

A chemical manufacturer produces three chemicals: A, B and C. These chemical are produced by two processes: 1 and 2.

- *Running process 1 for 1 hour costs $4 and yields 3 units of chemical A, 1 unit of chemical B and 1 unit of chemical C.*
- *Running process 2 for 1 hour costs $1 and produces 1 units of chemical A, and 1 unit of chemical B (but none of Chemical C).*

To meet customer demand,

- *at least 10 units of chemical A,*
- *at least 5 units of chemical B and*
- *at least 3 units of chemical C*

must be produced daily.

Assume that the chemical manufacturer wants to minimize the cost of production.
Develop a linear programming problem describing the constraints and objectives of the chemical manufacturer.

[Hint: Let x_1 be the amount of time Process 1 is executed and let x_2 be amount of time Process 2 is executed. Use the coefficients above to express the cost of running Process 1 for x_1 time and Process 2 for x_2 time. Do the same to compute the amount of chemicals A, B, and C that are produced.]

3.2 Modeling and Assumptions in Linear Programming

Outcomes

1. *Address crucial assumptions when chosing to model a problem with linear programming.*

A Generic Linear Program (LP)

Linear programming (LP) is a mathematical optimization framework widely used in decision-making and resource allocation. In its most basic form, an LP seeks to optimize (maximize or minimize) a linear objective function subject to a set of linear constraints. These constraints typically represent limitations or requirements in real-world scenarios, such as budgets, capacities, or resource availability.

Decision Variables:
The decision variables represent the quantities we want to determine. For example:

- x_i: Continuous variables ($x_i \in \mathbb{R}$, i.e., a real number), $\forall i = 1, \ldots, n$.

These variables could represent amounts of products to produce, resources to allocate, or investments to make. We can use any letters to represent these variables. Typically we use x_i, but frequently will use y_i or other letters depending on the context and what makes reading the problem most convenient.

Parameters (Known Input Parameters):
Parameters are the given data in the problem, which are assumed to be known in advance:

- c_i: Cost coefficients for each decision variable, $\forall i = 1,\ldots,n$. These values represent the contribution of each variable to the objective function.
- a_{ij}: Constraint coefficients, $\forall i = 1,\ldots,n$ and $j = 1,\ldots,m$. These define the relationships between decision variables and the constraints.
- b_j: Right-hand side values of the constraints, $\forall j = 1,\ldots,m$. These represent limits or requirements for each constraint.

The general form of a linear program can be written as follows:

$$\begin{aligned} \max \quad & z = c_1x_1 + \cdots + c_nx_n \\ s.t. \quad & a_{11}x_1 + \cdots + a_{1n}x_n \leq b_1 \\ & \qquad\qquad \vdots \\ & a_{m1}x_1 + \cdots + a_{mn}x_n \leq b_m \\ & x_i \geq 0 \quad \forall i = 1,\ldots,n. \end{aligned} \tag{3.1}$$

Features of a Linear Program

- **Objective Function**: The linear function $z = c_1x_1 + \cdots + c_nx_n$ represents the goal to optimize. This could involve maximizing profit, minimizing cost, or achieving some other measurable outcome.
- **Constraints**: The system of inequalities (or equalities) restricts the values of the decision variables. Each constraint reflects a real-world limitation, such as resource availability or capacity limits.
- **Feasibility**: The set of all possible solutions that satisfy the constraints is called the feasible region. The optimal solution must lie within this region.
- **Linear Relationships**: Both the objective function and constraints are linear, meaning the relationships between variables are additive and proportional.
- **Sets**: For specific optimization problem, it can be useful to define sets of variables or pramaters to help write down the problem in a generic forumlation.

Example of a Linear Program

To illustrate, consider a specific example with three decision variables and four constraints. This includes a mix of constraint types ($\leq$, $\geq$, and $=$), showcasing the versatility of linear programming:

Decision Variables:
x_i: Continuous variables ($x_i \in \mathbb{R}$, i.e., a real number), $\forall i = 1,\ldots,3$.

Parameters (Known Input Parameters):

- c_i: Cost coefficients, $\forall i = 1,\ldots,3$.
- a_{ij}: Constraint coefficients, $\forall i = 1,\ldots,3$ and $j = 1,\ldots,4$.
- b_j: Right-hand side values for each constraint, $\forall j = 1,\ldots,4$.

The linear program can be written as:

$$\text{Minimize } z = c_1x_1 + c_2x_2 + c_3x_3 \tag{3.2}$$
$$\text{Subject to } a_{11}x_1 + a_{12}x_2 + a_{13}x_3 \geq b_1 \tag{3.3}$$
$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 \leq b_2 \tag{3.4}$$
$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3 \tag{3.5}$$
$$a_{41}x_1 + a_{42}x_2 + a_{43}x_3 \geq b_4 \tag{3.6}$$
$$x_1 \geq 0,\ x_2 \leq 0,\ x_3 \text{ unrestricted.} \tag{3.7}$$

A key feature of linear programming is the use of linear functions, defined as follows:

Definition 3.2: Linear Function

A function $z : \mathbb{R}^n \to \mathbb{R}$ is linear if there are constants $c_1,\ldots,c_n \in \mathbb{R}$ such that:

$$z(x_1,\ldots,x_n) = c_1x_1 + \cdots + c_nx_n. \tag{3.8}$$

Linear functions are characterized by their simplicity and additivity, making them well-suited for optimization problems where proportionality is assumed. This simplicity allows linear programs to be solved efficiently, even for large-scale problems.

3.2.1. Assumptions

Inspecting Example 3.1 (or the more general problem in Equation (3.1)) we can see there are several assumptions that must be satisfied when using a linear programming model. We enumerate these below:

Proportionality Assumption A problem can be phrased as a linear program only if the contribution to the objective function *and* the left-hand-side of each constraint by each decision variable $(x_1,\ldots,x_n)$ is proportional to the value of the decision variable.

Additivity Assumption A problem can be phrased as a linear programming problem only if the contribution to the objective function *and* the left-hand-side of each constraint by any decision variable x_i $(i = 1,\ldots,n)$ is completely independent of any other decision variable x_j $(j \neq i)$ and additive.

Divisibility Assumption A problem can be phrased as a linear programming problem only if the quantities represented by each decision variable are infinitely divisible (i.e., fractional answers make sense).

Certainty Assumption A problem can be phrased as a linear programming problem only if the coefficients in the objective function and constraints are known with certainty.

The first two assumptions simply assert (in English) that both the objective function and functions on the left-hand-side of the (in)equalities in the constraints are linear functions of the variables $x_1, \ldots, x_n$.

The third assumption asserts that a valid optimal answer could contain fractional values for decision variables. It's important to understand how this assumption comes into play–even in the toy making example. Many quantities can be divided into non-integer values (ounces, pounds etc.) but many other quantities cannot be divided. For instance, can we really expect that it's reasonable to make $\frac{1}{2}$ a plane in the toy making example? When values must be constrained to true integer values, the linear programming problem is called an *integer programming problem*. There is a *vast* literature dealing with these problems. For many problems, particularly when the values of the decision variables may become large, a fractional optimal answer could be obtained and then rounded to the nearest integer to obtain a reasonable answer. For example, if our toy problem were re-written so that the optimal answer was to make 1045.3 planes, then we could round down to 1045.

The final assumption asserts that the coefficients (e.g., profit per plane or boat) is known with absolute certainty. In traditional linear programming, there is no lack of knowledge about the make up of the objective function, the coefficients in the left-hand-side of the constraints or the bounds on the right-hand-sides of the constraints. There is a literature on *stochastic programming* that relaxes some of these assumptions, but this too is outside the scope of the course.

Exercise 3.3

In a short sentence or two, discuss whether the problem given in Example 3.1 meets all of the assumptions of a scenario that can be modeled by a linear programming problem. Do the same for Exercise 3.1.
[Hint: Can you make $\frac{2}{3}$ of a toy? Can you run a process for $\frac{1}{3}$ of an hour?]

3.3 Examples

Outcomes

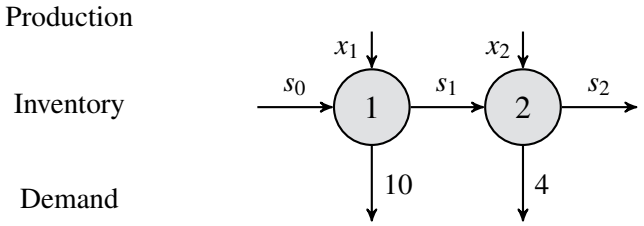
1. *Identify decision variables and paramters*
2. *Learn how to format a linear optimization problem.*
3. *Identify and understand common classes of linear optimization problems.*

We will begin with a few examples, and then discuss specific problem types that occur often.

Example 3.2: Production Planning Excel PuLP Gurobipy

A manufacturing firm seeks to optimize its production schedule over a two-day planning horizon to meet daily demand while minimizing costs. The firm starts with an initial inventory of s_0 units. The costs associated with production and inventory are as follows:

- **Production costs:** \$10 per unit on Day 1, and \$16 per unit on Day 2.
- **Inventory cost:** \$5 per unit for carrying excess inventory from one day to the next.



The goal is to determine the number of units to produce each day and the inventory levels to minimize total costs while satisfying the demand for each day.

Solution. Sets:

- Days of production $= \{1, 2\}$.

Parameters:

- c_{p1}: Per unit production cost on Day 1 (\$10).
- c_{p2}: Per unit production cost on Day 2 (\$16).
- c_{inv}: Per unit inventory cost (\$5).
- d_1: Demand on Day 1 (10 units).
- d_2: Demand on Day 2 (4 units).
- s_0: Inventory at the start of Day 1.

Decision variables:

- x_1: Number of units produced on Day 1.
- x_2: Number of units produced on Day 2.
- s_1: Inventory at the end of Day 1.
- s_2: Inventory at the end of Day 2.

Objective and Constraints:

$$
\begin{aligned}
\min \quad & z = 10x_1 + 16x_2 + 5s_1 + 5s_2 && \text{(Total cost)} \\
\text{s.t.} \quad & s_0 + x_1 = 10 + s_1 && \text{(Day 1 balance)} \\
& s_1 + x_2 = 4 + s_2 && \text{(Day 2 balance)} \\
& x_1, x_2, s_0, s_1, s_2 \geq 0 && \text{(Non-negativity constraints).}
\end{aligned}
$$

♠

Definition 3.4: Optimization model components

When defining an optimization model, it is essential to identify and clearly articulate the following five components:

- ***Sets:*** *What lists of data do you need to write down? These could include indices, categories, or groups relevant to the problem.*
- ***Parameters (Data):*** *What is the actual data for the problem? These are the fixed numerical values such as costs, capacities, or resource availabilities.*
- ***Decision Variables:*** *What decisions need to be made to provide a solution? These variables represent the quantities that can be controlled or adjusted, such as production levels or resource allocations.*
- ***Objective Function:*** *The expression that we want to maximize or minimize. This is the goal of the optimization, such as minimizing cost or maximizing profit.*
- ***Constraints:*** *Certain rules that those decision variables should satisfy. These include limitations or requirements, such as resource capacities or demand fulfillment.*

Each of these components plays a crucial role in formulating a well-structured optimization model that accurately represents the problem and provides actionable solutions. The role of Sets will become more important as we get into more complicated optimization models.

Example 3.3: Bakery Production Excel PuLP Gurobipy

A small bakery produces three products each day: cakes, cookies, and muffins. Each item requires a specific amount of ingredients and baking time, and each contributes to the bakery's profit. The bakery operates with daily limits on flour, sugar, and baking time due to ingredient supply and staffing constraints.

Each cake earns a profit of \$6.00, requires 500 grams of flour, 200 grams of sugar, and 60 minutes of baking time. Each cookie earns a profit of \$1.50, requires 100 grams of flour, 50 grams of sugar, and 10 minutes of baking time. Each muffin earns a profit of \$2.50, requires 200 grams of flour, 80 grams of sugar, and 20 minutes of baking time.

The bakery has at most 4,800 grams of flour, 2,060 grams of sugar, and 480 minutes of baking time available each day.

Determine how many cakes, cookies, and muffins the bakery should produce each day in order to maximize its total profit, while staying within the limits on flour, sugar, and baking time.

Linear Programming Model

$$
\begin{aligned}
\text{Maximize} \quad & 6x_1 + 1.5x_2 + 2.5x_3 \\
\text{subject to} \quad & 500x_1 + 100x_2 + 200x_3 \leq 4800 \quad \text{(Flour)} \\
& 200x_1 + 50x_2 + 80x_3 \leq 2060 \quad \text{(Sugar)} \\
& 60x_1 + 10x_2 + 20x_3 \leq 480 \quad \text{(Baking time)} \\
& x_1, x_2, x_3 \geq 0 \\
& x_1, x_2, x_3 \in \mathbb{Z} \quad \text{(if integer production is required)}
\end{aligned}
$$

Example 3.4: Production with welding robot Excel PuLP Gurobipy

You have 21 units of transparent aluminum alloy (TAA), LazWeld1, a joining robot leased for 23 hours, and CrumCut1, a cutting robot leased for 17 hours of aluminum cutting. You also have production code for a bookcase, desk, and cabinet, along with commitments to buy any of these you can produce for \$18, \$16, and \$10 apiece, respectively. A bookcase requires 2 units of TAA, 3 hours of joining, and 1 hour of cutting, a desk requires 2 units of TAA, 2 hours of joining, and 2 hour of cutting, and a cabinet requires 1 unit of TAA, 2 hours of joining, and 1 hour of cutting.

Formulate an LP to maximize your revenue given your current resources.

Solution.

Sets:

- The types of objects = { bookcase, desk, cabinet}.

Parameters:

- Purchase cost of each object
- Units of TAA needed for each object
- Hours of joining needed for each object
- Hours of cutting needed for each object
- Hours of TAA, Joining, and Cutting available on robots

Decision variables:

x_i : number of units of product i to produce,
for all i =bookcase, desk, cabinet.

Objective and Constraints:

$$\begin{aligned} \max \;& z = 18x_1 + 16x_2 + 10x_3 && \text{(profit)} \\ s.t.\;& 2x_1 + 2x_2 + 1x_3 \leq 21 && (TAA) \\ & 3x_1 + 2x_2 + 2x_3 \leq 23 && (LazWeld1) \\ & 1x_1 + 2x_2 + 1x_3 \leq 17 && (CrumCut1) \\ & x_1, x_2, x_3 \geq 0. \end{aligned}$$

♠

Example 3.5: The Diet Problem Excel PuLP Gurobipy

In the future (as envisioned in a bad 70's science fiction film) all food is in tablet form, and there are four types, green, blue, yellow, and red. A balanced, futuristic diet requires, at least 20 units of Iron, 25 units of Vitamin B, 30 units of Vitamin C, and 15 units of Vitamin D. Formulate an LP that ensures a balanced diet at the minimum possible cost.

Tablet	Iron	B	C	D	Cost ($)
Chem 1	6	6	7	4	1.25
Chem 2	4	5	4	9	1.05
Chem 3	5	2	5	6	0.85
Chem 4	3	6	3	2	0.65

Solution. Sets:

- Set of tablets $\{1,2,3,4\}$

Parameters:

- Iron in each tablet
- Vitamin B in each tablet
- Vitamin C in each tablet
- Vitamin D in each tablet
- Cost of each tablet

Decision variables:
x_i : number of tablet of type i to include in the diet, $\forall i \in \{1,2,3,4\}$.

Objective and Constraints:

$$\begin{aligned} \text{Min} \quad & z = 1.25x_1 + 1.05x_2 + 0.85x_3 + 0.65x_4 \\ \text{s.t.} \quad & 6x_1 + 4x_2 + 5x_3 + 3x_4 \geq 20 \\ & 6x_1 + 5x_2 + 2x_3 + 6x_4 \geq 25 \\ & 7x_1 + 4x_2 + 5x_3 + 3x_4 \geq 30 \\ & 4x_1 + 9x_2 + 6x_3 + 2x_4 \geq 15 \\ & x_1, x_2, x_3, x_4 \geq 0. \end{aligned}$$

♠

Example 3.6: The Next Diet Problem Excel PuLP Gurobipy

Progress is important, and our last problem had too many tablets, so we are going to produce a single, purple, 10 gram tablet for our futuristic diet requires, which are at least 20 units of Iron, 25 units of Vitamin B, 30 units of Vitamin C, and 15 units of Vitamin D, and 2000 calories. The tablet is made from blending 4 nutritious chemicals; the following table shows the units of our nutrients per, and cost of, grams of each chemical.

Formulate an LP that ensures a balanced diet at the minimum possible cost.

Tablet	Iron	B	C	D	Calories	Cost ($)
Chem 1	6	6	7	4	1000	1.25
Chem 2	4	5	4	9	250	1.05
Chem 3	5	2	5	6	850	0.85
Chem 4	3	6	3	2	750	0.65

Solution.

Sets:

- Set of chemicals $\{1,2,3,4\}$

Parameters:

- Iron in each chemical
- Vitamin B in each chemical
- Vitamin C in each chemical
- Vitamin D in each chemical
- Cost of each chemical

Decision variables:
x_i : grams of chemical i to include in the purple tablet, $\forall i = 1,2,3,4$.
Objective and Constraints:

$$
\begin{aligned}
\min \quad & z = 1.25x_1 + 1.05x_2 + 0.85x_3 + 0.65x_4 \\
s.t. \quad & 6x_1 + 4x_2 + 5x_3 + 3x_4 \geq 20 \\
& 6x_1 + 5x_2 + 2x_3 + 6x_4 \geq 25 \\
& 7x_1 + 4x_2 + 5x_3 + 3x_4 \geq 30 \\
& 4x_1 + 9x_2 + 6x_3 + 2x_4 \geq 15 \\
& 1000x_1 + 250x_2 + 850x_3 + 750x_4 \geq 2000 \\
& x_1 + x_2 + x_3 + x_4 = 10 \\
& x_1, x_2, x_3, x_4 \geq 0.
\end{aligned}
$$

♠

Example 3.7: Work Scheduling Problem Excel PuLP Gurobipy

You are the manager of LP Burger. The following table shows the minimum number of employees required to staff the restaurant on each day of the week. Each employees must work for five consecutive days. Formulate an LP to find the minimum number of employees required to staff the restaurant.

Day of Week	Workers Required
1 = Monday	6
2 = Tuesday	4
3 = Wednesday	5
4 = Thursday	4
5 = Friday	3
6 = Saturday	7
7 = Sunday	7

Solution. This problem has multiple optimal solutions for which days workers begin working on, all of which result in 8 total workers hired.

Decision variables:
x_i : the number of workers that start 5 consecutive days of work on day i, $i = 1,\ldots,7$
Objective and Constraints:

$$
\begin{aligned}
\text{Min} \quad & z = x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 \\
\text{s.t.} \quad & x_1 + x_4 + x_5 + x_6 + x_7 \geq 6 \\
& x_2 + x_5 + x_6 + x_7 + x_1 \geq 4 \\
& x_3 + x_6 + x_7 + x_1 + x_2 \geq 5 \\
& x_4 + x_7 + x_1 + x_2 + x_3 \geq 4 \\
& x_5 + x_1 + x_2 + x_3 + x_4 \geq 3 \\
& x_6 + x_2 + x_3 + x_4 + x_5 \geq 7 \\
& x_7 + x_3 + x_4 + x_5 + x_6 \geq 7 \\
& x_1, x_2, x_3, x_4, x_5, x_6, x_7 \geq 0.
\end{aligned}
$$

One solution is as follows:

LP Solution	IP Solution
$z_{LP} = 7.333$	$z_I = 8.0$
$x_1 = 0$	$x_1 = 0$
$x_2 = 0.333$	$x_2 = 0$
$x_3 = 1$	$x_3 = 0$
$x_4 = 2.333$	$x_4 = 3$
$x_5 = 0$	$x_5 = 0$
$x_6 = 3.333$	$x_6 = 4$
$x_7 = 0.333$	$x_7 = 1$

♠

Example 3.8: LP Burger - extended Excel PuLP Gurobipy

LP Burger has changed its policy, and allows, at most, two part time workers, who work for two consecutive days in a week. Formulate this problem.

Solution. Decision variables:
x_i : the number of workers that start 5 consecutive days of work on day i, $i = 1, \cdots, 7$
y_i : the number of workers that start 2 consecutive days of work on day i, $i = 1, \cdots, 7$.

Objective and Constraints:

$$
\begin{aligned}
\text{Min} \quad & z = 5(x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7) \\
& \quad +2(y_1 + y_2 + y_3 + y_4 + y_5 + y_6 + y_7) \\
\text{s.t.} \quad & x_1 + x_4 + x_5 + x_6 + x_7 + y_1 + y_7 \geq 6 \\
& x_2 + x_5 + x_6 + x_7 + x_1 + y_2 + y_1 \geq 4 \\
& x_3 + x_6 + x_7 + x_1 + x_2 + y_3 + y_2 \geq 5 \\
& x_4 + x_7 + x_1 + x_2 + x_3 + y_4 + y_3 \geq 4 \\
& x_5 + x_1 + x_2 + x_3 + x_4 + y_5 + y_4 \geq 3 \\
& x_6 + x_2 + x_3 + x_4 + x_5 + y_6 + y_5 \geq 7 \\
& x_7 + x_3 + x_4 + x_5 + x_6 + y_7 + y_6 \geq 7 \\
& y_1 + y_2 + y_3 + y_4 + y_5 + y_6 + y_7 \leq 2 \\
& x_i \geq 0, y_i \geq 0, \forall i = 1, \cdots, 7.
\end{aligned}
$$

3.3.1. Knapsack Problem

We consider now a simple example where the variables can only take the values of 0 or 1. We call these *binary variables*.

Example 3.9: Camping Trip Excel PuLP Gurobipy

Imagine you are preparing for a week-long camping trip to the mountains. You have a backpack with a weight capacity of 20 kilograms. Your goal is to pack the most valuable items to ensure a comfortable and safe trip without exceeding the backpack's weight limit. Each item has a weight and a value associated with it, representing its importance and utility for the trip. The table below lists the potential items you can take, their weights in kilograms, and their values on a scale from 1 to 10 (with 10 being the most valuable).
Formulate an Integer Program to find the optimal items you should bring on your trip with you.

Item	Index	Weight (kg)	Value
Tent	A	4	10
Stove	B	2	9
Sleeping Bag	C	3	8
First Aid Kit	D	1	7
Food Supplies	E	5	9
Water Filter	F	1	6
Clothes	G	4	5
Map and Compass	H	0.5	7

Solution. Decision variables:
x_i : whether to include item i in the backpack, where $i \in \{A,B,C,D,E,F,G,H\}$, $x_i = 1$ if item i is included, and $x_i = 0$ otherwise.

Objective and Constraints:

$$\begin{aligned} \text{Max} \quad & z = 10x_A + 9x_B + 8x_C + 7x_D + 9x_E + 6x_F + 5x_G + 7x_H \\ \text{s.t.} \quad & 4x_A + 2x_B + 3x_C + 1x_D + 5x_E + 1x_F + 4x_G + 0.5x_H \leq 20 \\ & x_A, x_B, x_C, x_D, x_E, x_F, x_G, x_H \in \{0,1\}. \end{aligned}$$

♠

3.3.2. Capital Investment

We next see a similar application of binary variables.

Example 3.10: Capital Allocation Problem Excel PuLP Gurobipy

You are a financial planner for an investment firm. The firm has $100,000 to invest in a portfolio of projects. Each project has a projected return and requires an initial investment. The goal is to maximize the total return of the portfolio while not exceeding the available capital. The following table lists potential projects, their required investments, and their projected returns.

Formulate an LP to maximize the total return of the portfolio while not exceeding the available investment capital.

Project	Investment Required ($)	Projected Return ($)
A	20,000	30,000
B	30,000	40,000
C	50,000	75,000
D	10,000	15,000
E	40,000	60,000

Solution.

Sets:

- Projects: $P = \{A, B, C, D, E\}$

Parameters:

- inv_i: investment required for project i
- ret_i: projected return from project i
- $B = 100{,}000$: total available capital

Decision Variables:
$x_i \in \{0,1\}$: 1 if project i is selected, 0 otherwise

Objective and Constraints:

$$\begin{aligned} \max \quad & z = 30{,}000x_A + 40{,}000x_B + 75{,}000x_C + 15{,}000x_D + 60{,}000x_E \\ \text{s.t.} \quad & 20{,}000x_A + 30{,}000x_B + 50{,}000x_C + 10{,}000x_D + 40{,}000x_E \leq 100{,}000 \\ & x_A, x_B, x_C, x_D, x_E \in \{0,1\} \end{aligned}$$

Compact Form:

$$\begin{aligned} \max \quad & \sum_{i \in P} \text{ret}_i \cdot x_i \\ \text{s.t.} \quad & \sum_{i \in P} \text{inv}_i \cdot x_i \leq B \\ & x_i \in \{0,1\} \quad \forall i \in P \end{aligned}$$

♠

3.3.3. Transportation problem

Example 3.11: Solar Panel Shipping Excel PuLP Gurobipy

SolTranz Inc. manufactures high-efficiency solar panels at three factories located in Reno (R), Boulder (B), and Tucson (T). The maximum production at each facility is 4,500, 5,500, and 7,000 panels, respectively. These panels must be shipped to five warehouses (labeled 1 through 5), each with a fixed demand.

The cost of producing a solar panel depends on the factory, and the shipping costs to each warehouse are also known. However, no more than 2,000 panels can be shipped from a factory to a single warehouse. The table below summarizes the data.

Formulate a linear program that minimizes the total cost of production and shipping to satisfy all warehouse demands.

Factory	Prod. Cost ($)	WH1	WH2	WH3	WH4	WH5
Reno (R)	150	60	70	75	65	50
Boulder (B)	140	80	55	65	70	60
Tucson (T)	130	85	60	55	50	65

Shipping cost from each factory to each warehouse (in $ per panel).

Warehouse Demands:

WH1: 3000 WH2: 3500 WH3: 2500 WH4: 4000 WH5: 3000

Solution.

Sets:

- Factories: $F = \{R, B, T\}$
- Warehouses: $W = \{1, 2, 3, 4, 5\}$

Parameters:

- C_f: production cost per panel at factory f
- S_f: supply capacity of factory f
- D_w: demand at warehouse w
- $T_{f,w}$: shipping cost per panel from factory f to warehouse w
- $M = 2000$: max panels shipped from any factory to any warehouse

Decision Variables:
$x_{f,w}$: number of solar panels shipped from factory f to warehouse w

Objective and Constraints:

$$\begin{aligned}
\min \quad & \sum_{f\in F}\sum_{w\in W}(C_f + T_{f,w})\cdot x_{f,w} \\
\text{s.t.} \quad & \sum_{w\in W} x_{R,w} \le 4500 && \text{(Reno capacity)} \\
& \sum_{w\in W} x_{B,w} \le 5500 && \text{(Boulder capacity)} \\
& \sum_{w\in W} x_{T,w} \le 7000 && \text{(Tucson capacity)} \\
& \sum_{f\in F} x_{f,1} = 3000 && \text{(WH1 demand)} \\
& \sum_{f\in F} x_{f,2} = 3500 && \text{(WH2 demand)} \\
& \sum_{f\in F} x_{f,3} = 2500 && \text{(WH3 demand)} \\
& \sum_{f\in F} x_{f,4} = 4000 && \text{(WH4 demand)} \\
& \sum_{f\in F} x_{f,5} = 3000 && \text{(WH5 demand)} \\
& 0 \le x_{f,w} \le 2000 && \forall f \in F, \forall w \in W
\end{aligned}$$

3.3.4. Assignment Problem

Consider the assignment of 3 employees to 3 tasks, where each employee has a cost to do each ask. How to figure out the best assignment to the tasks?

Example 3.12: Hiring for tasks Excel PuLP Gurobipy

In this assignment problem, we need to hire three people (Person 1, Person 2, Person 3) to three tasks (Task 1, Task 2, Task 3). In the table below, we list the cost of hiring each person for each task, in dollars. Since each person has a different cost for each task, we must make an assignment to minimize our total cost.

Cost	Task 1	Task 2	Task 3
Person 1	40	47	80
Person 2	72	36	58
Person 3	24	61	71

Given the specific costs of assigning three people to three tasks, we can write out the mathematical model explicitly using the given numbers.

Solution.

Sets: Set of tasks and set of persons

Parameters: Cost c_{ij} for person i to complete task j.

Variables: Let $x_{ij} = 1$ if person i is assigned to task j and 0 otherwise.

Objective Function:

Minimize the total cost of assignments:

$$z = 40x_{11} + 47x_{12} + 80x_{13} + 72x_{21} + 36x_{22} + 58x_{23} + 24x_{31} + 61x_{32} + 71x_{33} \tag{3.1}$$

Subject to Constraints:

Each person is assigned to exactly one task:

$$x_{11} + x_{12} + x_{13} = 1 \tag{3.2}$$
$$x_{21} + x_{22} + x_{23} = 1 \tag{3.3}$$
$$x_{31} + x_{32} + x_{33} = 1 \tag{3.4}$$

Each task is assigned to exactly one person:

$$x_{11} + x_{21} + x_{31} = 1 \tag{3.5}$$
$$x_{12} + x_{22} + x_{32} = 1 \tag{3.6}$$
$$x_{13} + x_{23} + x_{33} = 1 \tag{3.7}$$

Binary constraints on the variables:

$$x_{ij} \in \{0,1\} \quad \forall i \in \{1,2,3\}, \forall j \in \{1,2,3\} \tag{3.8}$$

3.4 Exercises

Exercise 3.13 *Which ones are linear functions?*

1. $x^2 + 3y + 2z$
2. $\sqrt{x_1 x_2 x_3}$
3. $\sum_{i=1}^{n} a_i 2^{x_i}$, *where* $a_1, a_2, \ldots, a_n$ *are constant*
4. $\sum_{i=1}^{n} a_i x_i$, *where* $a_1, a_2, \ldots, a_n$ *are constant*

Exercise 3.14 *Which one is a linear function?*

1. $2x_1 + x_2 x_3 - 7x_3$
2. $\sqrt{2}x_1 + x_2 + 3^{1.6} x_3$

Exercise 3.15 *Which ones are linear inequalities? Can any be made into linear inequalities?*

1. $\frac{x_1}{x_1+x_2+x_3} \geq 0.5$
2. $\sin x_1 + x_2 + x_3 = 0.5$
3. $\sum_{i=1}^{n} a_i 2^{x_i} \geq b$, *where* $a_1, a_2, \ldots, a_n, b$ *are constant*
4. $\sum_{i=1}^{n} a_i x_i = b$, *where* $a_1, a_2, \ldots, a_n, b$ *are constant*
5. $\sum_{i=1}^{n} a_i x_i \geq b$, *where* $a_1, a_2, \ldots, a_n, b$ *are constant*

4. Software - Excel

Outcomes

- *Learn relevant excel functions*
- *Setup linear programs in excel*
- *Solve linear programs in excel*

4.1 Using Excel Solver

Excel Solver is a powerful tool for solving optimization problems with linear and nonlinear constraints. Before diving into using Solver, it is essential to understand some basic Excel functions and techniques that are commonly used when setting up optimization problems. These functions help organize data, compute intermediate values, and define objective functions and constraints.

4.1.1. Useful Excel Functions and Techniques

USING FORMULAS IN CELLS In Excel, formulas begin with an equals sign (=). For example, typing `=A1+B1` in a cell will calculate the sum of the values in cells `A1` and `B1`.

`SUM()` The `SUM()` function adds a range of numbers. For example, `=SUM(A1:A10)` calculates the sum of all values in the range `A1` to `A10`.

`SUMPRODUCT()` The `SUMPRODUCT()` function multiplies corresponding elements in two or more arrays and returns the sum of those products. This is particularly useful for computing dot products or weighted sums. For example, `=SUMPRODUCT(A1:A10, B1:B10)` computes $\sum_{i=1}^{10} A_i B_i$.

`COUNT()` The `COUNT()` function counts the number of numeric entries in a range. For example, `=COUNT(A1:A10)` returns the number of numeric cells in the range `A1` to `A10`.

`COUNTIF()` The `COUNTIF()` function counts the number of cells in a range that meet a specific condition. For example, `=COUNTIF(A1:A10, ">5")` counts the number of cells in `A1:A10` with values greater than 5.

DUPLICATING FORMULAS ACROSS ROWS OR COLUMNS To apply a formula to multiple cells, you can drag the fill handle (a small square at the bottom-right corner of the selected cell) across the desired range. Excel automatically adjusts cell references based on the relative position (relative referencing). To keep a reference constant, use the dollar sign (`$`) in the cell reference (absolute referencing). For example, `$A$1` always refers to cell `A1`.

CONDITIONAL FORMATTING Conditional formatting allows you to highlight cells that meet specific criteria. For example, you can format cells with values above a threshold in bold or a particular color to identify important data visually.

DATA VALIDATION Data validation helps restrict the type of data entered in a cell. For example, you can set a rule that only allows numbers within a specific range or text from a predefined list.

NAMING RANGES Naming ranges makes formulas easier to read and maintain. Instead of `=SUM(A1:A10)`, you can name the range `"Sales"` and write `=SUM(Sales)`.

USING LOGICAL FUNCTIONS Logical functions like `IF()`, `AND()`, and `OR()` are helpful for decision-making within formulas. For example, `=IF(A1>10, Ḧigh, L̈ow)` returns Ḧighïf `A1` is greater than 10, and L̈owötherwise.

With these foundational tools, setting up optimization problems becomes much more manageable. In the next section, we will explore how to use these functions in combination with Excel Solver to define and solve optimization models.

Logical functions like `IF()`, `AND()`, and `OR()` are helpful for decision-making within formulas. For example, `=IF(A1>10, Ḧigh, L̈ow)` returns Ḧighïf `A1` is greater than 10, and L̈owötherwise.

With these foundational tools, setting up optimization problems becomes much more manageable. In the next section, we will explore how to use these functions in combination with Excel Solver to define and solve optimization models.

4.1.2. How to Use Excel Solver

Excel Solver is a powerful add-in that allows users to define and solve optimization problems. This section provides a step-by-step guide on how to effectively layout data and use Solver for optimization.

STEP 1: LAYOUT THE DATA Organizing your data clearly is crucial for using Solver effectively. Use color coding to distinguish between different types of cells:

- **Data:** Cells containing fixed input values, such as coefficients or constants, should be highlighted in light blue.
- **Variables:** Cells representing decision variables should be highlighted in yellow.
- **Formulas:** Cells with formulas, such as the objective function or summarizing constraints values should be highlighted in light green.

Other color combinations are also acceptable, as long as they consistently distinguish between these components.

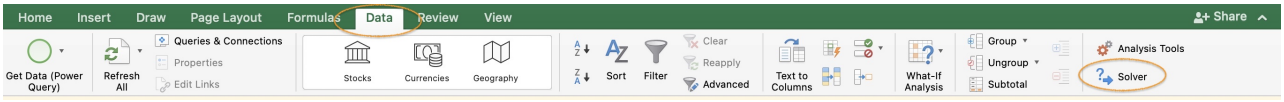
Objective	=SUMPRODUCT(B6:C6,B8:C8)				
Variables	x1	x2			
obj. coeff	7	6			
			Resources Used		
constraint 1	3	1	=SUMPRODUCT($B$6:$C$6,B11:C11)	<=	120
constraint 2	1	2	=SUMPRODUCT($B$6:$C$6,B12:C12)	<=	160
constraint 3	1	0	=SUMPRODUCT($B$6:$C$6,B13:C13)	<=	35

STEP 2: DEFINE THE OBJECTIVE FUNCTION Identify the cell that represents the objective function. This cell should contain a formula that calculates the value to be maximized or minimized, such as total profit or cost.

STEP 3: SET UP CONSTRAINTS Clearly specify the constraints of the problem. Constraints can be represented using formulas that involve the variables, coefficients, and right-hand side values. For example, a constraint ensuring that total production does not exceed capacity can be written as `=SUMPRODUCT(Production, Coefficients) <= Capacity`.

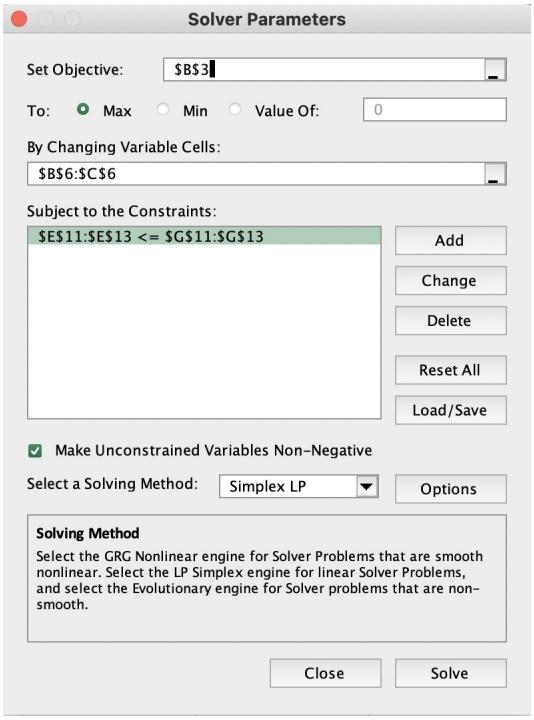
STEP 4: OPEN SOLVER Ensure Solver is enabled in Excel. If not, you can activate it by navigating to `File > Options > Add-Ins > Manage Excel Add-ins` and checking the box for Solver.

Once Solver is enabled, go to `Data > Solver` to open the Solver Parameters dialog box.



STEP 5: DEFINE SOLVER PARAMETERS In the Solver Parameters dialog box:

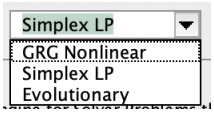
- Set the **Objective** to the cell containing the objective function.
- Choose **Max** or **Min** to indicate whether the objective is to be maximized or minimized.
- Specify the **Variable Cells** by selecting the range of cells representing decision variables.
- Add **Constraints** by specifying the relationship (e.g., <=, >=, =) and the cells involved.
- When the variables are non-negative, you can select `Make Unconstrainted Variables Non-Negative`.



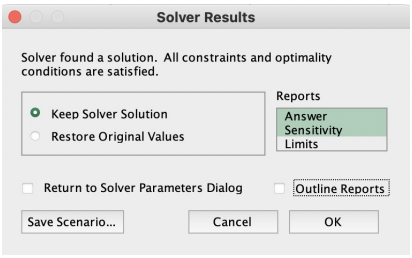
STEP 6: CHOOSE A SOLVING METHOD Solver provides three solving methods:

- **GRG Nonlinear:** For smooth nonlinear problems.
- **Simplex LP:** For linear programming problems.
- **Evolutionary:** For non-smooth problems or those with integer constraints.

Select the method that matches the problem type.

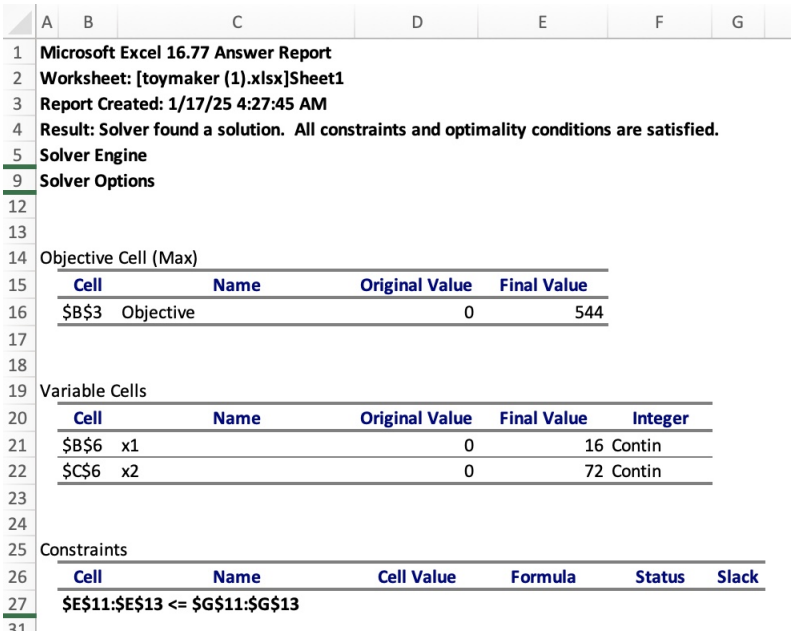


STEP 7: SOLVE THE PROBLEM Click `Solve` to run Solver. If Solver finds a solution, it will display the results and allow you to keep or discard them. Analyze the solution to ensure it meets the problem's requirements.



STEP 8: INTERPRET THE RESULTS Review the values of the decision variables and the objective function. Check that all constraints are satisfied and assess the solution's feasibility and quality.

- Answer report



	A	B	C	D	E	F	G
1	**Microsoft Excel 16.77 Answer Report**						
2	**Worksheet: [toymaker (1).xlsx]Sheet1**						
3	**Report Created: 1/17/25 4:27:45 AM**						
4	**Result: Solver found a solution. All constraints and optimality conditions are satisfied.**						
5	**Solver Engine**						
9	**Solver Options**						
12							
13							
14	Objective Cell (Max)						
15		**Cell**	**Name**	**Original Value**	**Final Value**		
16		$B$3	Objective	0	544		
17							
18							
19	Variable Cells						
20		**Cell**	**Name**	**Original Value**	**Final Value**	**Integer**	
21		$B$6	x1	0	16	Contin	
22		$C$6	x2	0	72	Contin	
23							
24							
25	Constraints						
26		**Cell**	**Name**	**Cell Value**	**Formula**	**Status**	**Slack**
27		**$E$11:$E$13 <= $G$11:$G$13**					

- Sensitivity report

Microsoft Excel 16.77 Sensitivity Report
Worksheet: [toymaker (1).xlsx]Sheet1
Report Created: 1/17/25 4:27:46 AM

Variable Cells

Cell	Name	Final Value	Reduced Cost	Objective Coefficient	Allowable Increase	Allowable Decrease
$B$6	x1	16	0	7	11	4
$C$6	x2	72	0	6	8	3.666666667

Constraints

Cell	Name	Final Value	Shadow Price	Constraint R.H. Side	Allowable Increase	Allowable Decrease
$E$11:$E$13	<= $G$11:$G$13					

- Solution on spreadsheet

	A	B	C	D	E	F	G
3	Objective	544					
5	Variables	x1	x2				
6		16	72				
8	obj. coeff	7	6				
10					Resources Used		
11	constraint 1	3	1		120	<=	120
12	constraint 2	1	2		160	<=	160
13	constraint 3	1	0		16	<=	35

Resources

Examples and guides

- *Installing Excel Solver on Windows*
- `https://www.excel-easy.com/data-analysis/solver.html`
- *Excel Solver - Introduction on Youtube*
- *Some notes from MIT*

Some videos

- *Solving a linear program*
- *Optimal product mix*
- *Set Cover*
- *Introduction to Designing Optimization Models Using Excel Solver*
- *Traveling Salesman Problem*
- *Also Travelin Salesman Problem*
- *Multiple Traveling Salesman Problem*
- *Shortest Path*

Some other links

- *Loan Example*
- *Several Examples including TSP*

4.2 Exercises

Exercise 4.1 *Consider the following table indicating the nutritional value of different food types:*

	Price (\$) *per serving*	*Calories per serving*	*Fat* (g) *per serving*	*Protein* (g) *per serving*	*Carbohydrate* (g) *per serving*
Raws carrots	0.14	*23*	0.1	0.6	*6*
Baked potatoes	0.12	*171*	0.2	3.7	*30*
Wheat bread	0.2	*65*	*0*	2.2	*13*
Cheddar cheese	0.75	*112*	9.3	*7*	*0*
Peanut butter	0.15	*188*	*16*	7.7	*2*

You need to decide how many servings of each food to buy each day so that you minimize the total cost of buying your food while satisfying the following daily nutritional requirements:

- *calories must be at least 2000,*
- *fat must be at least* 50 g,
- *protein must be at least* 100 g,
- *carbohydrates must be at least* 250 g.

Write an LP that will help you decide how many servings of each of the aforementioned foods are needed to meet all the nutritional requirement, while minimizing the total cost of the food (you may buy fractional numbers of servings).

Solve this problem using Excel Solver. Provide a screenshot of your spreadsheet with the optimal solution.

5. Modeling with Compact Notation

Outcomes

- *Review summation notation*
- *Work on Sets, Parameters, Variables, and Model*
- *Write compact formulations for linear programs*

A Brief Review of Summation Notation and Its Relation to Vector Products

Summation notation is a concise way to express the addition of a series of terms. It is commonly written as

$$\sum_{i=m}^{n} \text{expression involving } i,$$

where i is the index of summation, m is the starting index, and n is the ending index.

1. Sum of x_i:

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n.$$

In vector terminology, if we let $\mathbf{x}$ be an n-dimensional vector,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix},$$

then $\sum_{i=1}^{n} x_i$ represents the sum of all components of $\mathbf{x}$.

2. Sum of $a_i x_i$:

$$\sum_{i=1}^{n} a_i x_i = a_1 x_1 + a_2 x_2 + \cdots + a_n x_n.$$

This expression is the dot product (or inner product) of two n-dimensional vectors $\mathbf{a}$ and $\mathbf{x}$, where

$$\mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}.$$

In vector notation, the dot product is written as:

$$\mathbf{a}^\top \mathbf{x} = \mathbf{a} \cdot \mathbf{x} = \sum_{i=1}^{n} a_i x_i.$$

This product yields a scalar value that combines the corresponding components of the two vectors.

5.0.1. Notes to expand on

SETS. . Sets are important to help see what constraints and variables might iterating over.

We typically denote sets with capital letters and the index as the corresponding lower case letter.

For example:

Let I be the set of months in the year.

then we will index the months with the index i.

If

$$I = \{1,2,3,4,5,6,7,8,9,10,11,12\}$$

then we can write a summation over this index set in a few ways:

$$\sum_{i \in I} a_i x_i \quad \text{or} \quad \sum_{i=1}^{1} 2a_i x_i.$$

Suppose we have another set $J = \{'red','green','blue'\}$.

And suppse we have a variable y_{ij} indexed by both $i \in I$ and $j \in J$.

Consider for each color $j \in J$, we must sum over all the months and set that equal to 1. We can write this as

$$\sum_{i \in I} y_{ij} = 1 \quad \forall j \in J.$$

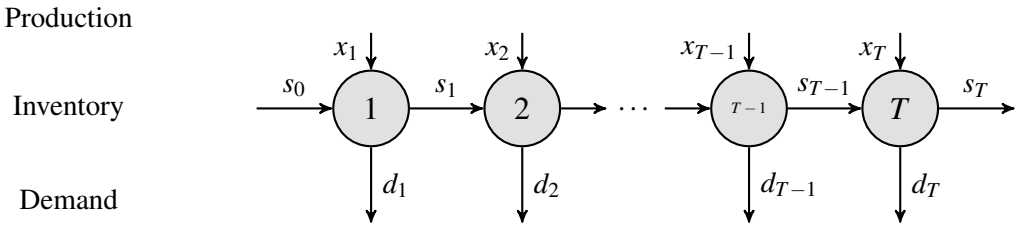
What does this encapsulate? It is shorthand for 3 constraints where in each constraint 12 variables are summed to equal 1.

IMPORTANT! . Every index used needs to appear in either a sum or a forall statement.

In the above example, we use the variable y_{ij} in the constraint. This means that in order to make the expression make sense mathematically, we need to define what i and j mean in the constraint. This is done by the summation $\sum_{i \in I}$ iterating over the i index, while the $\forall j \in J$ iterates over the j index.

5.1 Production Planning Models

Production Planning Model (for T Periods)



Sets:

- $t \in \{1, 2, \ldots, T\}$: Periods in the planning horizon.

Parameters:

- c_t^{prod}: Production cost per unit in period t (for $t = 1, \ldots, T$).
- c^{inv}: Inventory holding cost per unit carried from one period to the next.
- d_t: Demand in period t.
- s_0: Initial inventory available at the start of period 1.

Decision Variables:

- x_t: Number of units produced in period t.
- s_t: Inventory level at the end of period t.

Objective and Constraints:

$$\begin{aligned}
\min_{x,s} \quad & z = \sum_{t=1}^{T} \left(c_t^{prod} x_t + c^{inv} s_t \right) \quad \text{(Total Cost)} \\
\text{s.t.} \quad & s_0 + x_1 = d_1 + s_1, \quad \text{(Period 1 Balance)} \\
& s_{t-1} + x_t = d_t + s_t, \quad \forall t = 2, \ldots, T \quad \text{(Inventory Balance)} \\
& x_t \geq 0, \quad s_t \geq 0, \quad \forall t = 1, \ldots, T \quad \text{(Non-negativity)}
\end{aligned}$$

Example 5.1: Production Planning with 10 Periods ($T = 10$) Excel PuLP Gurobipy

Initial Inventory:

$$s_0 = 5 \quad \text{units}$$

Data: (Download the data)

Parameter	1	2	3	4	5	6	7	8	9	10
Production Cost ($ per unit)	10	12	14	16	18	20	22	24	26	28
Demand (units)	8	6	9	7	10	5	8	6	4	7

Inventory Holding Cost:

$$c^{inv} = 5 \quad \text{per unit}$$

Decision Variables:

- x_t: Number of units produced in period t.
- s_t: Inventory level at the end of period t.

Mathematical Formulation:

$$\begin{aligned}
\min_{x,s} \quad & z = 10x_1 + 12x_2 + 14x_3 + 16x_4 + 18x_5 + 20x_6 + 22x_7 + 24x_8 + 26x_9 + 28x_{10} \\
& \quad + 5\,(s_1 + s_2 + s_3 + s_4 + s_5 + s_6 + s_7 + s_8 + s_9 + s_{10}) \\
\text{s.t.} \quad & 5 + x_1 = 8 + s_1, \quad \text{(Period 1)} \\
& s_1 + x_2 = 6 + s_2, \quad \text{(Period 2)} \\
& s_2 + x_3 = 9 + s_3, \quad \text{(Period 3)} \\
& s_3 + x_4 = 7 + s_4, \quad \text{(Period 4)} \\
& s_4 + x_5 = 10 + s_5, \quad \text{(Period 5)} \\
& s_5 + x_6 = 5 + s_6, \quad \text{(Period 6)} \\
& s_6 + x_7 = 8 + s_7, \quad \text{(Period 7)} \\
& s_7 + x_8 = 6 + s_8, \quad \text{(Period 8)} \\
& s_8 + x_9 = 4 + s_9, \quad \text{(Period 9)} \\
& s_9 + x_{10} = 7 + s_{10}, \quad \text{(Period 10)} \\
& x_t \geq 0, \quad s_t \geq 0, \quad \forall t = 1, \ldots, 10.
\end{aligned}$$

Production Planning with Overtime

A manufacturing firm seeks to optimize its production schedule over a T-day planning horizon to meet daily demand while minimizing costs. The firm starts with an initial inventory of s_0 units. Production can be carried out during regular time and via overtime. The costs are as follows:

- **Regular production cost:** c_t^{prod} dollars per unit on day t.
- **Overtime production cost:** c_t^{over} dollars per unit on day t.
- **Inventory cost:** c^{inv} dollars per unit carried over from one day to the next.

The goal is to determine the number of units to produce during regular time and overtime on each day, and the inventory levels, in order to minimize the total cost while satisfying daily demand.

Sets:

- Days of production $= \{1, 2, \ldots, T\}$.

Parameters:

- c_t^{prod}: Regular production cost per unit on day t (for $t = 1, \ldots, T$).
- c_t^{over}: Overtime production cost per unit on day t (for $t = 1, \ldots, T$).
- c^{inv}: Inventory holding cost per unit carried to the next day.
- d_t: Demand on day t.
- s_0: Initial inventory at the start of day 1.

Decision Variables:

- x_t: Number of units produced in regular time on day t.
- y_t: Number of units produced in overtime on day t.
- s_t: Inventory at the end of day t.

Objective and Constraints:

$$\begin{aligned}
\min \quad & z = \sum_{t=1}^{T} \left(c_t^{prod} x_t + c_t^{over} y_t + c^{inv} s_t \right) \quad \text{(Total Cost)} \\
\text{s.t.} \quad & s_0 + x_1 + y_1 = d_1 + s_1 \quad \text{(Day 1 balance)} \\
& s_{t-1} + x_t + y_t = d_t + s_t, \quad t = 2, \ldots, T \quad \text{(Inventory balance)} \\
& x_t, y_t, s_t \geq 0, \quad t = 1, \ldots, T \quad \text{(Non-negativity).}
\end{aligned}$$

Example 5.2: Excel PuLP Gurobipy

Consider a 10-day planning horizon ($T = 10$) with the following data:

Data:

Parameter	1	2	3	4	5	6	7	8	9	10
Regular Production Cost ($)	10	12	14	16	18	20	22	24	26	28
Overtime Production Cost ($)	16	18	20	22	24	26	28	30	32	34
Demand (units)	8	6	9	7	10	5	8	6	4	7

data

Assume the following additional parameters:

- Inventory cost: $c^{inv} = 5$ dollars per unit.
- Initial inventory: $s_0 = 0$ units.

Decision Variables:

- x_t: Regular production on day t.
- y_t: Overtime production on day t.
- s_t: Inventory at the end of day t.

Mathematical Formulation:

$$
\begin{aligned}
\min \quad & z = 10x_1 + 16y_1 + 5s_1 \\
& \quad + 12x_2 + 18y_2 + 5s_2 \\
& \quad + 14x_3 + 20y_3 + 5s_3 \\
& \quad + 16x_4 + 22y_4 + 5s_4 \\
& \quad + 18x_5 + 24y_5 + 5s_5 \\
& \quad + 20x_6 + 26y_6 + 5s_6 \\
& \quad + 22x_7 + 28y_7 + 5s_7 \\
& \quad + 24x_8 + 30y_8 + 5s_8 \\
& \quad + 26x_9 + 32y_9 + 5s_9 \\
& \quad + 28x_{10} + 34y_{10} + 5s_{10} \\
\text{s.t.} \quad & s_0 + x_1 + y_1 = 8 + s_1 \quad \text{(Day 1 balance)} \\
& s_1 + x_2 + y_2 = 6 + s_2 \quad \text{(Day 2 balance)} \\
& s_2 + x_3 + y_3 = 9 + s_3 \quad \text{(Day 3 balance)} \\
& s_3 + x_4 + y_4 = 7 + s_4 \quad \text{(Day 4 balance)} \\
& s_4 + x_5 + y_5 = 10 + s_5 \quad \text{(Day 5 balance)} \\
& s_5 + x_6 + y_6 = 5 + s_6 \quad \text{(Day 6 balance)} \\
& s_6 + x_7 + y_7 = 8 + s_7 \quad \text{(Day 7 balance)} \\
& s_7 + x_8 + y_8 = 6 + s_8 \quad \text{(Day 8 balance)} \\
& s_8 + x_9 + y_9 = 4 + s_9 \quad \text{(Day 9 balance)} \\
& s_9 + x_{10} + y_{10} = 7 + s_{10} \quad \text{(Day 10 balance)} \\
& x_t, y_t, s_t \geq 0, \quad t = 1, \ldots, 10.
\end{aligned}
$$

In this 10-day example, the decision is how many units to produce on regular time (x_t) and overtime (y_t) on each day, and what the end-of-day inventory levels (s_t) should be, so as to satisfy daily demand at minimum total cost.

5.2 Assignment Problem

The *assignment problem* (machine/person to job/task assignment) seeks to assign tasks to machines in a way that is most efficient. This problem can be thought of as having a set of machines that can complete various tasks (textile machines that can make t-shirts, pants, socks, etc) that require different amounts of time to complete each task, and given a demand, you need to decide how to alloacte your machines to tasks.

Alternatively, you could be an employer with a set of jobs to complete and a list of employees to assign to these jobs. Each employee has various abilities, and hence, can complete jobs in differing amounts of time. And each employee's time might cost a different amout. How should you assign your employees to jobs in order to minimize your total costs?

Assignment Problem:

Given m machines and n jobs, find a least cost assignment of jobs to machines. The cost of assigning job j to machine i is c_{ij}.

Example 5.3: Machine Assignment Gurobipy

Sets:

- Let $I = \{0,1,2,3\}$ set of machines.
- Let $J = \{0,1,2,3\}$ be the set of tasks.

Parameters:

- c_{ij} - the cost of assigning machine i to job j

Variables:

- Let

$$x_{ij} = \begin{cases} 1 & \text{if machine } i \text{ assigned to job } j \\ 0 & \text{otherwise.} \end{cases}$$

Model:

$$\begin{aligned}
\min \quad & \sum_{i \in I, j \in J} c_{ij} x_{ij} && && \text{(Minimize cost)} \\
s.t. \quad & \sum_{i \in I} x_{ij} = 1 && \text{for all } j \in J && \text{(All jobs are assigned one machine)} \\
& \sum_{j \in J} x_{ij} = 1 && \text{for all } i \in I && \text{(All machines are assigned to a job)} \\
& x_{ij} \in \{0,1\} \forall i \in I, j \in J
\end{aligned}$$

Example 5.4: School Bus Routing Problem PuLP

A school district has a set of schools I and a fleet of school buses J. Each bus needs to be assigned to a school every morning to pick up students. The cost c_{ij} represents the fuel cost for bus j to reach school i and complete the route. The district aims to minimize the total fuel cost while ensuring that each school is served by exactly one bus and each bus is assigned to exactly one school.

The fuel costs can be found in the following table. They are also in csv file format here.

Bus/School	School A	School B	School C	School D	School E
Bus 1	50	80	60	90	100
Bus 2	60	85	55	70	110
Bus 3	75	65	50	85	90
Bus 4	70	90	55	80	95
Bus 5	80	70	60	75	85

We can model this as follows:

Indices:

- i - Index for schools, $i \in I$
- j - Index for buses, $j \in J$

Parameters:

- c_{ij} - Fuel cost for bus j to serve school i.

Decision Variables:

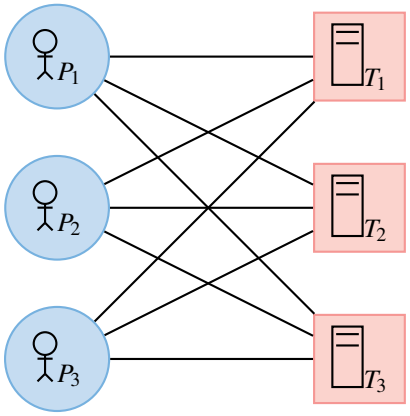
- x_{ij} - 1 if bus j is assigned to school i, 0 otherwise.

Model:

$$\begin{aligned}
\min \quad & \sum_{i \in I, j \in J} c_{ij} x_{ij} && && \text{(Minimize total fuel cost)} \\
s.t. \quad & \sum_{i \in I} x_{ij} = 1 && \text{for all } j \in J && \text{(Each bus is assigned to one school)} \\
& \sum_{j \in J} x_{ij} = 1 && \text{for all } i \in I && \text{(Each school is served by one bus)} \\
& x_{ij} \in \{0,1\} \forall i \in I, j \in J
\end{aligned}$$

5.3 Assignment Problem

The **assignment problem** is a fundamental optimization problem in combinatorial optimization. It involves assigning a set of n agents (or workers) to n tasks (or jobs) such that each agent is assigned exactly one task, and each task is assigned to exactly one agent. The goal is to minimize the total assignment cost.



Sets:

- I – Set of agents (or workers), indexed by i.
- J – Set of tasks (or jobs), indexed by j.

Parameters:

- C_{ij} – Cost associated with assigning agent i to task j.

Decision Variables:

- x_{ij} – Binary decision variable, where:

$$x_{ij} = \begin{cases} 1, & \text{if agent } i \text{ is assigned to task } j, \\ 0, & \text{otherwise.} \end{cases}$$

Mathematical Model:

$$\min \quad \sum_{i \in I} \sum_{j \in J} C_{ij} x_{ij} \tag{5.1}$$

$$\text{s.t.} \quad \sum_{j \in J} x_{ij} = 1, \qquad \forall i \in I \quad \text{(Each agent gets exactly one task)} \tag{5.2}$$

$$\sum_{i \in I} x_{ij} = 1, \qquad \forall j \in J \quad \text{(Each task is assigned to exactly one agent)} \tag{5.3}$$

$$x_{ij} \in \{0,1\}, \qquad \forall i \in I,\ j \in J. \tag{5.4}$$

Explanation:

- **Objective function** (5.1): Minimizes the total assignment cost based on given costs C_{ij}.
- **Constraints** (5.2): Each agent is assigned to exactly one task.
- **Constraints** (5.3): Each task is assigned to exactly one agent.
- **Binary constraint** (5.4): Ensures each assignment is either made ($x_{ij} = 1$) or not ($x_{ij} = 0$).

This problem can be efficiently solved using algorithms such as the **Hungarian method** for the classical case or integer programming solvers like **branch and bound** for larger instances.

Example 5.5: Hiring for tasks Excel PuLP Gurobipy

Recall Example 3.12. We define the following sets, parameters, and variables to construct the mathematical model.

Sets:

- $I = \{1,2,3\}$, the set of people.
- $J = \{1,2,3\}$, the set of tasks.

Parameters:

- C_{ij}, the cost of assigning person $i \in I$ to task $j \in J$. The costs are given in the following table:

$$C = \begin{bmatrix} 40 & 47 & 80 \\ 72 & 36 & 58 \\ 24 & 61 & 71 \end{bmatrix}$$

Variables:

- $x_{ij} = \begin{cases} 1 & \text{if person } i \text{ is assigned to task } j \\ 0 & \text{otherwise} \end{cases}$ for all $i \in I, j \in J$.

Model:

The objective is to minimize the total cost of assignments:

$$\text{Minimize} \quad z = \sum_{i \in I} \sum_{j \in J} C_{ij} x_{ij} \tag{5.5}$$

Subject to the constraints:

1. Each person is assigned to exactly one task:

$$\sum_{j \in J} x_{ij} = 1 \quad \forall i \in I \tag{5.6}$$

2. Each task is assigned to exactly one person:

$$\sum_{i \in I} x_{ij} = 1 \quad \forall j \in J \tag{5.7}$$

3. Binary constraints on the variables:

$$x_{ij} \in \{0,1\} \quad \forall i \in I, \forall j \in J \tag{5.8}$$

This model ensures that each person is assigned to exactly one task, each task is assigned to exactly one person, and the total cost of the assignments is minimized.

The assignment problem has an integrality property, such that if we remove the binary restriction on the x variables (now just non-negative, i.e., $x_{ij} \geq 0$) then we still get binary assignments, despite the fact that it is now an LP. This property is very interesting and useful.

Of course, the objective function might not quite what we want, we might be interested ensuring that the team with the worst assignment is as good as possible (a fairness criteria). One way of doing this is to modify the assignment problem using a max-min objective:

Min-max Assignment-like Formulation

$$\begin{aligned}
&\min z \\
&\text{s.t. } z \geq \sum_{i \in I} C_{ij} x_{ij}, && \forall j \in J \\
&\sum_{i \in I} x_{ij} = 1, && \forall j \in J \\
&\sum_{j \in J} x_{ij} = 1, && \forall i \in I \\
&x_{ij} \geq 0, && \forall i \in I,\ j \in J
\end{aligned}$$

Does this formulation have the integrality property? (It is not an assignment problem.)

Consider a simple example where two teams are to be assigned to two projects, and the teams provide the following rankings (lower ranking is better):

	Project 1	Project 2
Team 1	2	1
Team 2	2	1

Both teams prefer Project 2.

If we remove the binary restriction on the x-variable, the values can range between 0 and 1. For the original assignment problem, the optimal solution has $z = 2$, and fractional x-values do not improve z.

For the min-max assignment problem, however, this is not the case. The optimal solution has $z = 1.5$, occurring when each team is assigned half of each project (i.e., for Team 1, we have $x_{11} = 0.5$ and $x_{21} = 0.5$). By fractionalizing assignments, the worst-case project cost is minimized.

Case Study:
Preference-Based Housing Assignment in Uruguayan Cooperatives

Housing cooperatives in Uruguay play a crucial role in providing affordable housing to their members. Traditionally, once a new cooperative building was completed, the assignment of individual housing units was conducted through a random draw. This method, while simple, ignored individual preferences such as sunlight orientation, floor level, and proximity to shared facilities. Recognizing the potential for improved member satisfaction, researchers developed an alternative assignment method based on **mixed-integer programming (MIP)**.

The Challenge

In Uruguayan housing cooperatives, every member is entitled to a housing unit. Since the assignment process happens only once per building, it is crucial to ensure fairness while maximizing overall satisfaction. The existing lottery-based system failed to account for members' preferences, sometimes leading to dissatisfaction and informal exchanges of units after the assignment. The challenge was to develop a new assignment process that:

- Ensured fairness among all members,
- Incorporated individual preferences,
- Was easy to explain and implement,
- Could be executed in real time during cooperative meetings.

The Solution: A Two-Stage Optimization Model

A team of researchers proposed a **two-stage mixed-integer programming (MIP) model** to address these concerns. The assignment problem was formulated with the following structure:
Stage 1 – Ensuring Fairness: The model maximizes the satisfaction of the least satisfied member. This ensures that no one is assigned an extremely undesirable unit.
Stage 2 – Optimizing Global Satisfaction: Given the fairness constraints from Stage 1, the model then seeks to optimize the total satisfaction across all cooperative members.
The assignment process was implemented using a software tool called **MTAV (Mejor Tecnología de Asignación de Viviendas – A Better Technology for Housing Assignment)**, designed to be transparent and user-friendly.

Implementation and Results

The first real-world test of the model occurred in **2016** with the **Virazón housing cooperative**, consisting of **50 members**. Before deployment, the cooperative members reviewed and voted to approve the new method. Their individual housing preferences were collected and input into the system.
On the day of the assignment:

- The housing allocation was carried out in **real-time** during a cooperative meeting.

- The results were projected on a large screen for transparency.
- A notary was present to certify the fairness of the process.
- The optimization model took only a few **seconds** to compute the assignments.

The outcomes were striking:

- **Four-bedroom apartments:** Both members received their first-choice units.
- **Three-bedroom apartments:** More than 37% of members received their first-choice unit, and the average satisfaction ranking was significantly better than a random assignment.
- **Two-bedroom apartments:** Over **half** of the members received one of their top four choices, with all assignments being significantly better than expected under a random draw.

The cooperative members overwhelmingly supported the new system, reporting higher satisfaction compared to previous assignment methods.

Broader Impact

Since the successful implementation at Virazón, **over 30 housing cooperatives** in Uruguay have adopted the model. Further developments have included:

- The introduction of a **graphical user interface** to make data input more intuitive.
- Support for **partial order preferences**, allowing members to indicate that they have no strong preference between certain units.
- Exploration of additional fairness criteria and trade-offs between individual and collective satisfaction.

Beyond its technical achievements, this case study highlights how **operations research techniques** can be applied to solve real-world social challenges. By integrating mathematical optimization with participatory decision-making, the project has significantly improved the way housing is assigned in cooperative communities, setting a precedent for future applications in housing policy and beyond.

Optimization models

The optimization model required the following input parameters and variables:

Sets:

- I: Set of cooperative members needing housing assignments.
- J: Set of available housing units.

Parameters:

- p_{ij}: Preference score of member i for housing unit j, where lower values indicate higher preference (e.g., 1 = most preferred).
- Some versions of the model supported **partial order preferences**, allowing members to express equal preference for multiple units.

Decision Variables:

- $x_{ij} \in \{0,1\}$: Binary variable indicating whether member i is assigned to unit j:

$$x_{ij} = \begin{cases} 1, & \text{if member } i \text{ is assigned to unit } j \\ 0, & \text{otherwise} \end{cases}$$

Two-Stage Optimization Approach

Stage 1: Ensuring Fairness

- Define S as the satisfaction level of the least satisfied member:

$$S = \min_{i \in I} \sum_{j \in J} p_{ij} x_{ij}$$

- Ensure each member is assigned exactly one unit and each unit is assigned to exactly one member.

This is achieved by the model

$$S = \min z \tag{5.9}$$

$$\text{s.t. } z \geq \sum_{j \in J} p_{ij} x_{ij}, \qquad \forall i \in I \tag{5.10}$$

$$\sum_{j \in J} x_{ij} = 1, \qquad \forall i \in I \tag{5.11}$$

$$\sum_{i \in I} x_{ij} = 1, \qquad \forall j \in J \tag{5.12}$$

$$x_{ij} \in \{0,1\}, \qquad \forall i \in I,\ j \in J \tag{5.13}$$

$$z \in \mathbb{R} \tag{5.14}$$

Stage 2: Optimizing Global Satisfaction

- Given the fairness threshold S from Stage 1, optimize total satisfaction:

$$\min \sum_{i \in I} \sum_{j \in J} p_{ij} x_{ij}$$

- Subject to the fairness constraint:

$$\sum_{j \in J} p_{ij} x_{ij} \leq S, \quad \forall i \in I$$

- Ensures that no member receives a unit less desirable than the worst outcome allowed by Stage 1.

This is acheived by the model

$$\min \sum_{i \in I} \sum_{j \in J} p_{ij} x_{ij} \tag{5.15}$$

$$\text{s.t. } \sum_{j \in J} x_{ij} = 1, \qquad \forall i \in I \tag{5.16}$$

$$\sum_{i \in I} x_{ij} = 1, \qquad \forall j \in J \tag{5.17}$$

$$\sum_{j \in J} p_{ij} x_{ij} \leq S, \qquad \forall i \in I \tag{5.18}$$

$$x_{ij} \in \{0,1\}, \qquad \forall i \in I,\ j \in J \tag{5.19}$$

Implementation Notes

- The model was formulated as a **mixed-integer programming (MIP)** model.
- The assignment was computed in **real-time** during cooperative meetings.
- The **MTAV** software was used to collect user preferences and compute assignments efficiently.
- The system ensured transparency, allowing cooperative members to verify their preferences before the final optimization.

References

- IFORS Newsletter December 2024 - Section 4: OR Impact
- M. Prino, E. Sánchez, H. Cancela. "Optimal distribution of habitational units in a cooperative: A mathematical application to optimize satisfaction" (text in Spanish). Proceedings of CLEI 2016 (Conferencia Latinoamericana de Informática), 2016. https://doi.org/10.1109/CLEI.2016.7833357

5.4 Modeling Tricks

5.4.1. Maximizing a minimum

When the constraints could be general, we will write $x \in X$ to define general constraints. For instance, we could have $X = \{x \in \mathbb{R}^n : Ax \leq b\}$ of $X = \{x \in \mathbb{R}^n : Ax \leq b, x \in \mathbb{Z}^n\}$ or many other possibilities.

Consider the problem

$$\begin{aligned} \max \quad & \min\{x_1, \ldots, x_n\} \\ \text{such that} \quad & x \in X \end{aligned}$$

Having the minimum on the inside is inconvenient. To remove this, we just define a new variable y and enforce that $y \leq x_i$ and then we maximize y. Since we are maximizing y, it will take the value of the smallest x_i. Thus, we can recast the problem as

$$\begin{aligned} \max \quad & y \\ \text{such that} \quad & y \leq x_i \text{ for } i = 1, \ldots, n \\ & x \in X \end{aligned}$$

Example 5.1: Minimizing an Absolute Value

Note that

$$|t| = \max(t, -t),$$

Thus, if we need to minimize $|t|$ *we can instead write*

$$\begin{aligned} \min \quad & z && (5.1)\\ \textit{such that} \quad & t \leq z && (5.2)\\ & -t \leq z. && (5.3) \end{aligned}$$

5.5 Network Flow Models and Applications

Network flow models are used to optimize the movement of commodities, resources, or information through a network while satisfying capacity and demand constraints. These models have a broad range of applications in various fields:

- **Transportation and Logistics**: Optimizing the movement of goods through a supply chain while minimizing transportation costs.
- **Telecommunications**: Managing data traffic flow in networks to minimize congestion and latency.
- **Water and Energy Distribution**: Balancing water flow in pipeline systems or optimizing power grid distribution to minimize energy loss.
- **Project Scheduling**: Allocating tasks to workers while minimizing delays and costs.
- **Assignment and Matching Problems**: Assigning resources to tasks, such as employees to projects or students to schools, in an optimal manner.

To begin a discussion on Network flow, we first need to discuss graphs.

5.5.1. Graphs

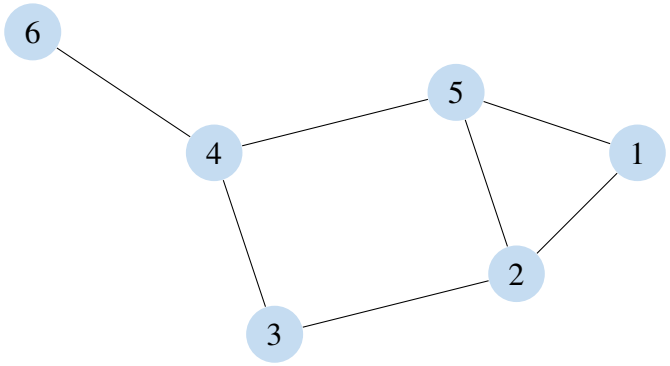
5.5.1.1. Undirected Graphs

Definition 5.2

A (undirected) graph $G = (V, E)$ *is defined by a set of vertices* V *and a set of edges* E *that contains pairs of vertices.*

For example, the following graph G can be described by:

- the vertex set $V = \{1,2,3,4,5,6\}$ and
- the edge set $E = \{(4,6),(4,5),(5,1)(1,2),(2,5),(2,3),(3,4)\}$.



In an undirected graph, we do not distinguish the direction of the edge. That is, for two vertices $i, j \in V$, we can equivalently write (i, j) or (j,i) to represent the edge.

5.5.1.2. Directed Graphs

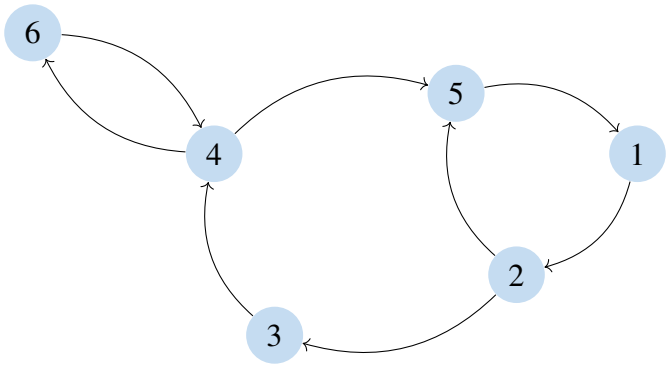
Alternatively, we will want to consider directed graphs.

Definition 5.3

A directed graph *(or di-graph for short) is denoted as $G = (V,A)$ where V is a set of vertices and A is a set of ordred pairs of vertices called arcs. That is, an arc is like an edge, but the direction of it matters.*

For example, the following directed graph G can be described by

- the vertex set $V = \{1,2,3,4,5,6\}$ and
- the arc set $A = \{(4,6),(6,4),(4,5),(5,1)(1,2),(2,5),(2,3),(3,4)\}$.



5.5.2. Example: Minimum-Cost Flow in a Transportation Network

Example 5.6: Excel PuLP Gurobipy

To illustrate the minimum-cost network flow problem, consider a company that wants to transport goods from two warehouses to two retail stores at the lowest possible cost. The company can send goods through a network of transportation routes, each with a given capacity and per-unit transportation cost.

The transportation network consists of:

- **Two warehouses** (supply nodes) that store goods:
  - **Warehouse 1** can supply up to 20 units.
  - **Warehouse 2** can supply up to 30 units.
- **Two retail stores** (demand nodes) that require goods:
  - **Store 1** requires 25 units.
  - **Store 2** requires 25 units.
- Transportation routes (**arcs**) connecting warehouses to stores, each with a transportation cost per unit and capacity limit.

Graph Representation

The transportation network can be represented as a directed graph $G = (V, A)$, where:

- $V = \{W_1, W_2, S_1, S_2\}$ (two warehouses and two stores).
- $A = \{(W_1, S_1), (W_1, S_2), (W_2, S_1), (W_2, S_2)\}$ (directed transportation routes).
- Each arc (i, j) has a given transportation cost C_{ij} and capacity u_{ij}.

The network data is summarized in the following table:

Route	Cost per unit (c_{ij})	Capacity (u_{ij})
Warehouse 1 to Store 1	4	15
Warehouse 1 to Store 2	6	10
Warehouse 2 to Store 1	5	20
Warehouse 2 to Store 2	2	20

Mathematical Formulation

Let x_{ij} represent the amount of goods transported from warehouse i to store j. The problem is formulated as:

$$\begin{aligned}
\min \quad & 4x_{W_1,S_1} + 6x_{W_1,S_2} + 5x_{W_2,S_1} + 2x_{W_2,S_2} && && (5.1)\\
\text{s.t.} \quad & x_{W_1,S_1} + x_{W_1,S_2} \leq 20, && \text{(Supply constraint for Warehouse 1)} && (5.2)\\
& x_{W_2,S_1} + x_{W_2,S_2} \leq 30, && \text{(Supply constraint for Warehouse 2)} && (5.3)\\
& x_{W_1,S_1} + x_{W_2,S_1} = 25, && \text{(Demand constraint for Store 1)} && (5.4)\\
& x_{W_1,S_2} + x_{W_2,S_2} = 25, && \text{(Demand constraint for Store 2)} && (5.5)\\
& 0 \leq x_{ij} \leq u_{ij}, && \forall (i,j) \in A. && (5.6)
\end{aligned}$$

Solution

Solving the linear program, we obtain the following optimal flow values:

Route	Optimal Flow (x_{ij})
Warehouse 1 to Store 1	15
Warehouse 1 to Store 2	5
Warehouse 2 to Store 1	10
Warehouse 2 to Store 2	20

The total minimum cost is given by:

$$(4 \times 15) + (6 \times 5) + (5 \times 10) + (2 \times 20) = 60 + 30 + 50 + 40 = 180.$$

Interpretation of the Solution

- **Warehouse 1** sends **15 units** to Store 1 and **5 units** to Store 2.
- **Warehouse 2** sends **10 units** to Store 1 and **20 units** to Store 2.
- The **total transportation cost is minimized to 180**.

This solution meets all constraints while ensuring goods are transported at the lowest possible cost.

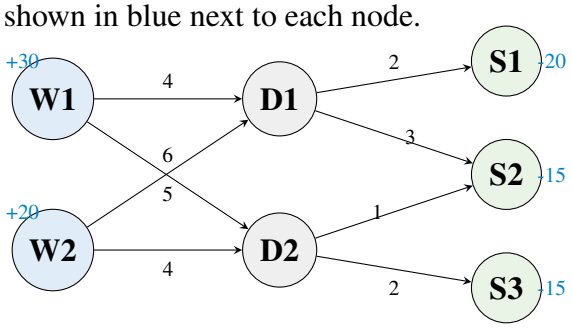
5.5.3. Minimum Cost Network Flow Problem

The *minimum cost network flow problem* generalizes basic flow models by incorporating arc capacities, arc costs, and supply/demand at the nodes. The goal is to determine how to route flow through a network at the lowest possible cost while meeting demand and respecting all constraints.

Real-World Example: Amazon's Distribution Network. Amazon must ship goods from large regional warehouses to local retail stores through intermediate distribution centers. Each warehouse has a certain supply capacity, each store has a demand, and each route incurs a transportation cost (fuel, time, and labor). Amazon wants to decide how to route products to minimize cost, while ensuring demand is met and no transportation link exceeds its capacity.

The figure below shows a small example. There are:

- Two warehouses (W1, W2) with a total of 50 units of supply,
- Two distribution centers (D1, D2),
- Three stores (S1, S2, S3) that need 20, 15, and 15 units, respectively,
- Arc costs labeled in black,
- Node demands/supplies shown in blue next to each node.



The problem is to determine how many units to send along each route to minimize the total shipping cost.

Sets:

- V: set of nodes in the network (warehouses, distribution centers, stores)
- A: set of directed arcs (i, j) representing shipment routes

Parameters:

- c_{ij}: cost per unit shipped on arc $(i, j) \in A$
- u_{ij}: maximum capacity of arc $(i, j) \in A$
- d_i: net demand at node $i \in V$ (positive = demand, negative = supply)

Decision Variables:
x_{ij}: amount of flow shipped along arc $(i, j) \in A$

Objective and Constraints:

$$\begin{aligned} \min \quad & \sum_{(i,j)\in A} c_{ij} \cdot x_{ij} \\ \text{s.t.} \quad & \sum_{i:(i,k)\in A} x_{ik} - \sum_{j:(k,j)\in A} x_{kj} = d_k && \forall k \in V \quad \text{(flow balance)} \\ & 0 \le x_{ij} \le u_{ij} && \forall (i,j) \in A \quad \text{(capacity constraints)} \end{aligned}$$

This problem can be solved using linear programming, and efficient specialized algorithms exist due to its structure. Applications span:

- Freight transportation,
- Supply chain optimization,
- Scheduling and telecommunications.

5.5.4. (Unstructured) Minimum Cost Network Flow Problem

The minimum cost network flow problem is an extension of the network flow problem that considers not only the capacities of the arcs, but also the costs associated with sending flow along the arcs and the demands at the nodes. The objective is to find the flow of minimum cost that satisfies the demands at the nodes and the capacity constraints on the arcs.

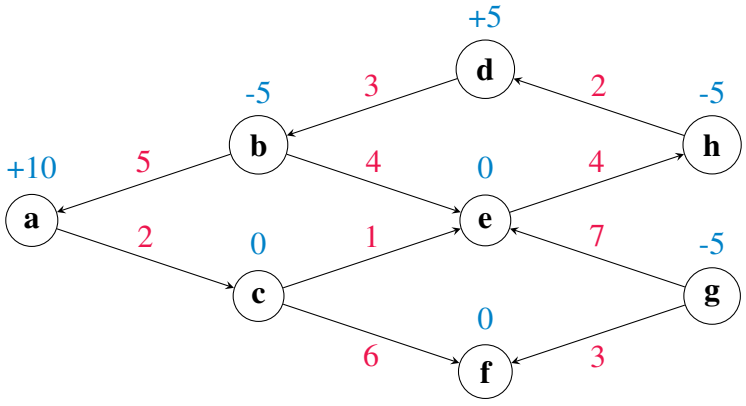
A network can be described as a directed graph $G = (V, A)$, where V is the set of nodes and A is the set of arcs. Each arc $(i, j) \in A$ has a capacity u_{ij} and a cost c_{ij}. Each node $i \in V$ has a demand d_i, which can be positive, negative, or zero. A positive demand means that the node requires that amount of flow, a negative demand means that the node supplies that amount of flow, and a zero demand means that the node neither requires nor supplies flow.

Consider, for example: The set of vertices is given by:

$$V = \{a, b, c, d, e, f, g, h\}$$

The set of directed arcs is:

$$A = \{(b,a), (a,c), (b,d), (b,e), (c,e), (c,f), (h,d), (g,e), (e,h), (g,f)\}$$



From	To	Cost
b	a	5
a	c	2
d	b	3
b	e	4
c	e	1
c	f	6
h	d	2
g	e	7
e	h	4
g	f	3

Table 5.1: Arc Costs in the Network Flow Problem

Node	Net Flow
a	+10
b	-5
c	0
d	+5
e	0
f	0
g	-5
h	-5

Table 5.2: Net Flow at Each Node

The minimum cost network flow problem can be defined as follows: Given a network $G = (V,A)$, capacities u_{ij} and costs c_{ij} on the arcs, and demands d_i at the nodes, find the flow x_{ij} on the arcs that minimizes the total cost of the flow, while satisfying the demands at the nodes and the capacity constraints on the arcs.

Sets:

- V: Set of nodes in the network.
- A: Set of arcs in the network.

Parameters:

- u_{ij}: Capacity of arc $(i, j) \in A$.
- c_{ij}: Cost per unit of flow on arc $(i, j) \in A$.
- d_i: Change at node $i \in V$.

Variables:

- x_{ij}: Flow on arc $(i, j) \in A$.

Model: The minimum cost network flow problem can be formulated as a linear programming problem as follows:

$$\begin{aligned} \text{Minimize} \quad & \sum_{(i,j)\in A} c_{ij}x_{ij} \\ \text{Subject to} \quad & \sum_{i:(i,k)\in A} x_{ij} - \sum_{j:(k,j)\in A} x_{ji} = d_k, \quad \forall k \in V \\ & 0 \le x_{ij} \le u_{ij}, \quad \forall (i,j) \in A \end{aligned}$$

Objective Function - The objective is to minimize the total cost of the flow.

Constraints - Flow Conservation: The first set of constraints ensures that the flow into each node minus the flow out of the node is equal to the demand at the node.

- Capacity Constraints: The second set of constraints ensures that the flow on each arc does not exceed its capacity.

Minimum-cost network flow problems are widely used in transportation planning, supply chain logistics, and production scheduling, where the goal is to efficiently allocate resources at the lowest possible cost.

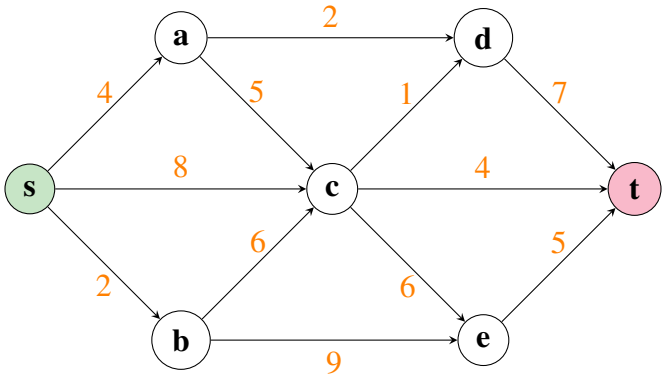
5.5.5. Maximum flow

The maximum flow problem is a fundamental problem in operations research and computer science, with applications in a wide variety of fields. It involves finding the maximum amount of flow that can be sent from a source node to a sink node in a network, without exceeding the capacities of the individual arcs, and ensuring that the flow on each arc is non-negative. This problem arises in many real-world situations, such as in transportation, where it can be used to find the maximum amount of goods that can be transported from a factory to a warehouse, or in telecommunications, where it can be used to find the maximum amount of data that can be sent from one server to another. Other applications include supply chain optimization, water distribution, and electrical power grid optimization. In this section, we will describe a linear programming formulation for the network flow problem.

A network can be described as a directed graph $G = (V, A)$, where V is the set of nodes and A is the set of arcs. Each arc $(i, j) \in A$ has a capacity u_{ij}, which is the maximum amount of flow that can traverse the arc from node i to node j.

The network flow problem can be defined as follows: Given a network $G = (V, A)$, a source node s, a sink node t, and capacities u_{ij} on the arcs, find the maximum flow from s to t that respects the capacity constraints.

For example, consider the directed graph here with capacities on the arcs.



Example 5.7: Airline Transfer Network: Maximizing Passenger Throughput Excel PuLP Gurobipy

An airline is trying to reroute as many stranded passengers as possible from an origin city (s) to a destination city (t) using a network of connecting flights. Each flight has a known number of available seats, and passengers may transfer at intermediate airports. The objective is to determine the maximum number of passengers that can be routed from origin to destination, using available seat capacity and respecting flight routes.

The network includes:

- Origin airport s,
- Intermediate hubs: a, b, c,
- Regional transfer airports: d, e,
- Final destination airport t,
- Directed flight legs (arcs) with known seat availability.

Sets and Parameters:

- $V = \{s, a, b, c, d, e, t\}$: set of airports.
- $A = \{(s,a),(s,b),(s,c),(a,c),(a,d),(b,c),(b,e),(c,d),(c,e),(c,t),(d,e),(e,t)\}$: set of flight legs.
- u_{ij}: number of available seats on flight from airport i to j for all $(i,j) \in A$.

Decision Variables:
x_{ij}: number of passengers assigned to fly from airport i to j for all $(i,j) \in A$

Objective and Constraints:

$$
\begin{aligned}
\max \quad & x_{sa} + x_{sb} + x_{sc} && \text{(max passengers from origin)} \\
\text{s.t.} \quad & x_{ac} + x_{ad} - x_{sa} = 0 && \text{(flow balance at } a\text{)} \\
& x_{bc} + x_{be} - x_{sb} = 0 && \text{(at } b\text{)} \\
& x_{cd} + x_{ce} + x_{ct} - x_{ac} - x_{bc} - x_{sc} = 0 && \text{(at } c\text{)} \\
& x_{de} - x_{ad} - x_{cd} = 0 && \text{(at } d\text{)} \\
& x_{et} - x_{be} - x_{ce} - x_{de} = 0 && \text{(at } e\text{)} \\
& 0 \le x_{sa} \le 4, \quad 0 \le x_{sb} \le 2, \quad 0 \le x_{sc} \le 8 && \text{(available seats on initial flights)} \\
& 0 \le x_{ac} \le 5, \quad 0 \le x_{ad} \le 2, \quad 0 \le x_{bc} \le 6, \quad 0 \le x_{be} \le 9 \\
& 0 \le x_{cd} \le 1, \quad 0 \le x_{ce} \le 3, \quad 0 \le x_{ct} \le 4, \quad 0 \le x_{de} \\
& \le 7, \quad 0 \le x_{et} \le 5
\end{aligned}
$$

In compact summation notation form, we have

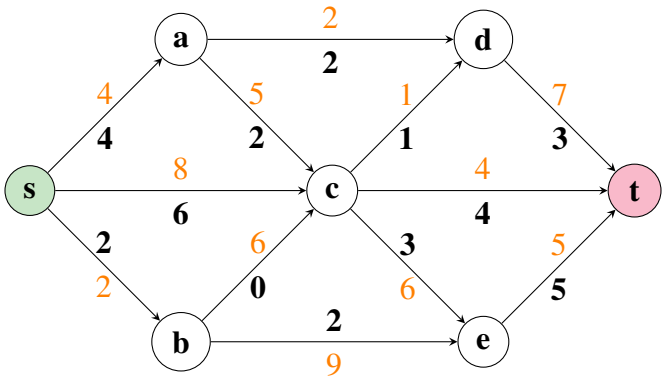
$$
\begin{aligned}
\max \quad & \sum_{j \in V:(s,j) \in A} x_{sj} \\
\text{s.t.} \quad & \sum_{j \in V:(k,j) \in A} x_{kj} - \sum_{i \in V:(i,k) \in A} x_{ik} = 0 && \forall k \in V \setminus \{s,t\} \\
& 0 \leq x_{ij} \leq u_{ij} && \forall (i,j) \in A
\end{aligned}
$$

This model helps the airline determine how to assign passengers to flight legs in a way that makes maximum use of the network's available capacity. It is especially useful during disruption scenarios like:

- severe weather cancellations,
- emergency rerouting due to maintenance issues,
- high-demand events requiring temporary overflow routing.

The same framework can be extended to include priorities, time windows, or rebooking costs.

The solution is shown here in bold values on the graph. There is a total flow of 12.



The maximum flow problem can be written mathematically in the following way.

Sets:

- V: Set of nodes in the network.
- A: Set of arcs in the network.

Parameters:

- u_{ij}: Capacity of arc $(i,j) \in A$, which is the maximum amount of flow that can traverse the arc from node i to node j.
- s: Source node.
- t: Sink node.

Variables:

- x_{ij}: Flow on arc $(i, j) \in A$.

Model: The network flow problem can be formulated as a linear programming problem as follows:

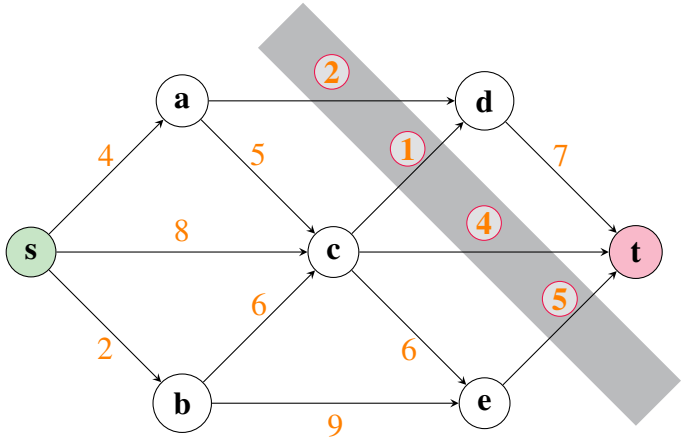
$$\begin{aligned} \text{Maximize} \quad & \sum_{j:(s,j)\in A} x_{sj} \\ \text{Subject to} \quad & \sum_{i:(i,k)\in A} x_{ik} - \sum_{j:(k,j)\in A} x_{kj} = 0, \quad \forall k \in V \setminus \{s,t\} \\ & 0 \le x_{ij} \le u_{ij}, \quad \forall (i,j) \in A \end{aligned}$$

Objective Function - The objective is to maximize the total flow from the source node s to the sink node t.

Constraints - Flow Balance: The first set of constraints ensures that the flow into each node is equal to the flow out of the node, except for the source and sink nodes.

- Capacity Constraints: The third set of constraints ensures that the flow on each arc does not exceed its capacity.

We will discuss duality later. But duality is a concept that helps create bounds on a problem. For this particular proble, we can think of a dual as a *cut* in the graph - somthing that separtes s from t. For example:



The selected arcs are a cut that separates s from t. The sum of their weights is $2+1+4+5=12$, which implies that there is not a flow more than 12 units from s to t.

This bound is a form of *duality*. We will explore the concept more in later chapters. This duality is captured in the following theorem.

Theorem 5.4

The maximum flow amount is equal to the size if the minimum cut.

5.6 Exercises

Exercise 5.8 *[Fair Housing Assignment for 12 Families - part 1]*

A new apartment complex has been completed, and 12 families need to be assigned to 12 available apartments. Each family has ranked the apartments (1 = most preferred, 12 = least preferred).

1. *Use Excel Solver to find an assignment that minimizes the sum of preferences.*

2. *What units were assigned to which families, and what was their preference score?*

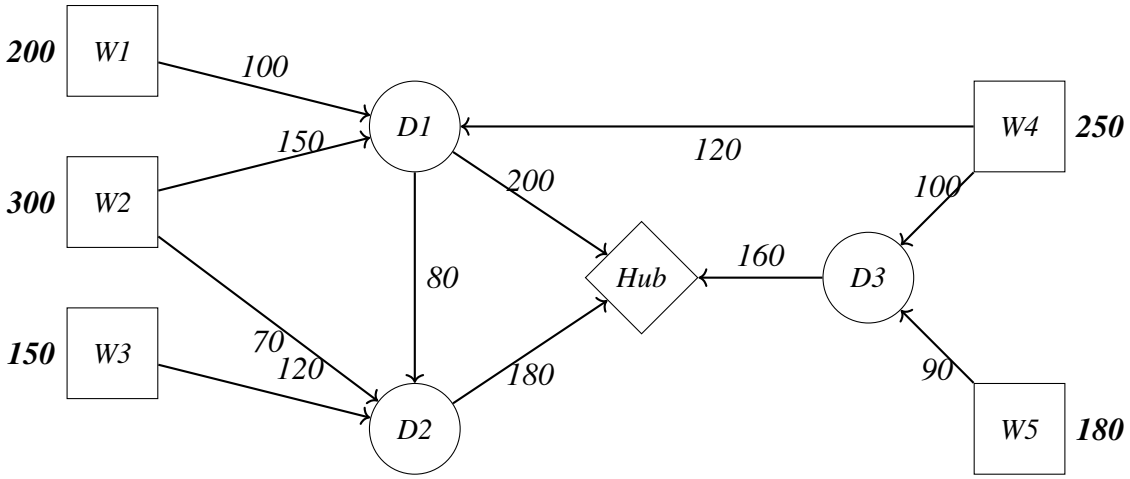
3. *What is the worst preference score assigned?*

Download the data for this problem here data.

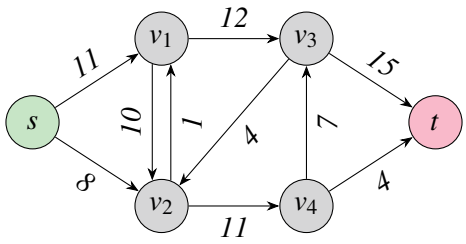
Exercise 5.9 *Maximum Network Flow to a Single Node A disaster relief effort is underway to transport supplies to a central distribution hub in a flooded region. The network consists of multiple warehouses (nodes) that currently have varying inventory levels of emergency supplies. Each transportation route (arc) between warehouses is serviced by a single truck with a limited carrying capacity. The goal is to determine the optimal transportation plan that maximizes the amount of supplies delivered to the central hub while respecting inventory constraints and truck capacities on each route.*

Formulate a max-flow optimization problem for this scenario. Define your decision variables, objective function, and constraints explicitly. Assume that you do not want to leave any inventory at the distribution centers.

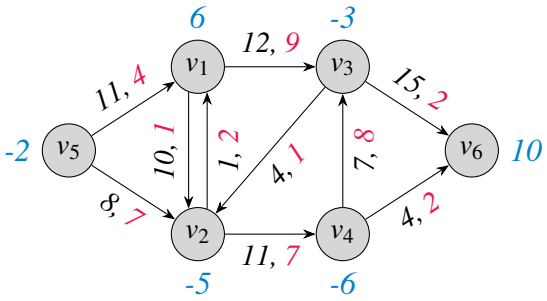
Node	***Type***	***Inventory (if applicable)***	***Connected To***	***Capacity***
W1	*Warehouse*	***200***	*D1*	*100*
W2	*Warehouse*	***300***	*D1*	*150*
W3	*Warehouse*	***150***	*D2*	*120*
W4	*Warehouse*	***250***	*D3*	*100*
W5	*Warehouse*	***180***	*D3*	*90*
D1	*Distribution*	*N/A*	*Hub, D2*	*200, 80*
D2	*Distribution*	*N/A*	*Hub*	*180*
Hub	*Central Hub*	*N/A*	*D3*	*160*
D3	*Distribution*	*N/A*	*W4, W5*	*100, 90*



Exercise 5.10 *Max flow Write out the explicit constraints that define the max flow problem given in the picture. Solve this problem as a linear program using Excel.*



Exercise 5.11 *Min Cost Network Flow Use the picture below to formulate the corresponding capacitated min cost network flow. Solve this problem using Excel.*



6. Graphically Solving Linear Programs

Outcomes

A. *Learn how to plot the feasible region and the objective function.*
B. *Identify and compute extreme points of the feasible region.*
C. *Find the optimal solution(s) to a linear program graphically.*
D. *Classify the type of result of the problem as infeasible, unbounded, unique optimal solution, or infinitely many optimal solutions.*

Linear Programs (LP's) with two variables can be solved graphically by plotting the feasible region along with the level curves of the objective function.[1] We will show that we can find a point in the feasible region that maximizes the objective function using the level curves of the objective function.

We will begin with an easy example that is bounded and investigate the structure of the feasible region. We will then explore other examples.

6.1 Nonempty and Bounded Problem

Consider the problem

$$\begin{aligned} \max \quad & 2x_1 + 5x_2 \\ \text{s.t.} \quad & x_1 + 2x_2 \leq 16 \\ & 5x_1 + 3x_2 \leq 45 \\ & x_1, x_2 \geq 0 \end{aligned}$$

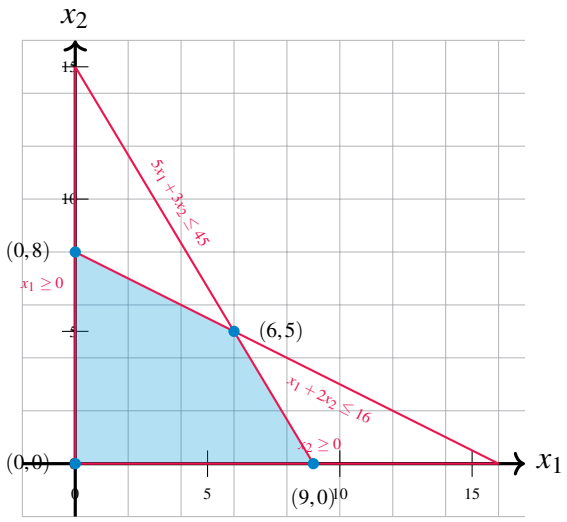
We want to start by plotting the *feasible region*, that is, the set points (x_1, x_2) that satisfy all the constraints.

We can plot this by first plotting the four lines

- $x_1 + 2x_2 = 16$
- $5x_1 + 3x_2 = 45$
- $x_1 = 0$
- $x_2 = 0$

[1] Special thanks to Joshua Emmanual and Christopher Griffin for sharing their content to help put this section together. Proper citations and referenes are forthcoming.

and then shading in the side of the space cut out by the corresponding inequality.



Vertex	Objective Value
$(0,0)$	0
$(0,8)$	40
$(6,5)$	37
$(9,0)$	18

Figure 6.1: Desmos: Interactive plot!

The resulting feasible region can then can be shaded in as the region that satisfies all the inequalties.

Notice that the feasible region is nonempty (it has points that satisfy all the inequalities) and also that it is bounded (the feasible points dont continue infinitly in any direction).

We want to identify the *extreme points* (i.e., the corners) of the feasible region. Understanding these points will be critical to understanding the optimal solutions of the model. Notice that all extreme points can be computed by finding the intersection of 2 of the lines. But! Not all intersections of any two lines are feasible.

We will later use the terminology *basic feasible solution* for an extreme point of the feasible region, and *basic solution* as a point that is the intersection of 2 lines, but is actually infeasible (does not satisfy all the constraints).

Theorem 6.1: Optimal Solution at a Vertex

If the feasible region of the linear program is nonempty and bounded, then there exists an optimal solution at vertex of the feasible region.

This theorem implies the following algorithm.

Algorithm 1 Enumerate Vertices Algorithm for Solving a Linear Programming Problem

1. **Identify the Feasible Region:** Determine the set of constraints defining the feasible region in the problem.
2. **Enumerate Vertices:** Find all the vertices (corner points) of the feasible region. This can be done by solving pairs of constraint equations to find their intersections, keeping only those that satisfy all constraints.
3. **Compute Objective Values:** Evaluate the objective function at each vertex of the feasible region.
4. **Compare Objective Values:** For a maximization problem, identify the vertex with the highest objective value. For a minimization problem, identify the vertex with the lowest objective value.
5. **Determine the Optimal Solution:** The vertex with the optimal objective value is the solution to the linear programming problem.

We will explore why Theorem 6.1 is true, and also what happens when the feasible region does not satisfy the assumtions of either nonempty or bounded.

6.2 Graphical Approach

We illustrate the idea first using the problem from Example 3.1. In a later section, we will formalize the mathematics to explain this result.

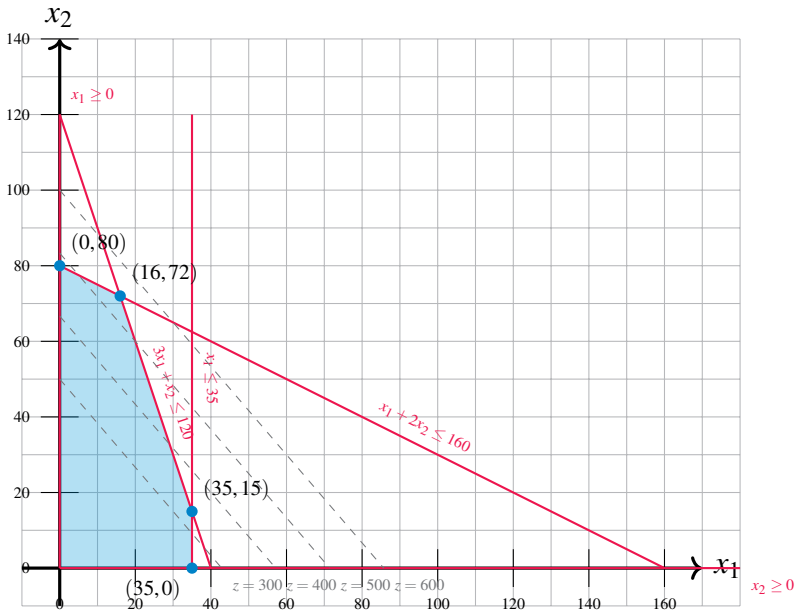


Figure 6.2: Feasible Region and Level Curves of the Objective Function: The shaded region in the plot is the feasible region and represents the intersection of the five inequalities constraining the values of x_1 and x_2. On the right, we see the optimal solution is the "last" point in the feasible region that intersects a level set as we move in the direction of increasing profit.

Example 6.2: Continuation of ToyMaker Example

Let's continue the example of the Toy Maker begin in Example 3.1. Solve this problem graphically.

Solution. To solve the linear programming problem graphically, begin by drawing the feasible region. This is shown in the blue shaded region of Figure 6.2.

After plotting the feasible region, the next step is to plot the level curves of the objective function. In our problem, the level sets will have the form:

$$7x_1 + 6x_2 = z \implies x_2 = \frac{-7}{6}x_1 + \frac{z}{6}$$

This is a set of parallel lines with slope $-7/6$ and intercept $z/6$ where z can be varied as needed. The level curves for various values of z are parallel lines. In Figure 6.2 they are shown in colors ranging from red to yellow depending upon the value of z. Larger values of z are more yellow.

To solve the linear programming problem, follow the level sets along the gradient (shown as the black arrow) until the last level set (line) intersects the feasible region. If you are doing this by hand, you can draw a single line of the form $7x_1 + 6x_2 = z$ and then simply draw parallel lines in the direction of the gradient $(7,6)$. At some point, these lines will fail to intersect the feasible region. The last line to intersect the feasible region will do so at a point that maximizes the profit. In this case, the point that maximizes $z(x_1,x_2) = 7x_1 + 6x_2$, subject to the constraints given, is $(x_1^*,x_2^*) = (16,72)$.

Note the point of optimality $(x_1^*, x_2^*) = (16, 72)$ is at a corner of the feasible region. This corner is formed by the intersection of the two lines: $3x_1 + x_2 = 120$ and $x_1 + 2x_2 = 160$. In this case, the constraints

$$3x_1 + x_2 \leq 120$$
$$x_1 + 2x_2 \leq 160$$

are both *binding*, while the other constraints are non-binding. In general, we will see that when an optimal solution to a linear programming problem exists, it will always be at the intersection of several binding constraints; that is, it will occur at a corner of a higher-dimensional polyhedron. ♠

We can now define an algorithm for identifying the solution to a linear programing problem in two variables with a *bounded* feasible region.

(First Try) Algorithm for Solving a Linear Programming Problem Graphically

Bounded Feasible Region, Unique Solution

1. Plot the feasible region defined by the constraints.
2. Plot the level sets of the objective function.
3. For a maximization problem, identify the level set corresponding the greatest (least, for minimization) objective function value that intersects the feasible region. This point will be at a corner.
4. The point on the corner intersecting the greatest (least) level set is a solution to the linear programming problem.

Example 6.3: Lemonade Vendor

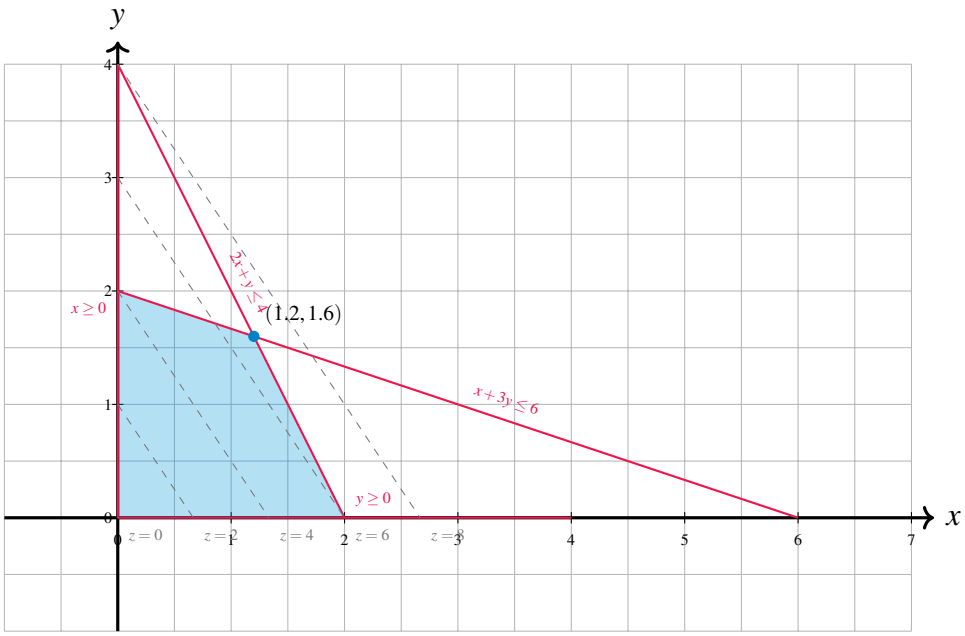
Say you are a vendor of lemonade and lemon juice. Each unit of lemonade requires 1 lemon and 2 litres of water. Each unit of lemon juice requires 3 lemons and 1 litre of water. Each unit of lemonade gives a profit of three dollars. Each unit of lemon juice gives a profit of two dollars. You have 6 lemons and 4 litres of water available. How many units of lemonade and lemon juice should you make to maximize profit?

If we let x denote the number of units of lemonade to be made and let y denote the number of units of lemon juice to be made, then the profit is given by $3x+2y$ dollars. We call $3x+2y$ the objective function. Note that there are a number of constraints that x and y must satisfied. First of all, x and y should be nonnegative. The number of lemons needed to make x units of lemonade and y units of lemon juice is $x+3y$ and cannot exceed 6. The number of litres of water needed to make x units of lemonade and y units of lemon juice is $2x+y$ and cannot exceed 4. Hence, to determine the maximum profit, we need to maximize $3x+2y$ subject to x and y satisfying the constraints $x+3y \leq 6$, $2x+y \leq 4$, $x \geq 0$, and $y \geq 0$.

A more compact way to write the problem is as follows:

$$\begin{array}{rrcrcl} \textit{maximize} & 3x & + & 2y & & \\ \textit{subject to} & x & + & 3y & \leq & 6 \\ & 2x & + & y & \leq & 4 \\ & x & & & \geq & 0 \\ & & & y & \geq & 0. \end{array}$$

Solution. We can solve this maximization problem graphically as follows. We first sketch the set of (x,y) satisfying the constraints, called the feasible region, on the (x,y)-plane. We then take the objective function $3x+2y$ and turn it into an equation of a line $3x+2y=z$ where z is a parameter. Note that as the value of z increases, the line defined by the equation $3x+2y=z$ moves in the direction of the normal vector $(3,2)$ We call this direction the direction of improvement. Determining the maximum value of the objective function, called the optimal value, subject to the contraints amounts to finding the maximum value of z so that the line defined by the equation $3x+2y=z$ still intersects the feasible region.



In the figure above, the lines with z at 0, 4 and 6.8 have been drawn. From the picture, we can see that if z is greater than 6.8, the line defined by $3x+2y=z$ will not intersect the feasible region. Hence, the profit cannot exceed 6.8 dollars.

As the line $3x+2y=6.8$ does intersect the feasible region, 6.8 is the maximum value for the objective function. Note that there is only one point in the feasible region that intersects the line $3x+2y=6.8$, namely $(x,y)=(1.2,1.6)$. In other words, to maximize profit, we want to make 1.2 units of lemonade and 1.6 units of lemon juice. ♠

The example linear programming problem presented in this has a single optimal solution. In general, the following outcomes can occur in solving a linear programming problem:

1. The linear programming problem has a unique solution. (We've already seen this.)
2. There are infinitely many alternative optimal solutions.
3. There is no solution and the problem's objective function can grow to positive infinity for maximization problems (or negative infinity for minimization problems).
4. There is no solution to the problem at all.

Case 3 above can only occur when the feasible region is unbounded; that is, it cannot be surrounded by a ball with finite radius. We will illustrate each of these possible outcomes in the next four sections. We will prove that this is true in a later chapter.

6.3 Infinitely Many Optimal Solutions

It can happen that there is more than one solution. In fact, in this case, there are infinitly many optimal solutions. We'll study a specific linear programming problem with an infinite number of solutions by modifying the objective function in Example 3.1.

Example 6.4: Toy Maker Alternative Solutions

Suppose the toy maker in Example 3.1 finds that it can sell planes for a profit of \$18 *each instead of* \$7 *each. The new linear programming problem becomes:*

$$\begin{aligned} \max\ & z(x_1,x_2) = 18x_1 + 6x_2 \\ s.t.\ & 3x_1 + x_2 \leq 120 \\ & x_1 + 2x_2 \leq 160 \\ & x_1 \leq 35 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \end{aligned} \quad . \tag{6.1}$$

Solution. Applying our graphical method for finding optimal solutions to linear programming problems yields the plot shown in Figure 6.3. The level curves for the function $z(x_1,x_2) = 18x_1 + 6x_2$ are *parallel* to one face of the polygon boundary of the feasible region. Hence, as we move further up and to the right in the direction of the gradient (corresponding to larger and larger values of $z(x_1,x_2)$) we see that there is not *one* point on the boundary of the feasible region that intersects that level set with greatest value, but instead a side of the polygon boundary described by the line $3x_1 + x_2 = 120$ where $x_1 \in [16,35]$.

Let:

$$P = \left\{ (x_1,x_2) \ \middle| \ \begin{array}{c} 3x_1 + x_2 \leq 120, \\ x_1 + 2x_2 \leq 160, \\ x_1 \leq 35, \\ x_1, x_2 \geq 0 \end{array} \right\}$$

that is, P is the feasible region of the problem. Then for any value of $x_1^* \in [16,35]$ and any value x_2^* so that $3x_1^* + x_2^* = 120$, we will have $z(x_1^*,x_2^*) \geq z(x_1,x_2)$ for all $(x_1,x_2) \in P$. Since there are infinitely many values that x_1 and x_2 may take on, we see this problem has an infinite number of alternative optimal solutions.

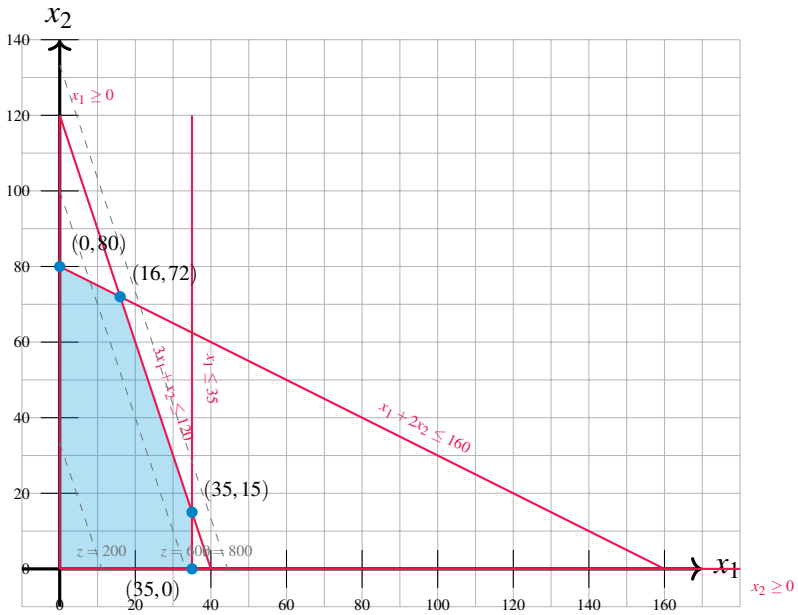


Figure 6.3: An example of infinitely many alternative optimal solutions in a linear programming problem. The level curves for $z(x_1,x_2) = 18x_1 + 6x_2$ are *parallel* to one face of the polygon boundary of the feasible region. Moreover, this side contains the points of greatest value for $z(x_1,x_2)$ inside the feasible region. Any combination of (x_1,x_2) on the line $3x_1 + x_2 = 120$ for $x_1 \in [16,35]$ will provide the largest possible value $z(x_1,x_2)$ can take in the feasible region S.

♠

Based on the example in this section, we can modify our algorithm for finding the solution to a linear programming problem graphically to deal with situations with an infinite set of alternative optimal solutions:

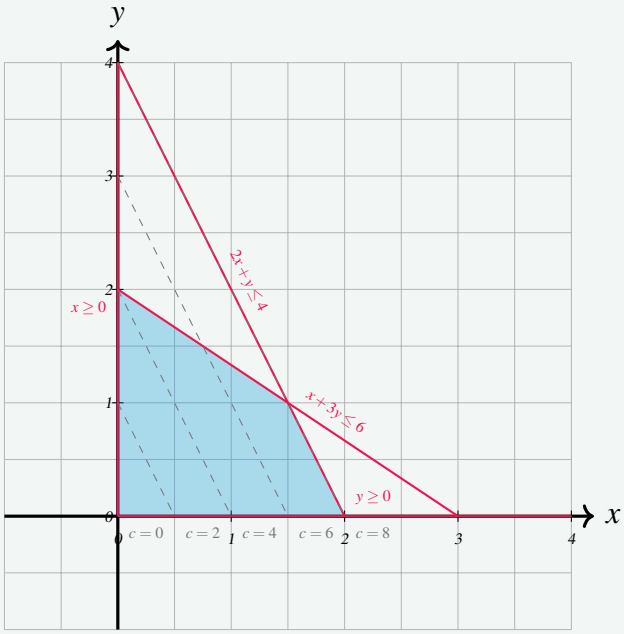
(Second try) Algorithm for Solving a Linear Programming Problem Graphically

Bounded Feasible Region

1. Plot the feasible region defined by the constraints.
2. Plot the level sets of the objective function.
3. For a maximization problem, identify the level set corresponding the greatest (least, for minimization) objective function value that intersects the feasible region. This point will be at a corner.
4. The point on the corner intersecting the greatest (least) level set is a solution to the linear programming problem.
5. **If the level set corresponding to the greatest (least) objective function value is parallel to a side of the polygon boundary next to the corner identified, then there are infinitely many alternative optimal solutions and any point on this side may be chosen as an optimal solution.**

Exercise 6.5

Consider the graphical representation below. How many optimal solutions are there if we are maximizing? How about if we are minimizing?



6.4 Problems with No Solution

Recall for *any* mathematical programming problem, the feasible set or region is simply a subset of $\mathbb{R}^n$. If this region is empty, then there is no solution to the mathematical programming problem and the problem is said to be *over constrained*. In this case, we say that the problem is *infeasible*. We illustrate this case for linear programming problems with the following example.

Example 6.6: Infeasible Problem

Consider the following linear programming problem:

$$\begin{aligned} \max\ & z(x_1,x_2) = 3x_1 + 2x_2 \\ s.t.\ & \frac{1}{40}x_1 + \frac{1}{60}x_2 \leq 1 \\ & \frac{1}{50}x_1 + \frac{1}{50}x_2 \leq 1 \\ & x_1 \geq 30 \\ & x_2 \geq 20 \end{aligned} \quad . \tag{6.1}$$

Solution. The level sets of the objective and the constraints are shown in Figure 6.4.

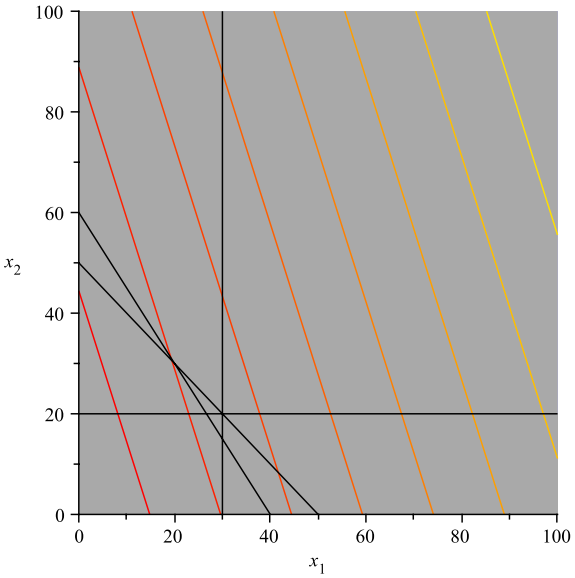


Figure 6.4: A Linear Programming Problem with no solution. The feasible region of the linear programming problem is empty; that is, there are no values for x_1 and x_2 that can simultaneously satisfy all the constraints. Thus, no solution exists.

The fact that the feasible region is empty is shown by the fact that in Figure 6.4 there is no blue region–i.e., all the regions are gray indicating that the constraints are not satisfiable. ♠

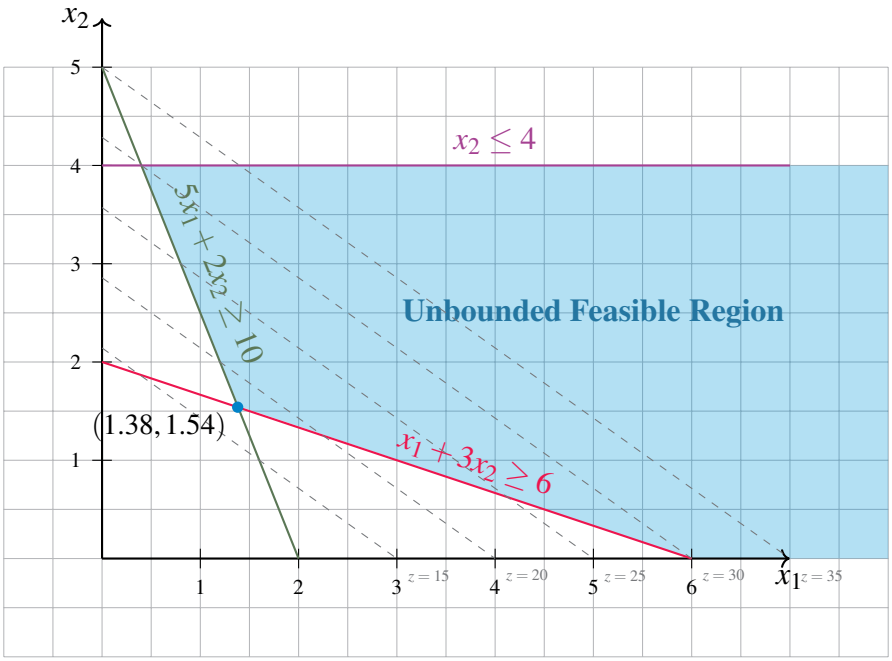
Based on this example, we will modify our previous algorithm for finding the solution to linear programming problems graphically to consider the case that it is empty.

Before stating complete algorithm, we also consider the unbounded case.

6.5 Problems with Unbounded Feasible Regions

Consider the problem

$$\begin{aligned} \min \quad & z = 5x_1 + 7x_2 \\ \text{s.t.} \quad & x_1 + 3x_2 \geq 6 \\ & 5x_1 + 2x_2 \geq 10 \\ & x_2 \leq 4 \\ & x_1, x_2 \geq 0 \end{aligned}$$



As you can see, the feasible region is *unbounded*. In particular, from any point in the feasible region, one can always find another feasible point by increasing the X coordinate (i.e., move to the right in the picture). However, this does not necessarily mean that the optimization problem is unbounded.

Indeed, the optimal solution is at $\mathbf{x}^* = (1.38, 1.54)$, the extreme point in the lower left hand corner, with optimal objective value

$$z^* = 5 \cdot 1.38 + 7 \cdot 1.54 = 17.7.$$

Consider however, if we consider a different problem where we try to maximize the objective

$$\begin{aligned} \max \quad & z = 5x_1 + 7x_2 \\ \text{s.t.} \quad & x_1 + 3x_2 \geq 6 \\ & 5x_1 + 2x_2 \geq 10 \\ & x_2 \leq 4 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Solution. This optimization problem is unbounded! For example, notice that the point $(x_1, x_2) = (n, 0)$ is feasible for all $n = 1, 2, 3, \ldots,$. Then the objective function $z = 5n + 0$ follows the sequence $5, 10, 15, \ldots,$, which diverges to infinity. ♠

Let's see another example.

Example 6.7

Consider the linear programming problem below:

$$\begin{aligned} \max \ & z(x_1,x_2) = 2x_1 - x_2 \\ s.t. \ & x_1 - x_2 \leq 1 \\ & 2x_1 + x_2 \geq 6 \\ & x_1, x_2 \geq 0 \end{aligned} \quad . \tag{6.1}$$

Solution. The feasible region and level curves of the objective function are shown in Figure 6.5.

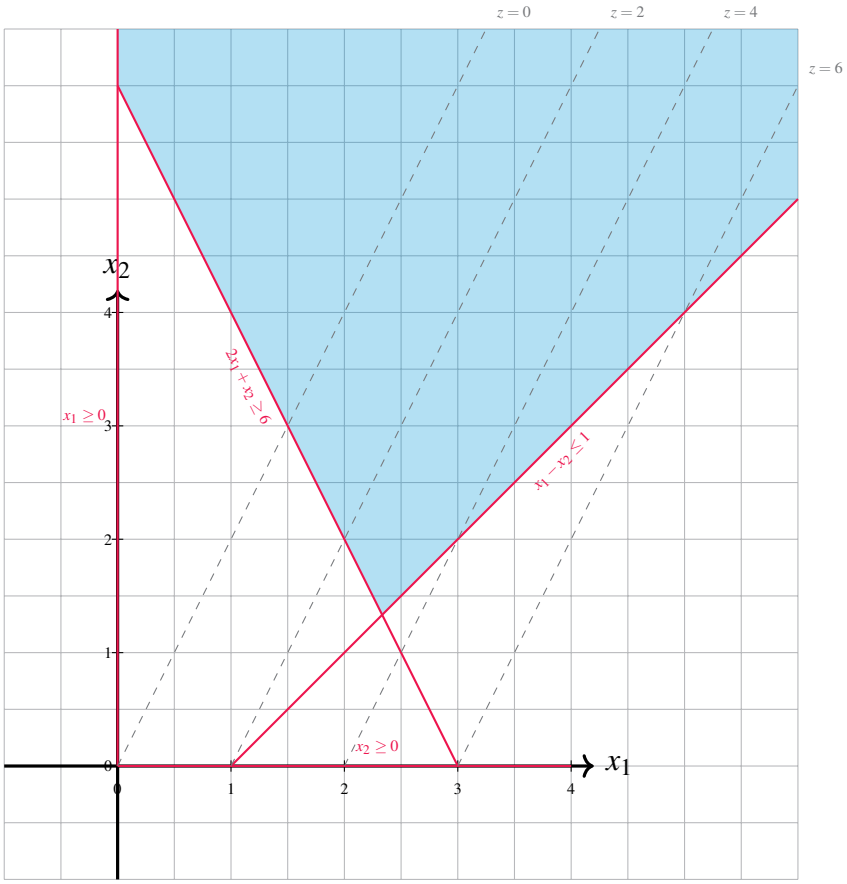


Figure 6.5: A Linear Programming Problem with Unbounded Feasible Region: Note that we can continue to make level curves of $z(x_1,x_2)$ corresponding to larger and larger values as we move down and to the right. These curves will continue to intersect the feasible region for any value of $v = z(x_1,x_2)$ we choose. Thus, we can make $z(x_1,x_2)$ as large as we want and still find a point in the feasible region that will provide this value. Hence, the optimal value of $z(x_1,x_2)$ subject to the constraints $+\infty$. That is, the problem is unbounded.

The feasible region in Figure 6.5 is clearly unbounded since it stretches upward along the x_2 axis infinitely far and also stretches rightward along the x_1 axis infinitely far, bounded below by the line $x_1 - x_2 = 1$. There is no way to enclose this region by a disk of finite radius, hence the feasible region is not bounded.

We can draw more level curves of $z(x_1,x_2)$ in the direction of increase (down and to the right) as long as we wish. There will always be an intersection point with the feasible region because it is infinite. That is, these curves will continue to intersect the feasible region for any value of $v = z(x_1,x_2)$ we choose. Thus, we can make $z(x_1,x_2)$ as large as we want and still find a point in the feasible region that will provide this value. Hence, the largest value $z(x_1,x_2)$ can take when (x_1,x_2) are in the feasible region is $+\infty$. That is, the problem is unbounded. ♠

Just because a linear programming problem has an unbounded feasible region does not imply that there is not a finite solution. We illustrate this case by modifying example 6.5.

Example 6.8: Continuation of Example

Consider the linear programming problem from Example 6.5 with the new objective function: $z(x_1,x_2) = (1/2)x_1 - x_2$. Then we have the new problem:

$$\begin{array}{lrl} \max & z(x_1,x_2) = \dfrac{1}{2}x_1 - x_2 & \\ s.t. & x_1 - x_2 \leq 1 & \\ & 2x_1 + x_2 \geq 6 & \\ & x_1, x_2 \geq 0 & \end{array} . \tag{6.2}$$

Solution. The feasible region, level sets of $z(x_1,x_2)$ and gradients are shown in Figure 6.6. In this case note, that the direction of increase of the objective function is *away* from the direction in which the feasible region is unbounded (i.e., downward). As a result, the point in the feasible region with the largest $z(x_1,x_2)$ value is $(7/3,4/3)$. Again this is a vertex: the binding constraints are $x_1 - x_2 = 1$ and $2x_1 + x_2 = 6$ and the solution occurs at the point these two lines intersect.

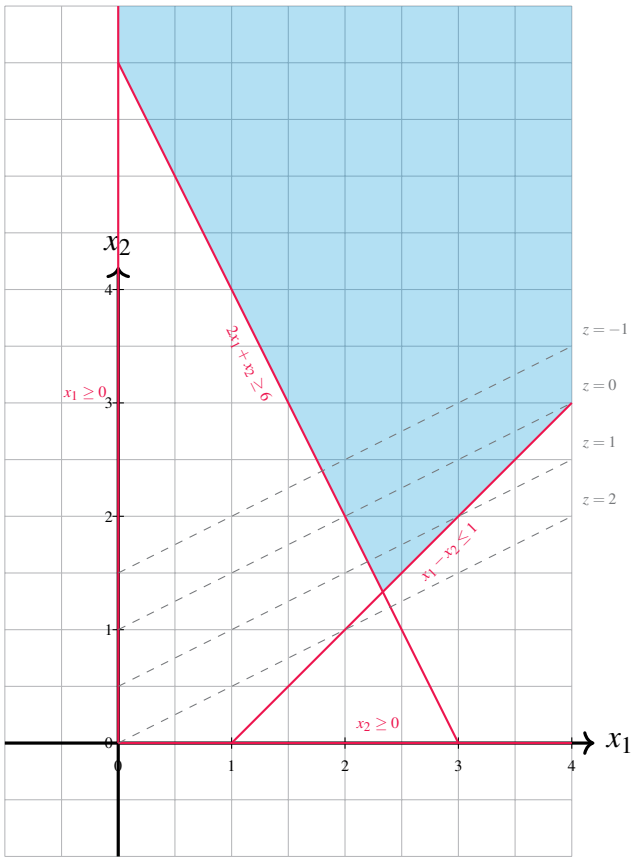


Figure 6.6: A Linear Programming Problem with Unbounded Feasible Region and Finite Solution: In this problem, the level curves of $z(x_1,x_2)$ increase in a more "southernly" direction that in Example 6.5–that is, *away* from the direction in which the feasible region increases without bound. The point in the feasible region with largest $z(x_1,x_2)$ value is $(7/3,4/3)$. Note again, this is a vertex.

♠

Based on these two examples, we can modify our algorithm for graphically solving a two variable linear programming problems to deal with the case when the feasible region is unbounded.

Algorithm 2 Algorithm for Solving a Two-Variable Linear Programming Problem Graphically

1. Plot the feasible region defined by the constraints.
2. If the feasible region is empty, there is no solution.
3. Plot the level sets of the objective function.
4. **Bounded Case:**
   (a) Identify the corner point where the optimal level set (maximization: greatest; minimization: least) intersects the feasible region.
   (b) If the level set is parallel to a side of the feasible region at this corner, any point on this side is an optimal solution (infinitely many solutions).
5. **Unbounded Case:**
   (a) If the level sets intersect the feasible region indefinitely (maximization: larger; minimization: smaller), the problem is unbounded ($+\infty$ or $-\infty$).
   (b) Otherwise, proceed as in the bounded case.

6.6 Exercises

Exercise 6.1 *Consider the following system of constraints:*

$$x+y \geq 5$$

$$x \geq 0, \quad y \geq 0$$

Which of the following statements is true about the feasible region?

1. *The feasible region is a bounded polygon.*
2. *The feasible region is an unbounded region.*
3. *The feasible region consists of a single point.*
4. *The problem is infeasible.*

Exercise 6.2 *Sketch all* $\begin{bmatrix} x \\ y \end{bmatrix}$ *satisfying*

$$x-2y \leq 2$$

on the (x,y)*-plane.*

Exercise 6.3 *Graphically Solving Graph the feasible region of the linear program:*

$$\begin{aligned} \min \quad & x_1 - x_2 \\ & x_1 + x_2 \leq 6 \\ & x_1 - x_2 \geq 1 \\ & x_2 - x_1 \geq 3 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Without considering the objective function, how many feasible solutions are there? Justify your answer.

Exercise 6.4 *Graphical Method Use the graphical method to determine two optimal solutions to linear program:*

$$\begin{aligned} \min \quad & 3x_1 + 5x_2 \\ & 3x_1 + 2x_2 \geq 36 \\ & 6x_1 + 10x_2 \geq 90 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Exercise 6.5 *Graph the feasible region of the linear program:*

$$\begin{aligned} \min \quad & x_1 - x_2 \\ & x_1 + x_2 \leq 6 \\ & x_1 - x_2 \geq 1 \\ & x_2 - x_1 \geq 3 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Without considering the objective function, how many feasible solutions are there? Justify your answer.

Exercise 6.6 *Determine the optimal value of*

$$\begin{aligned} \textit{Minimize} \quad & x + y \\ \textit{Subject to} \quad & 2x + y \geq 4 \\ & x + 3y \geq 1. \end{aligned}$$

Exercise 6.7 *Show that the problem*

$$\begin{aligned} \textit{Minimize} \quad & -x + y \\ \textit{Subject to} \quad & 2x - y \geq 0 \\ & x + 3y \geq 3 \end{aligned}$$

is unbounded.

Exercise 6.8 *Does the following problem have a bounded solution? Why?*

$$\begin{aligned} \min \quad & z(x_1, x_2) = 2x_1 - x_2 \\ s.t. \quad & x_1 - x_2 \leq 1 \\ & 2x_1 + x_2 \geq 6 \\ & x_1, x_2 \geq 0 \end{aligned} \quad . \tag{6.1}$$

[Hint: Use Figure 6.6 and Algorithm 2.]

Exercise 6.9 *Modify the objective function in Example 6.5 to produce a problem with an infinite number of solutions.*

Exercise 6.10 *Modify the objective function in Exercise 6.2 to produce a* ***minimization*** *problem that has a finite solution. Draw the feasible region and level curves of the objective to "prove" your example works.*

[Hint: Think about what direction of increase is required for the level sets of $z(x_1,x_2)$.]

Exercise 6.11 *Consider the following problem:*

$$\begin{array}{lrcrcl} \textit{minimize} & -2x_1 & + & x_2 & & \\ \textit{subject to} & -x_1 & + & x_2 & \leq & 3 \\ & x_1 & - & 2x_2 & \leq & 2 \\ & x_1 & & & \geq & 0 \\ & & & x_2 & \geq & 0. \end{array}$$

1. *Show that for any $t \geq 0$,*

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} t \\ t \end{bmatrix}$$

satisfies all the constraints. Show that the objective function is unbouned for this point as $t \to \infty$.

2. *Show that*

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2t+2 \\ t \end{bmatrix}$$

is feasible for any $t \geq 0$. What is the objective function value at this point? Show that the objective function is unbounded (yet again) by letting $t \to \infty$.

Exercise 6.12 *Consider the following problem:*

$$\begin{array}{lrcrcl} \textit{maximize/minimize} & 2x & + & 2y & & \\ \textit{subject to} & -x & + & 2y & \leq & 12 \\ & 2x & + & y & \geq & 4 \\ & x & + & 2y & \geq & 5 \\ & x & & & \geq & 0 \\ & & & y & \geq & 0. \end{array}$$

1. *Determine the minimum value of $2x+2y$ by solving (graphically) the corresponding optimization problem. Show that the minimum is attained at a bounded feasible point.*

2. *Show that the maximum value of $2x+2y$ is unbounded by finding a feasible solution of the form*

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 5 \\ 0 \end{bmatrix} + t \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

for any $t \geq 0$. Show that as $t \to \infty$, the objective function becomes unbounded.

3. *Show that*

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 6 \end{bmatrix} + t \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

is feasible for any $t \geq 0$. Compute the objective function value at this point and demonstrate that it is unbounded as $t \to \infty$.

Exercise 6.13 *Suppose that you are shopping for dietary supplements to satisfy your required daily intake of 0.40mg of nutrient M and 0.30mg of nutrient N. There are three popular products on the market. The costs and the amounts of the two nutrients are given in the following table:*

	Product 1	Product 2	Product 3
Cost	\$27	\$31	\$24
Daily amount of *M*	0.16 mg	0.21 mg	0.11 mg
Daily amount of *N*	0.19 mg	0.13 mg	0.15 mg

Table 6.1: Data for problem.

You want to determine how much of each product you should buy so that the daily intake requirements of the two nutrients are satisfied at minimum cost. Formulate your problem as a linear programming problem, assuming that you can buy a fractional number of each product.

Selected Solutions

Solution. (Exercise **??**) In fact, constraints 2 and 3 point away from eachother, and hence there are no feasible solutions. ♠

Solution. (Exercise 6.7)

The points (x,y) satisfying $x-2y \leq 2$ are precisely those above the line passing through $(2,0)$ and $(0,-1)$. ♠

Solution. (6.10)

We want to determine the minimum value z so that $x+y=z$ defines a line that has a nonempty intersection with the feasible region. However, we can avoid referring to a sketch by setting $x=z-y$ and substituting for x in the inequalities to obtain:

$$\begin{aligned} 2(z-y)+y &\geq 4 \\ (z-y)+3y &\geq 1, \end{aligned}$$

or equivalently,

$$z \geq 2 + \frac{1}{2}y$$
$$z \geq 1 - 2y,$$

Thus, the minimum value for z is $\min\{2 + \frac{1}{2}y, 1 - 2y\}$, which occurs at $y = -\frac{2}{5}$. Hence, the optimal value is $\frac{9}{5}$.

We can verify our work by doing the following. If our calculations above are correct, then an optimal solution is given by $x = \frac{11}{5}$, $y = -\frac{2}{5}$ since $x = z - y$. It is easy to check that this satisfies both inequalities and therefore is a feasible solution.

Now, taking $\frac{2}{5}$ times the first inequality and $\frac{1}{5}$ times the second inequality, we can infer the inequality $x + y \geq \frac{9}{5}$. The left-hand side of this inequality is precisely the objective function. Hence, no feasible solution can have objective function value less than $\frac{9}{5}$. But $x = \frac{11}{5}$, $y = -\frac{2}{5}$ is a feasible solution with objective function value equal to $\frac{9}{5}$. As a result, it must be an optimal solution.

Remark. We have not yet discussed how to obtain the multipliers $\frac{2}{5}$ and $\frac{1}{5}$ for inferring the inequality $x + y \geq \frac{9}{5}$. This is an issue that will be taken up later. In the meantime, think about how one could have obtained these multipliers for this particular exercise. ♠

Solution. (Exercise 6.11) We could glean some insight by first making a sketch on the (x,y)-plane.

The line defined by $-x + y = z$ has x-intercept $-z$. Note that for $z \leq -3$, $\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} -z \\ 0 \end{bmatrix}$ satisfies both inequalities and the value of the objective function at $\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} -z \\ 0 \end{bmatrix}$ is z. Hence, there is no lower bound on the value of the objective function. ♠

Solution. (Exercise 6.12) Let x_i denote the amount of Product i to buy for $i = 1,2,3$. Then, the problem can be formulated as

$$\begin{array}{lrcrcrcl} \text{minimize} & 27x_1 & + & 31x_2 & + & 24x_3 & & \\ \text{subject to} & 0.16x_1 & + & 0.21x_2 & + & 0.11x_3 & \geq & 0.30 \\ & 0.19x_1 & + & 0.13x_2 & + & 0.15x_3 & \geq & 0.40 \\ & x_1 & , & x_2 & , & x_3 & \geq & 0. \end{array}$$

Remark. If one cannot buy fractional amounts of the products, the problem can be formulated as

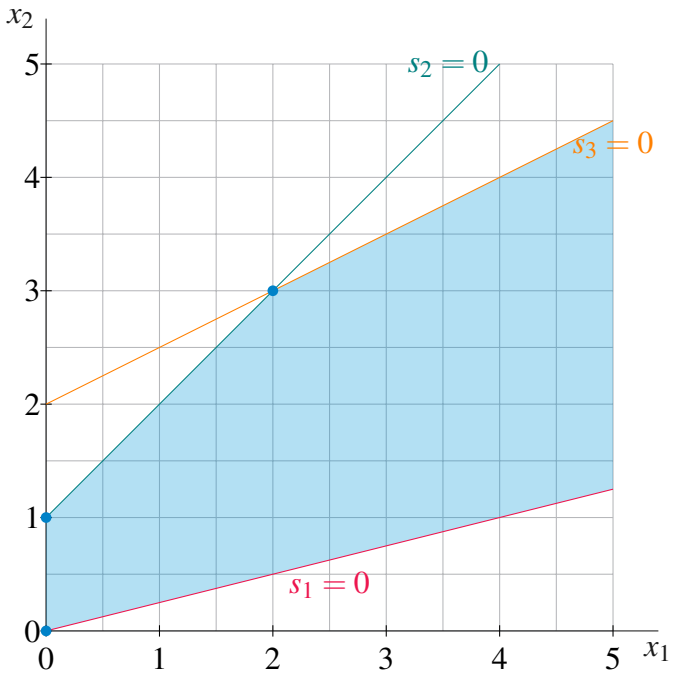
$$\begin{array}{lrcrcrcl} \text{minimize} & 27x_1 & + & 31x_2 & + & 24x_3 & & \\ \text{subject to} & 0.16x_1 & + & 0.21x_2 & + & 0.11x_3 & \geq & 0.30 \\ & 0.19x_1 & + & 0.13x_2 & + & 0.15x_3 & \geq & 0.40 \\ & x_1 & , & x_2 & , & x_3 & \geq & 0. \\ & x_1 & , & x_2 & , & x_3 & \in & \mathbb{Z}. \end{array}$$

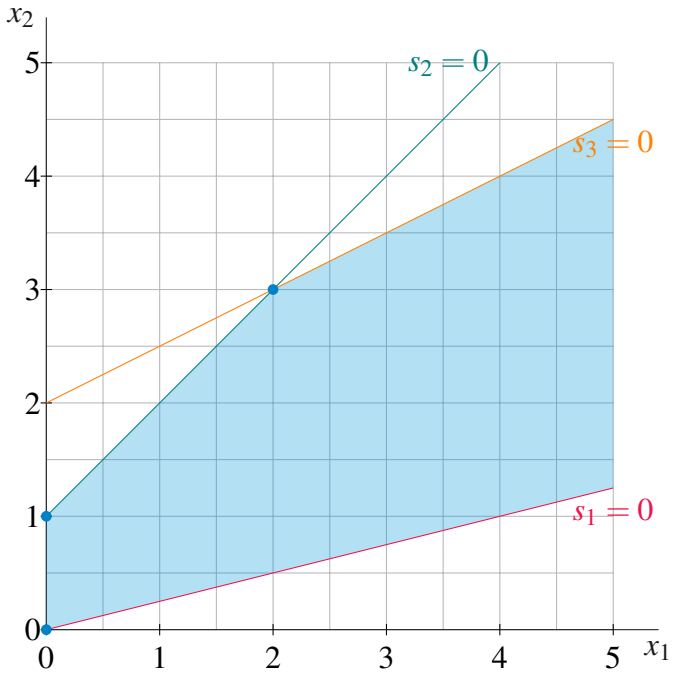
♠

6.7 Other Material: Extreme Directions

Let's define a procedure for finding the extreme directions, using the following LP's feasible region. Graphically, we can see that the extreme directions should follow the the $s_1 = 0$ (red) line and the $s_3 = 0$ (orange) line.

$$
\begin{aligned}
\max \quad & z = -5x_1 - x_2 \\
\text{s.t.} \quad & x_1 - 4x_2 + s_1 = 0 \\
& -x_1 + x_2 + s_2 = 1 \\
& -x_1 + 2x_2 + s_3 = 4 \\
& x_1, x_2, s_1, s_2, s_3 \geq 0.
\end{aligned}
$$





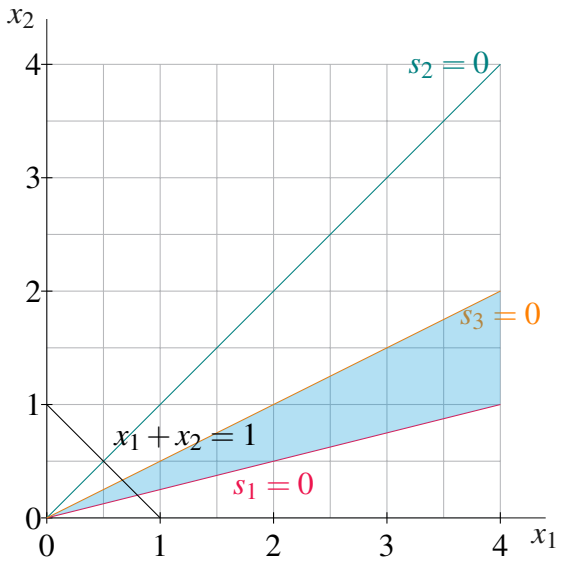
E.g., consider the $s_3 = 0$ (orange) line, to find the extreme direction start at extreme point (2,3) and find

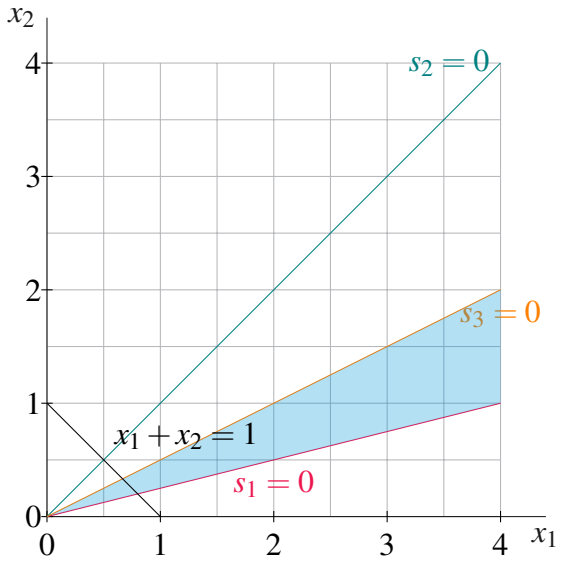
another feasible point on the orange line, say (4,4) and subtract (2,3) from (4,4), which yields (2,1).

This is related to the slope in two-dimensions, as discussed in class, the rise is 1 and the run is 2. So this direction has a slope of 1/2, but this does not carry over easily to higher dimensions where directions cannot be defined by a single number.

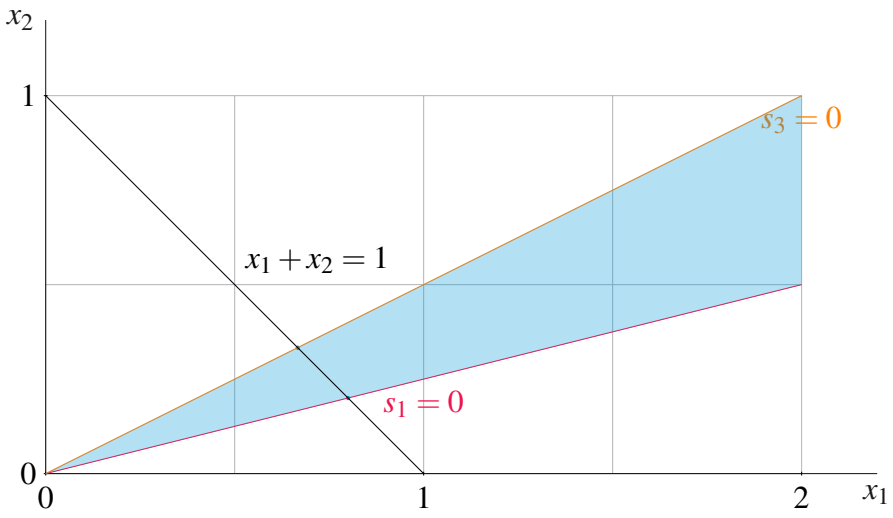
To find the extreme directions we can change the right-hand-side to $\mathbf{b} = 0$, which forms a polyhedral cone (in yellow), and then add the constraint $x_1 + x_2 = 1$. The intersection of the cone and $x_1 + x_2 = 1$ form a line segment.

$$
\begin{aligned}
\max \quad & z = -5x_1 - x_2 \\
\text{s.t.} \quad & x_1 - 4x_2 + s_1 = 0 \\
& -x_1 + x_2 + s_2 = 0 \\
& -x_1 + 2x_2 + s_3 = 0 \\
& x_1 + x_2 = 1 \\
& x_1, x_2, s_1, s_2, s_3 \geq 0.
\end{aligned}
$$





Magnifying for clarity, and removing the $s_2 = 0$ (teal) line, as it is redundant, and marking the extreme points of the new feasible region, (4/5, 1/5) and (2/3, 1/3), with red boxes, we have:



The extreme directions are thus (4/5, 1/5) and (2/3, 1/3).

7. Formal Mathematical Statements

Outcomes

- *Establish needed mathematical notation*
- *Understand structural results pertaining to linear programs*
- *Develop an intuition for why these results are true*

In this chapter, we formalize some of the mathematics about linear programs. We define some notation here and then state and prove several theorems.

CAUTION!

In this section, we provide rigirous mathematical proofs of several results. For this course, you do not need to be able to reproduce these proofs. However, try to understand the concepts and have a visual image in your head of why these results are true.

7.0.1. Vectors and Their Properties

Vectors: A vector is a mathematical object that represents a point in space or a direction. In n-dimensional real space ($\mathbb{R}^n$), a vector $\mathbf{v}$ is written as an ordered list of n elements:

$$\mathbf{v} = (v_1, v_2, \ldots, v_n).$$

Vectors can be written in two forms: as a row vector $[v_1, v_2, \ldots, v_n]$ or as a column vector:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}.$$

In linear programming, vectors are used to represent variables, constraints, and directions in optimization problems.

Vector Operations: Vector operations form the building blocks for linear programming, enabling the manipulation of decision variables and constraints.

Addition: Vectors of the same dimension can be added componentwise. For example, if $\mathbf{v} = (2,3)$ and $\mathbf{u} = (3,2)$, their sum is:

$$\mathbf{v} + \mathbf{u} = (2+3, 3+2) = (5,5).$$

This operation is essential in modeling systems where contributions of multiple variables are combined.

Scalar Multiplication: A vector can be scaled by multiplying it with a scalar (constant). For example, if $k = 2$ and $\mathbf{v} = (2,3)$, then:

$$k\mathbf{v} = (2 \cdot 2, 3 \cdot 2) = (4,6).$$

Scalar multiplication adjusts the magnitude of a vector while retaining its direction, a key concept in scaling constraints or objectives in linear programming.

Inner (Dot) Product: The dot product of two vectors $\mathbf{v}$ and $\mathbf{u}$ of the same size is calculated as:

$$\mathbf{v} \cdot \mathbf{u} = \sum_{i=1}^{n} v_i u_i.$$

For example, if $\mathbf{v} = (2,3)$ and $\mathbf{u} = (3,2)$, then:

$$\mathbf{v} \cdot \mathbf{u} = 2 \cdot 3 + 3 \cdot 2 = 10.$$

The dot product is crucial in defining constraints and objective functions in linear programming, as it represents weighted sums of variables.

7.0.2. Linear Combinations and Related Concepts

Linear Combination: A vector $\mathbf{u}$ is a linear combination of vectors $\mathbf{v}^1, \mathbf{v}^2, \ldots, \mathbf{v}^k$ if:

$$\mathbf{u} = \sum_{i=1}^{k} \lambda_i \mathbf{v}^i,$$

where $\lambda_1, \lambda_2, \ldots, \lambda_k$ are scalars. If all scalars are non-negative ($\lambda_i \geq 0$), the combination is called a *non-negative linear combination*. Linear combinations are used in linear programming to express feasible solutions as sums of variable contributions.

Convex Combination: A vector $\mathbf{u}$ is a convex combination of vectors $\mathbf{v}^1, \mathbf{v}^2, \ldots, \mathbf{v}^k$ if:

$$\mathbf{u} = \sum_{i=1}^{k} \lambda_i \mathbf{v}^i, \quad \text{where } \lambda_i \geq 0 \text{ and } \sum_{i=1}^{k} \lambda_i = 1.$$

Convex combinations are fundamental to linear programming because they describe feasible regions (e.g., convex polyhedra).

Definition 7.1: Linear Independence

A set of vectors $\{\mathbf{v}^1, \mathbf{v}^2, \ldots, \mathbf{v}^k\} \subseteq \mathbb{R}^n$ *is said to be* linearly independent *if the only solution to the equation*

$$\lambda_1 \mathbf{v}^1 + \lambda_2 \mathbf{v}^2 + \cdots + \lambda_k \mathbf{v}^k = 0$$

is $\lambda_1 = \lambda_2 = \cdots = \lambda_k = 0$.
Equivalently, no vector in the set can be written as a linear combination of the others.

Examples:

- The vectors $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ in $\mathbb{R}^2$ are linearly independent.
- The vectors $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $\begin{bmatrix} 2 \\ 4 \end{bmatrix}$ are linearly dependent, since the second is a scalar multiple of the first.

Definition 7.2: Linear Independence of Rows or Columns

Let $A \in \mathbb{R}^{m \times n}$ *be a matrix.*

- *The rows of* A *are said to be* linearly independent *if no row can be written as a linear combination of the other rows.*
- *The columns of* A *are* linearly independent *if no column can be written as a linear combination of the other columns.*
- *In either case, the vectors are linearly independent if the only solution to the homogeneous system* $A\mathbf{x} = 0$ *(or* $A^\top \mathbf{y} = 0$*) is the trivial solution* $\mathbf{x} = 0$ *(or* $\mathbf{y} = 0$*).*

Examples:

- The matrix $A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ has both linearly independent rows and columns.
- The matrix $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ has linearly dependent rows and columns; the second row is twice the first, and the second column is twice the first.

Definition 7.3: Rank of a Matrix

Let $A \in \mathbb{R}^{m\times n}$. The rank *of A, denoted* $\operatorname{rank}(A)$*, is the maximum number of linearly independent rows or columns of A.*
Equivalently, $\operatorname{rank}(A)$ *is:*

- *the dimension of the row space of A,*
- *the dimension of the column space of A,*
- *the number of pivot columns in the reduced row echelon form of A,*
- *the largest integer r such that A has an $r \times r$ submatrix with nonzero determinant.*

Examples:

- The matrix $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ has full rank 2.
- The matrix $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ has rank 1, since the second row is a multiple of the first.

Definition 7.4: Linearly Independent Constraints

Let $A\mathbf{x} \leq \mathbf{b}$ be a system of linear inequalities, where each constraint corresponds to a row $A_i\mathbf{x} \leq b_i$. A subset of these constraints is said to be linearly independent *at a point $\mathbf{x} \in \mathbb{R}^n$ if the set of row vectors $\{A_i : A_i\mathbf{x} = b_i\}$ (i.e., the tight constraints at $\mathbf{x}$) is linearly independent.*
Equivalently, the tight constraints at $\mathbf{x}$ are linearly independent if the matrix A_I formed by stacking the tight rows satisfies

$$\operatorname{rank}(A_I) = |I|,$$

where $I := \{i : A_i\mathbf{x} = b_i\}$.

Examples:

- Let $A\mathbf{x} \leq \mathbf{b}$ be the system:

$$\begin{aligned} x_1 &\leq 1 \\ x_2 &\leq 1 \\ x_1 + x_2 &\leq 2 \end{aligned}$$

  At $\mathbf{x} = (1,1)$, all three constraints are tight. The first two have row vectors $[1,0]$ and $[0,1]$, which are linearly independent. The third is $[1,1]$, which is a linear combination of the first two. So only two of the tight constraints are linearly independent.

- In contrast, for the system:

$$\begin{aligned} x_1 &\leq 0 \\ x_2 &\leq 0 \end{aligned}$$

  the point $\mathbf{x} = (0,0)$ has both constraints tight, and their rows $[1,0]$, $[0,1]$ are linearly independent. Hence the tight constraints are linearly independent at that point.

7.1 Algebraic Proof - Exists Optimial Solution at a Vertex

Definition 7.5: Vertex

Let $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}\}$ be a polyhedron. A point $\mathbf{x} \in P$ is called a vertex *(or* corner point*) of P if it is the unique solution to a system of n linearly independent constraints that are tight at $\mathbf{x}$. In other words, $\mathbf{x}$ is a vertex if there exists a subset $I \subseteq \{1,\ldots,m\}$ such that:*

- $A_i\mathbf{x} = b_i$ *for all* $i \in I$,
- *The rows* $\{A_i : i \in I\}$ *are linearly independent,*
- $|I| = n$.

Theorem 7.6

Let $\max\{\mathbf{c}^\top\mathbf{x} : A\mathbf{x} \leq \mathbf{b}\}$ *be a linear program with a nonempty, bounded feasible region P. If an optimal solution exists, then there exists an optimal solution that is a vertex of P.*

Proof. Let $\mathbf{x}^* \in P$ be an optimal solution, and suppose for contradiction that $\mathbf{x}^*$ is *not* a vertex.

Define the index set of tight (active) constraints at $\mathbf{x}^*$ as

$$I := \{i \in \{1,\ldots,m\} : A_i\mathbf{x}^* = b_i\},$$

where A_i is the ith row of A. Let A_I be the submatrix of A consisting of the rows indexed by I.

Since $\mathbf{x}^*$ is not a vertex, the tight constraints are linearly dependent: $\operatorname{rank}(A_I) < n$. Therefore, the homogeneous system $A_I\mathbf{d} = 0$ has a nonzero solution $\mathbf{d} \in \mathbb{R}^n \setminus \{0\}$.

Consider the line $\mathbf{x}(t) := \mathbf{x}^* + t\mathbf{d}$ for small $t \in \mathbb{R}$. For all $i \in I$, we have:

$$A_i\mathbf{x}(t) = A_i\mathbf{x}^* + tA_i\mathbf{d} = b_i + 0 = b_i,$$

so the tight constraints remain tight along the line.

For any $i \notin I$, we have $A_i\mathbf{x}^* < b_i$. Since $A_i\mathbf{d}$ is bounded, we can choose a small enough $\varepsilon > 0$ such that for all $t \in [-\varepsilon, \varepsilon]$, the inequality $A_i\mathbf{x}(t) \leq b_i$ holds. Thus, for small t, $\mathbf{x}(t) \in P$.

Now define the objective value along this line:

$$z(t) := \mathbf{c}^\top\mathbf{x}(t) = \mathbf{c}^\top\mathbf{x}^* + t \cdot (\mathbf{c}^\top\mathbf{d}).$$

- If $\mathbf{c}^\top\mathbf{d} > 0$, then for small $t > 0$, $\mathbf{x}(t) \in P$ and $\mathbf{c}^\top\mathbf{x}(t) > \mathbf{c}^\top\mathbf{x}^*$, contradicting optimality of $\mathbf{x}^*$.
- If $\mathbf{c}^\top\mathbf{d} < 0$, then for small $t < 0$, same contradiction.
- If $\mathbf{c}^\top\mathbf{d} = 0$, then $\mathbf{x}(t)$ is also optimal for all small t, and we can move in both directions while maintaining feasibility and optimality.

In all cases, we can find another feasible optimal point along the line $\mathbf{x}(t)$, with the same or better objective value. Moreover, we can choose such a point with a strictly larger set of linearly independent tight constraints. Repeating this process, we eventually reach an optimal point at which the tight constraints form a full-rank system (rank n), which uniquely determines the point. Such a point is, by definition, a vertex.

This contradicts our assumption that no optimal vertex exists. ♠

7.2 Convexity

A key property that will enable efficient algorithms is *convexity*. This comes in the form of *convex sets* and *convex functions*. Then the constraints to an optimization problem form a convex set and the objective function is a convex function, then we say that it is a convex optimization problem.

We explain these definitions in the next two subsections.

7.2.1. Convex Sets

Definition 7.7: Convex Combination

Given two points $\mathbf{x}, \mathbf{y}$*, a* convex combination *is any point* z *that lies on the line between* $\mathbf{x}$ *and* $\mathbf{y}$*. Algebraically, a* convex combination *is any point* $\mathbf{z}$ *that can be represented as* $\mathbf{z} = \lambda\mathbf{x} + (1-\lambda)\mathbf{y}$ *for some multiplier* $\lambda \in [0,1]$*.*

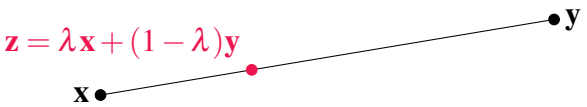


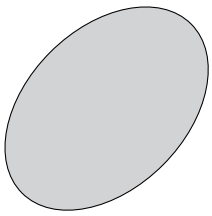
Figure 7.1: A convex combination of the points x and y is given by $\mathbf{z} = \lambda\mathbf{x} + (1-\lambda)\mathbf{y}$ with any $\lambda \in [0,1]$. Here we demonstrate this using $\lambda = 2/3$.

Definition 7.8: Convex Set

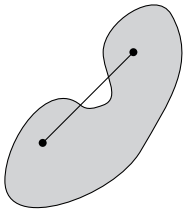
A set C *is convex if it contains all convex combinations of points in* C*. That is, for any* $\mathbf{x}, \mathbf{y} \in C$*, it holds that* $\lambda\mathbf{x} + (1-\lambda)\mathbf{y} \in C$ *for all* $\lambda \in [0,1]$*.*

Some examples of convex sets are:

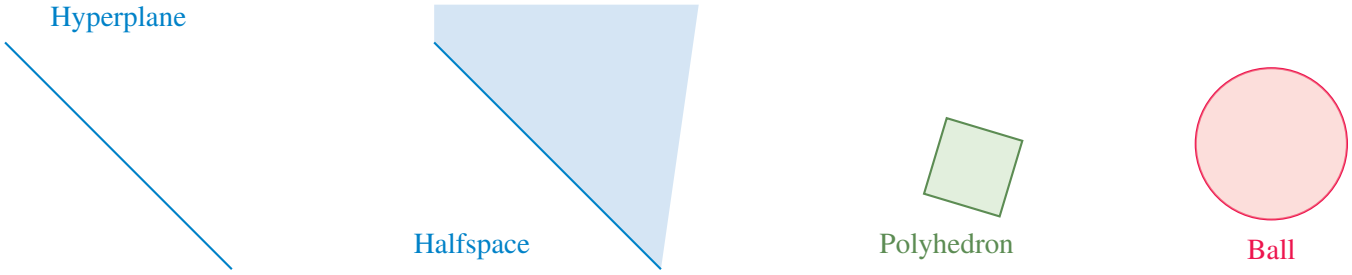
1. *Hyperplane* $H = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}^\top \mathbf{x} = b\}$
2. *Halfspace* $H = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}^\top \mathbf{x} \leq b\}$
3. *Polyhedron* $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}\}$



(a) Convex Set



(b) Non-convex Set



4. *Ball* $B = \{\mathbf{x} \in \mathbb{R}^n : \sum_{i=1}^n x_i^2 \leq 1\}$

Example 7.9: Proof that a Polyhedron is Convex

Let P be a polyhedron defined by:

$$P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}\}$$

where A is an $m \times n$ matrix and $\mathbf{b}$ is an $m \times 1$ vector.

To prove the convexity of P, we'll use the definition of convexity: For any $\mathbf{x}, \mathbf{y} \in P$ and any $\lambda \in [0,1]$, the point $\mathbf{z} = \lambda\mathbf{x} + (1-\lambda)\mathbf{y}$ must also belong to P.
Let $\mathbf{x}, \mathbf{y} \in P$. Then, by the definition of P, we have:

$$A\mathbf{x} \leq \mathbf{b} \quad \text{and} \quad A\mathbf{y} \leq \mathbf{b}$$

We want to demonstrate the inequality $A\mathbf{z} \leq \mathbf{b}$. We do this in just a few lines:

$$\begin{aligned} A\mathbf{z} &= A(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) && \text{Substituting the expression for } \mathbf{z} \\ &= \lambda A\mathbf{x} + (1-\lambda)A\mathbf{y} && \text{Expanding the matrix-vector product} \\ &\leq \lambda\mathbf{b} + (1-\lambda)\mathbf{b} && \text{Explanation below} \\ &= \mathbf{b} \end{aligned}$$

The inequality above uses the fact that $A\mathbf{x} \leq \mathbf{b}$ and $A\mathbf{y} \leq \mathbf{b}$, and since λ is in the interval $[0,1]$, it follows that:

$$\lambda A\mathbf{x} \leq \lambda\mathbf{b} \quad \text{and} \quad (1-\lambda)A\mathbf{y} \leq (1-\lambda)\mathbf{b}$$

The computation shows that $\mathbf{z} = \lambda\mathbf{x} + (1-\lambda)\mathbf{y}$ satisfies $A\mathbf{z} \leq \mathbf{b}$ and therefore $\mathbf{z} \in P$.
This shows that the polyhedron P is convex.

Lemma 7.10: Intersection of Convex Sets is Convex

Let C_1 and C_2 be convex sets. Then the intersection $C_1 \cap C_2$ is convex. In particular,

$$C_1 \cap C_2 := \{\mathbf{x} : \mathbf{x} \in C_1 \text{ and } \mathbf{x} \in C_2\}.$$

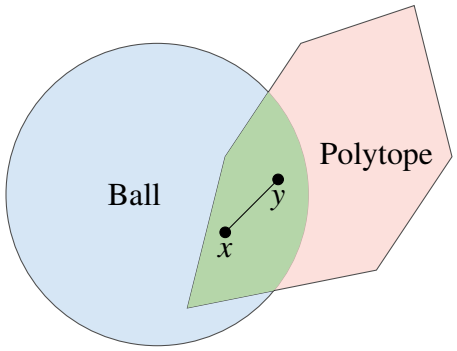


Figure 7.3: The green intersection of the convex sets that are the ball and the polytope is also convex. This can be seen by considering any points $x, y \in$ Ball $\cap$ Polytope. Since Ball is convex, the line segment between x and y is completey contained in Ball. And similarly, the line segment is completely contained in Polytope. Hence, the line segment is also contained in the intersection. This is how we can reason that the intersection is also convex.

Linear Independence: Vectors $\mathbf{v}^1, \mathbf{v}^2, \ldots, \mathbf{v}^k$ are linearly independent if the only solution to:

$$\sum_{i=1}^{k} \lambda_i \mathbf{v}^i = 0$$

is $\lambda_1 = \lambda_2 = \cdots = \lambda_k = 0$. In $\mathbb{R}^2$, two vectors are linearly dependent if they lie on the same line. Linear independence is critical in linear programming to ensure constraints are non-redundant.

7.2.2. Spanning Sets and Basis

Spanning Set: A set of vectors $\mathbf{v}^1, \mathbf{v}^2, \ldots, \mathbf{v}^k$ spans $\mathbb{R}^m$ if any vector in $\mathbb{R}^m$ can be expressed as a linear combination of these vectors:

$$\sum_{i=1}^{m} \lambda_i \mathbf{v}^i.$$

Basis: A set of vectors $\mathbf{v}^1, \mathbf{v}^2, \ldots, \mathbf{v}^m$ is a basis for $\mathbb{R}^m$ if:

- The vectors span $\mathbb{R}^m$.
- The vectors are linearly independent.

In linear programming, the concept of a basis is used to identify corner points of the feasible region.

7.2.3. Convex and Polyhedral Sets

Convex Set: A set $C \subseteq \mathbb{R}^n$ is convex if, for any two points $\mathbf{v}^1, \mathbf{v}^2 \in C$, the line segment joining them lies entirely within C:

$$\lambda \mathbf{v}^1 + (1-\lambda)\mathbf{v}^2 \in C, \quad \forall \lambda \in [0,1].$$

Convex sets form the feasible regions in linear programming problems.

Polyhedral Set: A polyhedral set is the intersection of finitely many half-spaces, described as:

$$P = \{\mathbf{x} : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq 0\}.$$

Polyhedral sets are convex and represent the feasible regions of linear programming models.

Extreme Points: An extreme point of a polyhedral set C is a point that cannot be written as a convex combination of other points in C. Linear programming solutions often occur at extreme points, as guaranteed by fundamental theorems of linear programming.

Lemma 7.11: Objective Values on a Line Segment

Given two points $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, a linear function evaluated at any point on the line segment between $\mathbf{x}$ and $\mathbf{y}$ is bounded above by the maximum of the evaluations at $\mathbf{x}$ and $\mathbf{y}$. Formally, for any $\mathbf{c} \in \mathbb{R}^n$,

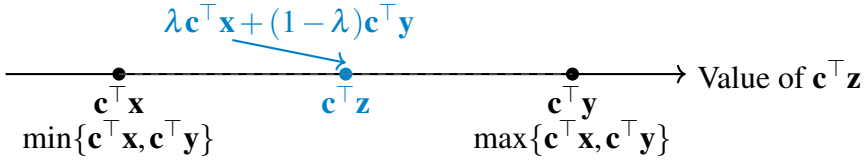
$$\min\{\mathbf{c}^\top \mathbf{x}, \mathbf{c}^\top \mathbf{y}\} \leq \mathbf{c}^\top \mathbf{z} \leq \max\{\mathbf{c}^\top \mathbf{x}, \mathbf{c}^\top \mathbf{y}\}.$$

where $\mathbf{z} = \lambda \mathbf{x} + (1-\lambda)\mathbf{y}$ for some $\lambda \in (0,1)$.
Either both of these inequalities are strict, or they are both held at equality.

Proof. Expanding $\mathbf{c}^\top \mathbf{z}$ based on the definition

$$\begin{aligned} \mathbf{c}^\top \mathbf{z} &= \mathbf{c}^\top \mathbf{z}(\lambda \mathbf{x} + (1-\lambda)\mathbf{y}) && \text{Replace forumla for } \mathbf{z} \\ &= \mathbf{c}^\top \lambda \mathbf{x} + \mathbf{c}^\top (1-\lambda)\mathbf{y} && \text{use distributivity} \\ &= \lambda \mathbf{c}^\top \mathbf{x} + (1-\lambda)\mathbf{c}^\top \mathbf{y} && \text{use linearity} \end{aligned}$$

This equation expresses $\mathbf{c}^\top \mathbf{z}$ as a convex combination of $\mathbf{c}^\top \mathbf{x}$ and $\mathbf{c}^\top \mathbf{y}$. Since $\lambda \in [0,1]$, we know $\lambda \geq 0$ and $1-\lambda \geq 0$, ensuring that the combination is convex.



By the properties of convex combinations, the value $\mathbf{c}^\top \mathbf{z}$ lies between $\mathbf{c}^\top \mathbf{x}$ and $\mathbf{c}^\top \mathbf{y}$. Formally:

$$\min\{\mathbf{c}^\top \mathbf{x}, \mathbf{c}^\top \mathbf{y}\} \leq \mathbf{c}^\top \mathbf{z} \leq \max\{\mathbf{c}^\top \mathbf{x}, \mathbf{c}^\top \mathbf{y}\}.$$

Therefore, the value of $\mathbf{c}^\top \mathbf{z}$ is bounded above by $\max\{\mathbf{c}^\top \mathbf{x}, \mathbf{c}^\top \mathbf{y}\}$, as required. ♠

Theorem 7.12: Optimal Solution at an Extreme Point

Let $C \subseteq \mathbb{R}^n$ be a bounded, closed, and convex set. Let the objective be to maximize $\mathbf{c}^\top \mathbf{x}$ over C. Then, there exists an optimal solution $\mathbf{x}^* \in C$ such that $\mathbf{x}^*$ is an extreme point of C.

Proof. Since C is bounded and closed, it is compact in $\mathbb{R}^n$. The linear function $\mathbf{c}^\top \mathbf{x}$ is continuous, and by the Extreme Value Theorem, it attains its maximum value on the compact set C. Let $\mathbf{x}^* \in C$ be such that:

$$\mathbf{c}^\top \mathbf{x}^* = \max\{\mathbf{c}^\top \mathbf{x} : \mathbf{x} \in C\}$$

Suppose $\mathbf{x}^*$ is not an extreme point of C. Then, $\mathbf{x}^*$ can be expressed as a convex combination of two distinct points $\mathbf{x}, \mathbf{y} \in C$:

$$\mathbf{x}^* = \lambda \mathbf{x} + (1-\lambda)\mathbf{y}, \quad \text{where } \lambda \in (0,1).$$

By Lemma 7.2.3, the value $\mathbf{c}^\top \mathbf{x}^*$ satisfies:

$$\min\{\mathbf{c}^\top \mathbf{x}, \mathbf{c}^\top \mathbf{y}\} \leq \mathbf{c}^\top \mathbf{x}^* \leq \max\{\mathbf{c}^\top \mathbf{x}, \mathbf{c}^\top \mathbf{y}\}.$$

Since $\mathbf{x}^*$ maximizes $\mathbf{c}^\top \mathbf{x}$ over C, it must be that $\mathbf{c}^\top \mathbf{x} = \mathbf{c}^\top \mathbf{x}^*$ and $\mathbf{c}^\top \mathbf{y} = \mathbf{c}^\top \mathbf{x}^*$. Thus, both $\mathbf{x}$ and $\mathbf{y}$ also maximize $\mathbf{c}^\top \mathbf{x}$ over C.

If $\mathbf{x}^*$ is not an extreme point, we can repeat this argument for $\mathbf{x}$ and $\mathbf{y}$, expressing them as convex combinations of other points in C. However, since C is compact, this process must terminate after a finite number of steps, at which point we reach an extreme point that maximizes $\mathbf{c}^\top \mathbf{x}$.

Hence, there exists an optimal solution at an extreme point of C. ♠

Theorem 7.13: Extreme Points and Vertices of a Polyhedron

Let $P \subseteq \mathbb{R}^n$ be a polyhedron. Then, a point $\mathbf{v} \in P$ is an extreme point if and only if $\mathbf{v}$ is a vertex of P.

Proof. (If v is not a vertex, then it is not an extreme point):

Suppose $\mathbf{v} \in P$ is not a vertex. By definition, this means $\mathbf{v}$ is not the unique solution to any subset of n linearly independent constraints active at $\mathbf{v}$. Let $A'\mathbf{x} = \mathbf{b}'$ represent the active constraints at $\mathbf{v}$, where $A' \in \mathbb{R}^{m' \times n}$ ($m' \leq n$) is the matrix of active constraints, and assume the rows of A' are linearly independent. Since $\mathbf{v}$ is not a vertex, the null space of A', denoted $\mathcal{N}(A')$, is nontrivial, i.e., there exists a nonzero direction $\mathbf{d} \in \mathbb{R}^n$ such that:

$$A'\mathbf{d} = 0.$$

Consider perturbing $\mathbf{v}$ in the directions $\mathbf{v}+\varepsilon\mathbf{d}$ and $\mathbf{v}-\varepsilon\mathbf{d}$, for some small $\varepsilon > 0$. Since $\mathbf{d} \in \mathcal{N}(A')$, the perturbed points satisfy the active constraints:

$$A'(\mathbf{v}+\varepsilon\mathbf{d}) = A'\mathbf{v}+\varepsilon A'\mathbf{d} = \mathbf{b}'.$$

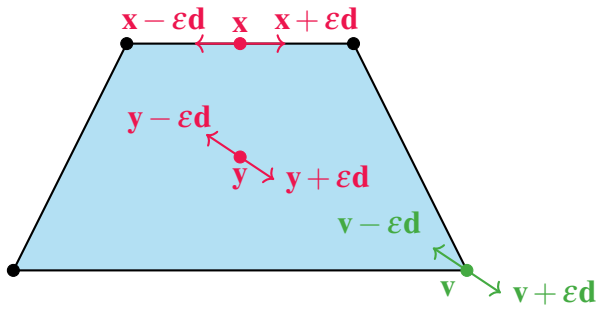
Similarly, $A'(\mathbf{v}-\varepsilon\mathbf{d}) = \mathbf{b}'$.

Furthermore, for sufficiently small $\varepsilon > 0$, the perturbed points also satisfy all other constraints defining P. This is because P is a polyhedron, and the constraints are continuous linear inequalities that remain satisfied under small perturbations of $\mathbf{v}$.

Now, we express $\mathbf{v}$ as a strict convex combination of $\mathbf{v}+\varepsilon\mathbf{d}$ and $\mathbf{v}-\varepsilon\mathbf{d}$:

$$\mathbf{v} = \frac{1}{2}(\mathbf{v}+\varepsilon\mathbf{d}) + \frac{1}{2}(\mathbf{v}-\varepsilon\mathbf{d}).$$

Since $\mathbf{v}+\varepsilon\mathbf{d} \neq \mathbf{v}$ and $\mathbf{v}-\varepsilon\mathbf{d} \neq \mathbf{v}$, and both points belong to P, this shows that $\mathbf{v}$ is not an extreme point of P.



(If v is a vertex, then it is an extreme point):

Assume $\mathbf{v} \in P$ is a vertex. By definition, there exists a subset of constraints, say $A'\mathbf{x} = \mathbf{b}'$, where $A' \in \mathbb{R}^{n\times n}$, such that $\mathbf{v}$ is the unique solution to this system.

Suppose, for the sake of contradiction, that $\mathbf{v}$ is not an extreme point. Then, $\mathbf{v}$ can be expressed as:

$$\mathbf{v} = \lambda\mathbf{x}_1 + (1-\lambda)\mathbf{x}_2, \quad \mathbf{x}_1, \mathbf{x}_2 \in P, \lambda \in (0,1),$$

where $\mathbf{x}_1 \neq \mathbf{v}$ and $\mathbf{x}_2 \neq \mathbf{v}$.

We claim that both $\mathbf{x}_1$ and $\mathbf{x}_2$ must satisfy $A'\mathbf{x} = \mathbf{b}'$. To see this, let $\mathbf{a}$ be a row of A' and b be the corresponding right hand side value in $\mathbf{b}'$ By Lemma **??**, the value of $\mathbf{a}^\top\mathbf{z}$ satisfies:

$$\min\{\mathbf{a}^\top\mathbf{x}_1, \mathbf{a}^\top\mathbf{x}_2\} \leq \mathbf{a}^\top\mathbf{v} \leq \max\{\mathbf{a}^\top\mathbf{x}_1, \mathbf{a}^\top\mathbf{x}_2\}.$$

Since $\mathbf{v}$ satisfies $\mathbf{a}^\top\mathbf{v} = b$, this implies:

$$\mathbf{a}^\top\mathbf{v} = \lambda\mathbf{a}^\top\mathbf{x}_1 + (1-\lambda)\mathbf{a}^\top\mathbf{x}_2 = b.$$

If either $\mathbf{a}^\top\mathbf{x}_1 \neq b$ or $\mathbf{a}^\top\mathbf{x}_2 \neq b$, the equation above cannot hold. Hence, both $\mathbf{x}_1$ and $\mathbf{x}_2$ must satisfy $\mathbf{a}^\top\mathbf{x} = b$.

Since $\mathbf{v}$ satisfies $A'\mathbf{x} = \mathbf{b}'$, both $\mathbf{x}_1$ and $\mathbf{x}_2$ must also satisfy $A'\mathbf{x} = \mathbf{b}'$, as $A'\mathbf{x} = \mathbf{b}'$ defines an affine subspace. However, this contradicts the assumption that $\mathbf{v}$ is the unique solution to $A'\mathbf{x} = \mathbf{b}'$.

Therefore, $\mathbf{v}$ cannot be expressed as a strict convex combination of distinct points in P, and $\mathbf{v}$ is an extreme point.

Conclusion: A point $\mathbf{v} \in P$ is an extreme point if and only if it is a vertex of P. ♠

7.2.4. Unbounded Sets and Rays

Rays and Directions: A ray in $\mathbb{R}^n$ is a set of points:

$$\{\mathbf{x} : \bar{\mathbf{x}} + \lambda \mathbf{d}, \lambda \geq 0\},$$

where $\bar{\mathbf{x}}$ is the starting point and $\mathbf{d}$ is the direction. Rays describe unbounded feasible regions in linear programming models.

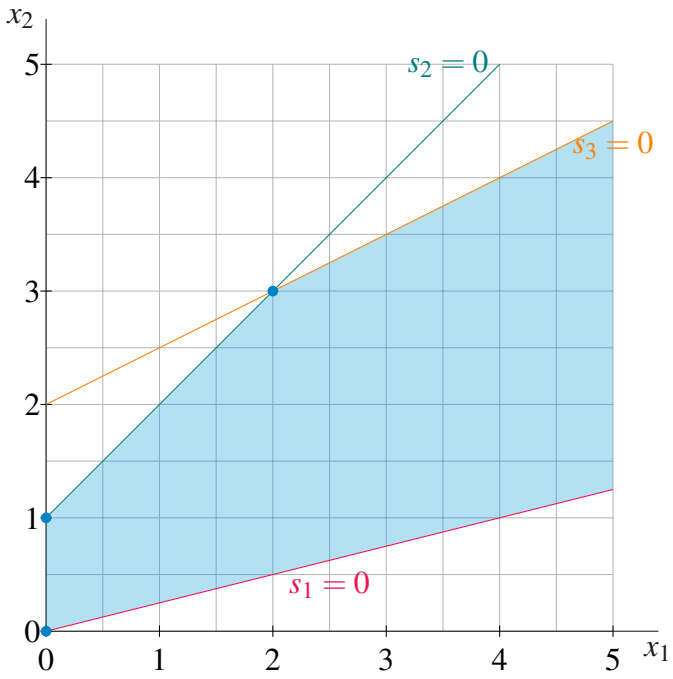
Extreme Directions: An extreme direction is one that cannot be written as a positive combination of other directions. In linear programming, extreme directions help characterize unbounded solutions.

This section provides the mathematical tools essential for understanding and formulating linear programming problems, emphasizing the geometric and algebraic structures that define feasible regions and optimal solutions.

Theorem 7.14: Representation Theorem

Let $\mathbf{x}^1, \mathbf{x}^2, \cdots \mathbf{x}^k$ be the set of extreme points of P, and if P is unbounded, $\mathbf{d}^1, \mathbf{d}^2, \cdots \mathbf{d}^l$ be the set of extreme directions. Then any $\mathbf{x} \in P$ is equal to a convex combination of the extreme points and a non-negative linear combination of the extreme directions: $\mathbf{x} = \sum_{j=1}^{k} \lambda_j \mathbf{x}^j + \sum_{j=1}^{l} \mu_j \mathbf{d}^j$, where $\sum_{j=1}^{k} \lambda_j = 1$, $\lambda_j \geq 0$, $\forall j = 1, 2, \cdots, k$, and $\mu_j \geq 0$, $\forall j = 1, 2, \cdots, l$.

$$
\begin{aligned}
\max \quad & z = -5x_1 - x_2 \\
\text{s.t.} \quad & x_1 - 4x_2 + s_1 = 0 \\
& -x_1 + x_2 + s_2 = 1 \\
& -x_1 + 2x_2 + s_3 = 4 \\
& x_1, x_2, s_1, s_2, s_3 \geq 0.
\end{aligned}
$$



Represent point (1/2, 1) as a convex combination of the extreme points of the above LP. Find λs to solve the following system of equations:

$$
\lambda_1 \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \lambda_2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} + \lambda_3 \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 1/2 \\ 1 \end{bmatrix}
$$

7.3 Exercises

Exercise 7.1 *Vector Operations Let* $\mathbf{v} = [3, -2, 4]$ *and* $\mathbf{u} = [-1, 0, 2]$. *Compute:*

1. $\mathbf{v} + \mathbf{u}$
2. $2\mathbf{v}$
3. $\mathbf{v} \cdot \mathbf{u}$

Exercise 7.2 *Linear and Convex Combinations Let* $\mathbf{v}^1 = [1, 0]$, $\mathbf{v}^2 = [0, 1]$, *and* $\mathbf{v}^3 = [1, 1]$.

1. *Which of the following are linear combinations of* $\mathbf{v}^1$ *and* $\mathbf{v}^2$*?*
   (a) $[2, 3]$
   (b) $[1, 1]$
   (c) $[1, -1]$

2. *Is the vector* $[1/2, 1/2]$ *a convex combination of any two of these vectors?*

Exercise 7.3 *Linear Independence and Matrix Rank Consider the matrix* $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$.

1. *Are the rows of A linearly independent?*
2. *Are the columns linearly independent?*
3. *What is* $\text{rank}(A)$*?*

Exercise 7.4 *Convex Combinations and Geometry Let P be the triangle with vertices at* $(0,0)$*,* $(2,0)$*, and* $(0,2)$.

1. *Plot the set P.*
2. *Determine whether the point* $(1,1)$ *is a convex combination of the vertices.*
3. *Determine whether the point* $(1.5, 1.5)$ *is in P.*

Exercise 7.5 *Halfspace Geometry and Convexity Consider the halfspace* $H = \{\mathbf{x} \in \mathbb{R}^2 : x_1 + 2x_2 \leq 4\}$.

1. *Plot the halfspace and the boundary.*
2. *Is the point* $(2,1)$ *in H? What about* $(1,2)$*?*
3. *Verify that the set is convex by checking whether the midpoint between* $(0,0)$ *and* $(2,1)$ *lies in H.*

8. Simplex Method

Outcomes

- *Convert linear programs to standard form*
- *Reformulate linear programs from the perspective of different bases*
- *Define* basic solutions *and* basic feasible solutions
- *Describe optimality condition from the perspective of a reformulation*
- *Develop the* Simplex Method *that* pivots *through basic feasible solutions*

In this section, we will develop the *Simplex Method*. This algortim has been lauded as one of the most important algorithms of the 20th century. It will iteratively find better vertex solutions to a linear program until it can prove optimality.

To implement the simplex algorithm, we must first discuss a *Standard Form* with which to represent linear programs. After, we will discuss structure of linear programs in this form using matrix notation and demonstrate how a linear program can be transformed to new representations. These new representations will be the backbone of the simplex algorithm. We will *pivot* between vertices, and each vertex will have a new representation of the linear program that will help us understand how to choose a new vertex to move to.

The whole algorithm runs very fast in practice. Here, we will teach you a basic version of the algorithm, but note that state-of-the art solvers will employ a number of tricks to make this process very fast. Linear programs can be solved with millions of variables!

8.1 Standard Form

To solve a linear program using methods such as the simplex algorithm, the problem must first be converted to a standard form. This ensures the problem adheres to a uniform structure, making it easier to analyze and solve algorithmically.

A linear program is in *standard form* if it satisfies all of the following:

- The objective is a maximization.
- All constraints are equalities (i.e., of the form $=$).
- All variables are bounded below by zero (i.e., $x_i \geq 0$).

Definition 8.1: Standard Form

A linear program is in standard form *if is written as*

$$\begin{aligned} \max \ & \mathbf{c}^\top \mathbf{x} \\ s.t. \ & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq 0. \end{aligned}$$

8.1.1. Converting to Standard Form

We now describe how to handle each of these aspects step-by-step, including common subcases and examples.

1. Objective: Convert Minimization to Maximization

If the linear program is given as a minimization, we convert it into a maximization by multiplying the objective function by -1. This transformation does not affect the feasible region, and the maximizer of the new function will be the minimizer of the original.

Example 8.2: Convert Min to Max

Consider the following objective:

$$\min \ 3x_1 + 4x_2.$$

To convert this to a maximization, we multiply the entire expression by -1*:*

$$\max \ (-3x_1 - 4x_2).$$

Now the problem can be treated using standard methods that assume a maximization objective.

2. Constraints: Convert Inequalities to Equalities

All inequality constraints must be transformed into equalities using slack or surplus variables, which represent the difference between the left- and right-hand sides of the constraint.

(a) **Convert $\leq$ to $=$ using a slack variable.**

A slack variable is added to "fill the gap" in a $\leq$ constraint and must be nonnegative.

Example 8.3: Slack Variable for $\leq$

Suppose we have the constraint:

$$x_1 + 2x_2 \leq 5.$$

We introduce a new variable $s_1 \geq 0$, called a slack variable, and rewrite the constraint as:

$$x_1 + 2x_2 + s_1 = 5.$$

Here, s_1 represents the unused portion of the right-hand side (i.e., how far the left-hand side is from 5).

(b) **Convert $\geq$ to $=$ using a surplus variable.**

A surplus variable is subtracted from the left-hand side of a $\geq$ constraint to convert it into an equality. This variable also must be nonnegative.

Example 8.4: Surplus Variable for $\geq$

Consider the constraint:

$$x_1 - x_2 \geq 3.$$

We introduce a new variable $s_2 \geq 0$, called a surplus variable, and rewrite the constraint as:

$$x_1 - x_2 - s_2 = 3.$$

In this case, s_2 accounts for the extra amount by which the left-hand side exceeds 3.

3. Variable Bounds: Ensure All Variables Are Nonnegative

The standard form requires all variables to satisfy $x_i \geq 0$. If any variable is unbounded below or is restricted in another way, we perform a variable substitution to enforce this condition.

(a) **Convert $x_i \leq 0$ to $x_i' \geq 0$ by renaming and negating.**

If a variable is constrained to be nonpositive, we can define a new nonnegative variable as its negation.

Example 8.5: Rename and Negate for ≤ 0

Suppose $x_3 \leq 0$. Define $x_3' = -x_3$, which implies $x_3' \geq 0$. Then substitute:

$$x_3 = -x_3', \quad \text{where } x_3' \geq 0.$$

This transformation allows us to enforce nonnegativity in the standard form.

(b) **Convert unrestricted variables into the difference of two nonnegative variables.**

If a variable is unrestricted in sign (i.e., it can be positive or negative), we express it as the difference of two new nonnegative variables.

Example 8.6: Unrestricted Variable to Two Variables

Suppose x_4 is unrestricted. We define:

$$x_4 = x_4^+ - x_4^-, \quad \text{with } x_4^+, x_4^- \geq 0.$$

Here, x_4^+ captures the positive part and x_4^- captures the negative part. At most one of them will be nonzero in a basic feasible solution.

After performing these transformations, the resulting linear program will be in standard form and ready for solution via the simplex method or other standard techniques.

Example 8.7: Complete Conversion to Standard Form

Consider the following linear program:

$$\begin{aligned} \min \quad & 2x_1 - 3x_2 + x_3 \\ \text{subject to} \quad & x_1 - x_2 \leq 4, \\ & 2x_1 + x_3 = 7, \\ & x_2 - x_3 \geq 2, \\ & x_1 \text{ is unrestricted}, \quad x_2 \geq 0, \quad x_3 \text{ is unrestricted}. \end{aligned}$$

We will now convert this problem into standard form.

Step 1: Convert the objective to maximization.
Since the problem is a minimization, we multiply the objective by -1*:*

$$\max\ (-2x_1 + 3x_2 - x_3).$$

Step 2: Replace unrestricted variables.
Both x_1 *and* x_3 *are unrestricted. We express them as the difference of two nonnegative variables:*

$$x_1 = x_1^+ - x_1^-, \quad x_3 = x_3^+ - x_3^-, \quad \text{with } x_1^+, x_1^-, x_3^+, x_3^- \geq 0.$$

Substitute these into the objective and constraints.

New objective:

$$\max\ [-2(x_1^+ - x_1^-) + 3x_2 - (x_3^+ - x_3^-)] = \max\ (-2x_1^+ + 2x_1^- + 3x_2 - x_3^+ + x_3^-).$$

New constraints:

$$\begin{aligned} (x_1^+ - x_1^-) - x_2 &\leq 4 && \textit{(original 1st constraint)} \\ 2(x_1^+ - x_1^-) + (x_3^+ - x_3^-) &= 7 && \textit{(original 2nd constraint)} \\ x_2 - (x_3^+ - x_3^-) &\geq 2 && \textit{(original 3rd constraint)} \end{aligned}$$

Step 3: Convert inequalities to equalities.

- *Add a slack variable* $s_1 \geq 0$ *to the first constraint:*

$$x_1^+ - x_1^- - x_2 + s_1 = 4$$

- *The second constraint is already an equality:*

$$2x_1^+ - 2x_1^- + x_3^+ - x_3^- = 7$$

- *Subtract a surplus variable* $s_2 \geq 0$ *from the third constraint:*

$$x_2 - x_3^+ + x_3^- - s_2 = 2$$

Final standard form:

$$\begin{aligned} \max \quad & -2x_1^+ + 2x_1^- + 3x_2 - x_3^+ + x_3^- \\ \textit{subject to} \quad & x_1^+ - x_1^- - x_2 + s_1 = 4 \\ & 2x_1^+ - 2x_1^- + x_3^+ - x_3^- = 7 \\ & x_2 - x_3^+ + x_3^- - s_2 = 2 \\ & x_1^+, x_1^-, x_2, x_3^+, x_3^-, s_1, s_2 \geq 0. \end{aligned}$$

This is now in standard form: a maximization problem with equality constraints and all variables nonnegative.

Remark 8.8: Variable Transformations Preserve Solution Meaning

To convert a linear program into standard form, we often introduce new variables:

- *Slack variables to convert $\leq$ constraints to equalities,*
- *Surplus variables for $\geq$ constraints,*
- *Split unrestricted variables as $x_j = x_j^+ - x_j^-$,*
- *Negate variables with $x_j \leq 0$.*

These transformations may change the number of variables and the way solutions are expressed, but:

1. *Every feasible solution to the transformed problem corresponds to a feasible solution of the original problem.*
2. *Every optimal solution of the transformed problem yields an optimal solution to the original.*

Thus, while the form *of the solution may look different, the* meaning *and the* value *of the solution are preserved.*

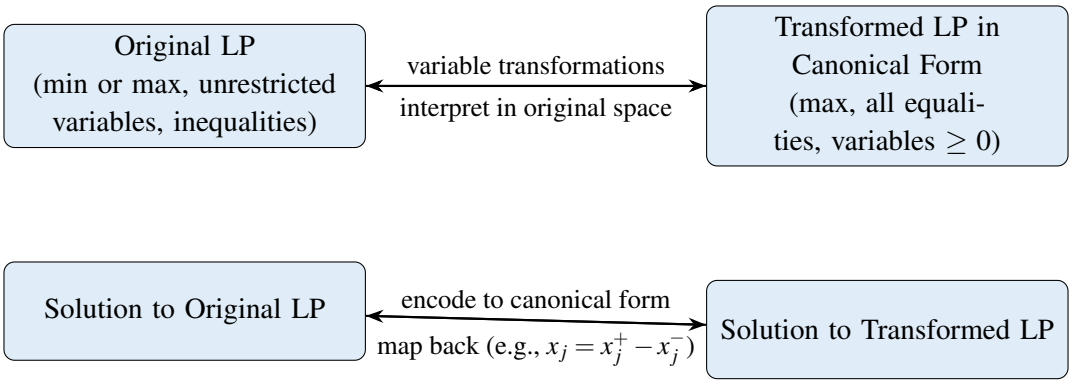


Figure 8.1: Correspondence between original and canonical form solutions.

Example 8.9: Transformation Preserves Solution Meaning

Consider the following linear program:

$$\begin{aligned} \min \quad & -x_1 + 2x_2 - 3x_3 \\ \textit{subject to} \quad & x_1 - x_2 + x_3 \geq 2, \\ & x_2 \leq 0, \\ & x_3 \textit{ unrestricted.} \end{aligned}$$

Step 1: Convert to maximization.

Multiply the objective by -1:

$$\max \quad x_1 - 2x_2 + 3x_3.$$

Step 2: Convert variables to be nonnegative.

- *$x_2 \leq 0 \rightsquigarrow$ define $x_2 = -x_2'$, with $x_2' \geq 0$.*
- *x_3 unrestricted $\rightsquigarrow$ write $x_3 = x_3^+ - x_3^-$, with $x_3^+, x_3^- \geq 0$.*

Substitute into the objective and constraint:

$$\max\ x_1 - 2(-x_2') + 3(x_3^+ - x_3^-) = x_1 + 2x_2' + 3x_3^+ - 3x_3^-.$$

Constraint: $x_1 + x_2' + x_3^+ - x_3^- \geq 2.$

Step 3: Convert constraint to equality using a surplus variable.
Introduce surplus variable $s_1 \geq 0$*:*

$$x_1 + x_2' + x_3^+ - x_3^- - s_1 = 2.$$

Now the problem is in standard form:

$$\begin{aligned} \max \quad & x_1 + 2x_2' + 3x_3^+ - 3x_3^- \\ \textit{subject to} \quad & x_1 + x_2' + x_3^+ - x_3^- - s_1 = 2, \\ & x_1, x_2', x_3^+, x_3^-, s_1 \geq 0. \end{aligned}$$

Example 8.10: Example Continued

Step 4: Example solution in standard form.
Consider a solution of the canonical form

$$x_1 = 1, \quad x_2' = 0, \quad x_3^+ = 1, \quad x_3^- = 0, \quad s_1 = 0.$$

Step 5: Map solution back to original variables.

$$x_2 = -x_2' = 0, \quad x_3 = x_3^+ - x_3^- = 1.$$

So the solution to the original problem is:

$$x_1 = 1, \quad x_2 = 0, \quad x_3 = 1,$$

and the original objective value is:

$$-1(1) + 2(0) - 3(1) = -4.$$

Conclusion: *The transformation allowed us to solve the LP using standard techniques, and we were able to directly interpret the solution in the space of the original variables. The solution meaning is preserved.*

8.1.2. Exercises: Converting to Standard Form

Exercise 8.1 *Convert the following linear program to standard form:*

$$\begin{aligned} \max \quad & 3x_1 - 5x_2 \\ \textit{subject to} \quad & 2x_1 + x_2 \leq 6, \\ & -x_1 + 3x_2 \geq 2, \\ & x_1, x_2 \geq 0. \end{aligned}$$

Exercise 8.2 *Convert the following linear program to standard form:*

$$\begin{aligned} \min \quad & -x_1 + 2x_2 + x_3 \\ \textit{subject to} \quad & x_1 - x_2 + 2x_3 = 4, \\ & -2x_1 + x_2 \leq 1, \\ & x_3 \textit{ is unrestricted}, \quad x_1, x_2 \geq 0. \end{aligned}$$

Exercise 8.3 *Convert the following linear program to standard form:*

$$\begin{aligned} \max \quad & 5x_1 + x_2 - 3x_3 \\ \textit{subject to} \quad & x_1 + 2x_2 - x_3 \geq 7, \\ & x_2 - 4x_3 = 3, \\ & x_1 \textit{ is unrestricted}, \quad x_2, x_3 \leq 0. \end{aligned}$$

Exercise 8.4 *Convert the following linear program to standard form:*

$$\begin{aligned} \min \quad & 2x_1 + 4x_2 \\ \textit{subject to} \quad & x_1 + x_2 \geq 3, \\ & 2x_1 - x_2 \leq 5, \\ & x_1 \textit{ is unrestricted}, \quad x_2 \geq 0. \end{aligned}$$

Exercise 8.5 *Convert the following linear program to standard form:*

$$\begin{aligned} \min \quad & 5y_1 - 3y_2 + y_3 \\ \textit{subject to} \quad & 2y_1 - y_2 + 3y_3 = 7, \\ & -y_1 + 4y_2 \leq 5, \\ & y_3 \textit{ is unrestricted}. \end{aligned}$$

Selected Solutions

Solution. (Exercise 8.4)

1. Convert the minimization problem to a maximization problem by negating the objective function:

$$\max \quad -2x_1 - 4x_2.$$

2. Convert the $\geq$ constraint into an equality by introducing a surplus variable $s_1 \geq 0$:

$$x_1 + x_2 - s_1 = 3.$$

3. Convert the $\leq$ constraint into an equality by introducing a slack variable $s_2 \geq 0$:

$$2x_1 - x_2 + s_2 = 5.$$

4. Since x_1 is unrestricted, replace it with $x_1 = x_1' - x_1''$, where $x_1', x_1'' \geq 0$.

Final Standard Form:

$$\begin{aligned} \max \quad & -2(x_1' - x_1'') - 4x_2 \\ \text{subject to} \quad & (x_1' - x_1'') + x_2 - s_1 = 3, \\ & 2(x_1' - x_1'') - x_2 + s_2 = 5, \\ & x_1', x_1'', x_2, s_1, s_2 \geq 0. \end{aligned}$$

♠

Solution. (Exercise 8.5)

1. Convert the minimization problem to a maximization problem by negating the objective function:

$$\max \quad -5y_1 + 3y_2 - y_3.$$

2. Convert the $\leq$ constraint into an equality by introducing a slack variable $s_1 \geq 0$:

$$-y_1 + 4y_2 + s_1 = 5.$$

3. Since y_3 is unrestricted, replace it with $y_3 = y_3' - y_3''$, where $y_3', y_3'' \geq 0$.

Final Standard Form:

$$\begin{aligned} \max \quad & -5y_1 + 3y_2 - (y_3' - y_3'') \\ \text{subject to} \quad & 2y_1 - y_2 + 3(y_3' - y_3'') = 7, \\ & -y_1 + 4y_2 + s_1 = 5, \\ & y_1, y_2, y_3', y_3'', s_1 \geq 0. \end{aligned}$$

♠